```
`include "zeroriscy_config.sv"

import zeroriscy_defines::*;

module zeroriscy_alu
(
  input  logic [ALU_OP_WIDTH-1:0]  operator_i,
  input  logic [31:0]              operand_a_i,
  input  logic [31:0]              operand_b_i,

  input  logic [32:0]              multdiv_operand_a_i,
  input  logic [32:0]              multdiv_operand_b_i,

  input  logic                     multdiv_en_i,

  output logic [31:0]              adder_result_o,
  output logic [33:0]              adder_result_ext_o,

  output logic [31:0]              result_o,
  output logic                     comparison_result_o,
  output logic                     is_equal_result_o
);

  logic [31:0] operand_a_rev;
  logic [32:0] operand_b_neg;

  // bit reverse operand_a for left shifts and bit counting
  generate
    genvar k;
    for(k = 0; k < 32; k++)
    begin : g_revloop
      assign operand_a_rev[k] = operand_a_i[31-k];
    end
  endgenerate


  ////////////////////////////////////
  //      _      _    _           //
  //     / \   __| | __| | ___ _ __  //
  //    / _ \ / _` |/ _` |/ _ \ '__| //
  //   / ___ \ (_| | (_| |  __/ |    //
  //  /_/   \_\__,_|\__,_|\___|_|    //
  //                                 //
  ////////////////////////////////////

  logic       adder_op_b_negate;
  logic [32:0] adder_in_a, adder_in_b;
  logic [31:0] adder_result;

  always_comb
  begin
    adder_op_b_negate = 1'b0;

    unique case (operator_i)
      // Adder OPs
      ALU_SUB,
```

```
    // Comparator OPs
    ALU_EQ,    ALU_NE,
    ALU_GTU,   ALU_GEU,
    ALU_LTU,   ALU_LEU,
    ALU_GTS,   ALU_GES,
    ALU_LTS,   ALU_LES,
    ALU_SLTS,  ALU_SLTU,
    ALU_SLETS, ALU_SLETU: adder_op_b_negate = 1'b1;

    default: ;
  endcase
end

// prepare operand a
assign adder_in_a    = multdiv_en_i ? multdiv_operand_a_i : {operand_a_i,1'b1};

// prepare operand b
assign operand_b_neg = {operand_b_i,1'b0} ^ {33{adder_op_b_negate}};
assign adder_in_b    = multdiv_en_i ? multdiv_operand_b_i : operand_b_neg ;

// actual adder
assign adder_result_ext_o = $unsigned(adder_in_a) + $unsigned(adder_in_b);

assign adder_result      = adder_result_ext_o[32:1];

assign adder_result_o     = adder_result;

////////////////////////////////////////
//  ____  _   _ ___ _____ _____       //
// / ___|| | | |_ _|  ___|_   _|      //
// \___ \| |_| || || |_    | |       //
//  ___) |  _  || ||  _|   | |       //
// |____/|_| |_|___|_|     |_|       //
//                                    //
////////////////////////////////////////

logic       shift_left;        // should we shift left
logic       shift_arithmetic;

logic [31:0] shift_amt;         // amount of shift, to the right
logic [31:0] shift_op_a;        // input of the shifter
logic [31:0] shift_result;
logic [31:0] shift_right_result;
logic [31:0] shift_left_result;


assign shift_amt = operand_b_i;


assign shift_left = (operator_i == ALU_SLL);

assign shift_arithmetic = (operator_i == ALU_SRA);

// choose the bit reversed or the normal input for shift operand a
assign shift_op_a    = shift_left ? operand_a_rev : operand_a_i;

// right shifts, we let the synthesizer optimize this
```

```
logic [32:0] shift_op_a_32;

assign shift_op_a_32 = { shift_arithmetic & shift_op_a[31], shift_op_a};

assign shift_right_result = $signed(shift_op_a_32) >>> shift_amt[4:0];

// bit reverse the shift_right_result for left shifts
genvar      j;
generate
  for(j = 0; j < 32; j++)
  begin : g_resrevloop
    assign shift_left_result[j] = shift_right_result[31-j];
  end
endgenerate

assign shift_result = shift_left ? shift_left_result : shift_right_result;


/////////////////////////////////////////////////////////////////
//   ____ ___  __  __ ____   _    ____  ___ ____   ___  _   _  //
//  / ___/ _ \|  \/  |  _ \ / \  |  _ \|_ _/ ___| / _ \| \ | |  //
// | |  | | | | |\/| | |_) / _ \ | |_) || |\___ \| | | |  \| |  //
// | |__| |_| | |  | |  __/ ___ \|  _ < | | ___) | |_| | |\  |  //
//  \____\___/|_|  |_|_| /_/   \_\_| \_\___|____/ \___/|_| \_|  //
//                                                             //
/////////////////////////////////////////////////////////////////

logic is_equal;
logic is_greater_equal;  // handles both signed and unsigned forms
logic cmp_signed;

always_comb
begin
  cmp_signed = 1'b0;

  unique case (operator_i)
    ALU_GTS,
    ALU_GES,
    ALU_LTS,
    ALU_LES,
    ALU_SLTS,
    ALU_SLETS: begin
      cmp_signed = 1'b1;
    end

    default:;
  endcase
end

assign is_equal = (adder_result == 32'b0);
assign is_equal_result_o = is_equal;


// Is greater equal
always_comb
begin
  if ((operand_a_i[31] ^ operand_b_i[31]) == 0)
```

```
    is_greater_equal = (adder_result[31] == 0);
  else
    is_greater_equal = operand_a_i[31] ^ (cmp_signed);
end

// GTE unsigned:
// (a[31] == 1 && b[31] == 1) => adder_result[31] == 0
// (a[31] == 0 && b[31] == 0) => adder_result[31] == 0
// (a[31] == 1 && b[31] == 0) => 1
// (a[31] == 0 && b[31] == 1) => 0

// GTE signed:
// (a[31] == 1 && b[31] == 1) => adder_result[31] == 0
// (a[31] == 0 && b[31] == 0) => adder_result[31] == 0
// (a[31] == 1 && b[31] == 0) => 0
// (a[31] == 0 && b[31] == 1) => 1



// generate comparison result
logic cmp_result;

always_comb
begin
  cmp_result = is_equal;

  unique case (operator_i)
    ALU_EQ:          cmp_result = is_equal;
    ALU_NE:          cmp_result = (~is_equal);
    ALU_GTS, ALU_GTU:  cmp_result = is_greater_equal && (~is_equal);
    ALU_GES, ALU_GEU:  cmp_result = is_greater_equal;
    ALU_LTS, ALU_SLTS,
    ALU_LTU, ALU_SLTU: cmp_result = (~is_greater_equal);
    ALU_SLETS,
    ALU_SLETU,
    ALU_LES, ALU_LEU:  cmp_result = (~is_greater_equal) || is_equal;

    default: ;
  endcase
end

assign comparison_result_o = cmp_result;



////////////////////////////////////////////////////////////
//   ____              _ _    __ __            //
//  |  _ \ ___  ___ _   _| | |_  |  \/  |_   ___  __  //
//  | |_) / _ \/ __| | | | | __| | |\/| | | | \ \/ /  //
//  |  _ <  __/\__ \ |_| | | |_  | |  | | |_| |> <  //
//  |_| \_\___||___/\__,_|_|\__| |_|  |_|\__,_/_/\_\  //
//                                                    //
////////////////////////////////////////////////////////////

always_comb
begin
  result_o   = '0;
```

```
    unique case (operator_i)
      // Standard Operations
      ALU_AND:  result_o = operand_a_i & operand_b_i;
      ALU_OR:   result_o = operand_a_i | operand_b_i;
      ALU_XOR:  result_o = operand_a_i ^ operand_b_i;

      // Adder Operations
      ALU_ADD, ALU_SUB: result_o = adder_result;

      // Shift Operations
      ALU_SLL,
      ALU_SRL, ALU_SRA: result_o = shift_result;

      // Comparison Operations
      ALU_EQ,    ALU_NE,
      ALU_GTU,   ALU_GEU,
      ALU_LTU,   ALU_LEU,
      ALU_GTS,   ALU_GES,
      ALU_LTS,   ALU_LES,
      ALU_SLTS,  ALU_SLTU,
      ALU_SLETS, ALU_SLETU: result_o = cmp_result;

      default: ; // default case to suppress unique warning
    endcase
  end

Endmodule

import zeroriscy_defines::*;

module zeroriscy_compressed_decoder
(
  input  logic [31:0] instr_i,
  output logic [31:0] instr_o,
  output logic        is_compressed_o,
  output logic        illegal_instr_o
);

  ////////////////////////////////////////////////////////////////////////////////////////////////////////////////
  //   ____                                  _   ____                  _          //
  //  / ___|___  _ __ ___  _ __  _ __ ___  ___ ___  ___  __| | |  _ \ ___  ___ ___   __| | ___ _ __  //
  // | |   / _ \| '_ ` _ \| '_ \| '__/ _ \/ __/ __|/ _ \/ _ \ | | | |/ _ \/ __/ _ \/ _` |/ _ \ '__| //
  // | |__| (_) | | | | | | | |_) | | |  __/\__ \__ \  __/ (_| | | | |_| |  __/ (_| (_) | (_| |  __/ |    //
  //  \____\___/|_| |_| |_| .__/|_|  \___||___/___/\___|\__,_| |____/ \___|\___\___/
\__,_|\___|_|    //
  //                    |_|                                                          //
  ////////////////////////////////////////////////////////////////////////////////////////////////////////////////

  always_comb
  begin
    illegal_instr_o = 1'b0;
    instr_o         = '0;

    unique case (instr_i[1:0])
      // C0
      2'b00: begin
```

```
      unique case (instr_i[15:13])
        3'b000: begin
          // c.addi4spn -> addi rd', x2, imm
          instr_o = {2'b0, instr_i[10:7], instr_i[12:11], instr_i[5], instr_i[6], 2'b00, 5'h02, 3'b000, 2'b01,
instr_i[4:2], OPCODE_OPIMM};
          if (instr_i[12:5] == 8'b0)  illegal_instr_o = 1'b1;
        end

        3'b010: begin
          // c.lw -> lw rd', imm(rs1')
          instr_o = {5'b0, instr_i[5], instr_i[12:10], instr_i[6], 2'b00, 2'b01, instr_i[9:7], 3'b010, 2'b01,
instr_i[4:2], OPCODE_LOAD};
        end

        3'b110: begin
          // c.sw -> sw rs2', imm(rs1')
          instr_o = {5'b0, instr_i[5], instr_i[12], 2'b01, instr_i[4:2], 2'b01, instr_i[9:7], 3'b010,
instr_i[11:10], instr_i[6], 2'b00, OPCODE_STORE};
        end

        default: begin
          illegal_instr_o = 1'b1;
        end
      endcase
    end

    // C1
    2'b01: begin
      unique case (instr_i[15:13])
        3'b000: begin
          // c.addi -> addi rd, rd, nzimm
          // c.nop
          instr_o = {{6 {instr_i[12]}}, instr_i[12], instr_i[6:2], instr_i[11:7], 3'b0, instr_i[11:7],
OPCODE_OPIMM};
        end

        3'b001, 3'b101: begin
          // 001: c.jal -> jal x1, imm
          // 101: c.j   -> jal x0, imm
          instr_o = {instr_i[12], instr_i[8], instr_i[10:9], instr_i[6], instr_i[7], instr_i[2], instr_i[11],
instr_i[5:3], {9 {instr_i[12]}}, 4'b0, ~instr_i[15], OPCODE_JAL};
        end

        3'b010: begin
          // c.li -> addi rd, x0, nzimm
          instr_o = {{6 {instr_i[12]}}, instr_i[12], instr_i[6:2], 5'b0, 3'b0, instr_i[11:7], OPCODE_OPIMM};
          if (instr_i[11:7] == 5'b0)  illegal_instr_o = 1'b1;
        end

        3'b011: begin
          // c.lui -> lui rd, imm
          instr_o = {{15 {instr_i[12]}}, instr_i[6:2], instr_i[11:7], OPCODE_LUI};

          if (instr_i[11:7] == 5'h02) begin
            // c.addi16sp -> addi x2, x2, nzimm
            instr_o = {{3 {instr_i[12]}}, instr_i[4:3], instr_i[5], instr_i[2], instr_i[6], 4'b0, 5'h02, 3'b000,
5'h02, OPCODE_OPIMM};
```

```
        end else if (instr_i[11:7] == 5'b0) begin
          illegal_instr_o = 1'b1;
        end

        if ({instr_i[12], instr_i[6:2]} == 6'b0) illegal_instr_o = 1'b1;
      end

      3'b100: begin
        unique case (instr_i[11:10])
          2'b00,
          2'b01: begin
            // 00: c.srli -> srli rd, rd, shamt
            // 01: c.srai -> srai rd, rd, shamt
            instr_o = {1'b0, instr_i[10], 5'b0, instr_i[6:2], 2'b01, instr_i[9:7], 3'b101, 2'b01, instr_i[9:7],
OPCODE_OPIMM};
            if (instr_i[12] == 1'b1)  illegal_instr_o = 1'b1;
            if (instr_i[6:2] == 5'b0) illegal_instr_o = 1'b1;
          end

          2'b10: begin
            // c.andi -> andi rd, rd, imm
            instr_o = {{6 {instr_i[12]}}, instr_i[12], instr_i[6:2], 2'b01, instr_i[9:7], 3'b111, 2'b01,
instr_i[9:7], OPCODE_OPIMM};
          end

          2'b11: begin
            unique case ({instr_i[12], instr_i[6:5]})
              3'b000: begin
                // c.sub -> sub rd', rd', rs2'
                instr_o = {2'b01, 5'b0, 2'b01, instr_i[4:2], 2'b01, instr_i[9:7], 3'b000, 2'b01, instr_i[9:7],
OPCODE_OP};
              end

              3'b001: begin
                // c.xor -> xor rd', rd', rs2'
                instr_o = {7'b0, 2'b01, instr_i[4:2], 2'b01, instr_i[9:7], 3'b100, 2'b01, instr_i[9:7],
OPCODE_OP};
              end

              3'b010: begin
                // c.or  -> or  rd', rd', rs2'
                instr_o = {7'b0, 2'b01, instr_i[4:2], 2'b01, instr_i[9:7], 3'b110, 2'b01, instr_i[9:7],
OPCODE_OP};
              end

              3'b011: begin
                // c.and -> and rd', rd', rs2'
                instr_o = {7'b0, 2'b01, instr_i[4:2], 2'b01, instr_i[9:7], 3'b111, 2'b01, instr_i[9:7],
OPCODE_OP};
              end

              3'b100,
              3'b101,
              3'b110,
              3'b111: begin
                // 100: c.subw
                // 101: c.addw
```

```
            illegal_instr_o = 1'b1;
          end
        endcase
      end
    endcase
  end

  3'b110, 3'b111: begin
    // 0: c.beqz -> beq rs1', x0, imm
    // 1: c.bnez -> bne rs1', x0, imm
    instr_o = {{4 {instr_i[12]}}, instr_i[6:5], instr_i[2], 5'b0, 2'b01, instr_i[9:7], 2'b00, instr_i[13],
instr_i[11:10], instr_i[4:3], instr_i[12], OPCODE_BRANCH};
  end

  default: begin
    illegal_instr_o = 1'b1;
  end
 endcase
end

// C2
2'b10: begin
 unique case (instr_i[15:13])
  3'b000: begin
    // c.slli -> slli rd, rd, shamt
    instr_o = {7'b0, instr_i[6:2], instr_i[11:7], 3'b001, instr_i[11:7], OPCODE_OPIMM};
    if (instr_i[11:7] == 5'b0)  illegal_instr_o = 1'b1;
    if (instr_i[12] == 1'b1 || instr_i[6:2] == 5'b0)  illegal_instr_o = 1'b1;
  end

  3'b010: begin
    // c.lwsp -> lw rd, imm(x2)
    instr_o = {4'b0, instr_i[3:2], instr_i[12], instr_i[6:4], 2'b00, 5'h02, 3'b010, instr_i[11:7],
OPCODE_LOAD};
    if (instr_i[11:7] == 5'b0)  illegal_instr_o = 1'b1;
  end

  3'b100: begin
    if (instr_i[12] == 1'b0) begin
      // c.mv -> add rd/rs1, x0, rs2
      instr_o = {7'b0, instr_i[6:2], 5'b0, 3'b0, instr_i[11:7], OPCODE_OP};

      if (instr_i[6:2] == 5'b0) begin
        // c.jr -> jalr x0, rd/rs1, 0
        instr_o = {12'b0, instr_i[11:7], 3'b0, 5'b0, OPCODE_JALR};
      end
    end else begin
      // c.add -> add rd, rd, rs2
      instr_o = {7'b0, instr_i[6:2], instr_i[11:7], 3'b0, instr_i[11:7], OPCODE_OP};

      if (instr_i[11:7] == 5'b0) begin
        // c.ebreak -> ebreak
        instr_o = {32'h00_10_00_73};
        if (instr_i[6:2] != 5'b0)
          illegal_instr_o = 1'b1;
      end else if (instr_i[6:2] == 5'b0) begin
        // c.jalr -> jalr x1, rs1, 0
```

```
            instr_o = {12'b0, instr_i[11:7], 3'b000, 5'b00001, OPCODE_JALR};
          end
        end
      end

      3'b110: begin
        // c.swsp -> sw rs2, imm(x2)
        instr_o = {4'b0, instr_i[8:7], instr_i[12], instr_i[6:2], 5'h02, 3'b010, instr_i[11:9], 2'b00,
OPCODE_STORE};
      end

      default: begin
        illegal_instr_o = 1'b1;
      end
    endcase
  end

  default: begin
    // 32 bit (or more) instruction
    instr_o = instr_i;
  end
 endcase
end

assign is_compressed_o = (instr_i[1:0] != 2'b11);

Endmodule

`include "zeroriscy_config.sv"

import zeroriscy_defines::*;


module zeroriscy_controller
#(
 parameter REG_ADDR_WIDTH     = 5
)
(
 input  logic      clk,
 input  logic      rst_n,

 input  logic      fetch_enable_i,          // Start the decoding
 output logic      ctrl_busy_o,             // Core is busy processing instructions
 output logic      first_fetch_o,           // Core is at the FIRST FETCH stage
 output logic      is_decoding_o,           // Core is in decoding state

 // decoder related signals
 output logic      deassert_we_o,           // deassert write enable for next instruction

 input  logic      illegal_insn_i,          // decoder encountered an invalid instruction
 input  logic      ecall_insn_i,            // ecall encountered an mret instruction
 input  logic      mret_insn_i,             // decoder encountered an mret instruction
 input  logic      pipe_flush_i,            // decoder wants to do a pipe flush
 input  logic      ebrk_insn_i,             // decoder encountered an ebreak instruction
 input  logic      csr_status_i,            // decoder encountered an csr status instruction

 // from IF/ID pipeline
```

```
  input  logic       instr_valid_i,            // instruction coming from IF/ID pipeline is valid

  // from prefetcher
  output logic       instr_req_o,              // Start fetching instructions

  // to prefetcher
  output logic       pc_set_o,                 // jump to address set by pc_mux
  output logic [2:0]  pc_mux_o,                // Selector in the Fetch stage to select the rigth PC (normal,
jump ...)
  output logic [1:0]  exc_pc_mux_o,            // Selects target PC for exception

  // LSU
  input  logic       data_misaligned_i,

  // jump/branch signals
  input  logic       branch_in_id_i,           // branch in id
  input  logic       branch_taken_ex_i,        // branch taken signal
  input  logic       branch_set_i,             // branch taken set signal
  input  logic       jump_set_i,               // jump taken set signal

  input  logic       instr_multicyle_i,        // multicycle instructions active


  // Interrupt Controller Signals
  input  logic       irq_req_ctrl_i,
  input  logic [4:0]  irq_id_ctrl_i,
  input  logic       m_IE_i,                   // interrupt enable bit from CSR (M mode)

  output logic       irq_ack_o,
  output logic [4:0]  irq_id_o,

  output logic [5:0]  exc_cause_o,
  output logic       exc_ack_o,
  output logic       exc_kill_o,

  output logic       csr_save_if_o,
  output logic       csr_save_id_o,
  output logic [5:0]  csr_cause_o,
  output logic       csr_restore_mret_id_o,
  output logic       csr_save_cause_o,

  // Debug Signals
  input  logic       dbg_req_i,                // a trap was hit, so we have to flush EX and WB
  output logic       dbg_ack_o,                // we stopped and give control to debug now

  input  logic       dbg_stall_i,              // Pipeline stall is requested
  input  logic       dbg_jump_req_i,           // Change PC to value from debug unit

  input  logic [DBG_SETS_W-1:0] dbg_settings_i,
  output logic       dbg_trap_o,

  // forwarding signals
  output logic [1:0]  operand_a_fw_mux_sel_o,    // regfile ra data selector form ID stage

  // stall signals
  output logic       halt_if_o,
  output logic       halt_id_o,
```

```
  input  logic       id_ready_i,              // ID stage is ready

  // Performance Counters
  output logic       perf_jump_o,             // we are executing a jump instruction   (j, jr, jal, jalr)
  output logic       perf_tbranch_o           // we are executing a taken branch instruction
);

  // FSM state encoding
  enum  logic [3:0] { RESET, BOOT_SET, WAIT_SLEEP, SLEEP, FIRST_FETCH,
                  DECODE, FLUSH, IRQ_TAKEN,
                  DBG_SIGNAL, DBG_SIGNAL_SLEEP, DBG_WAIT, DBG_WAIT_BRANCH, DBG_WAIT_SLEEP }
ctrl_fsm_cs, ctrl_fsm_ns;

  logic irq_enable_int;

`ifndef SYNTHESIS
  // synopsys translate_off
  // make sure we are called later so that we do not generate messages for
  // glitches
  always_ff @(negedge clk)
  begin
    // print warning in case of decoding errors
    if (is_decoding_o && illegal_insn_i) begin
      $display("%t: Illegal instruction (core %0d) at PC 0x%h:", $time, zeroriscy_core.core_id_i,
            zeroriscy_id_stage.pc_id_i);
    end
  end
  // synopsys translate_on
`endif


  ////////////////////////////////////////////////////////////////////////////////////////////////
  //   ____ ___  ____  _____   ____ ___  _   _ _____ ____   ___ _    _    _____ ____    //
  //  / ___/ _ \|  _ \| ____|  / ___/ _ \| \ | |_   _|  _ \ / _ \| |   | |   | ____|  _ \  //
  // | |  | | | | |_) |  _|   | |  | | | | \| | | | | |_) | | | | |   | |   |  _| | |_) | //
  // | |__| |_| |  _ <| |___  | |__| |_| | |\  | | | |  _ <| |_| | |___| |___| |___|  _ <  //
  //  \____\___/|_| \_\_____|  \____\___/|_| \_| |_| |_| \_\\___/|_____|_____|_____|_| \_\ //
  //                                                                                        //
  ////////////////////////////////////////////////////////////////////////////////////////////


  always_comb
  begin
    // Default values
    instr_req_o          = 1'b1;

    exc_ack_o            = 1'b0;
    exc_kill_o           = 1'b0;

    csr_save_if_o        = 1'b0;
    csr_save_id_o        = 1'b0;
    csr_restore_mret_id_o  = 1'b0;
    csr_save_cause_o      = 1'b0;

    exc_cause_o          = '0;
    exc_pc_mux_o         = EXC_PC_IRQ;
```

```
csr_cause_o          = '0;

pc_mux_o             = PC_BOOT;
pc_set_o             = 1'b0;

ctrl_fsm_ns          = ctrl_fsm_cs;

ctrl_busy_o          = 1'b1;
is_decoding_o        = 1'b0;
first_fetch_o        = 1'b0;

halt_if_o            = 1'b0;
halt_id_o            = 1'b0;
dbg_ack_o            = 1'b0;
irq_ack_o            = 1'b0;
irq_id_o             = irq_id_ctrl_i;
irq_enable_int       = m_IE_i;

// a trap towards the debug unit is generated when one of the
// following conditions are true:
// - ebreak instruction encountered
// - single-stepping mode enabled
// - illegal instruction exception and IIE bit is set
// - IRQ and INTE bit is set and no exception is currently running
// - Debuger requests halt
dbg_trap_o           = 1'b0;

perf_tbranch_o       = 1'b0;
perf_jump_o          = 1'b0;

unique case (ctrl_fsm_cs)
  // We were just reset, wait for fetch_enable
  RESET:
  begin
    ctrl_busy_o   = 1'b0;
    instr_req_o   = 1'b0;

    if (fetch_enable_i == 1'b1)
      ctrl_fsm_ns = BOOT_SET;
    else if (dbg_req_i) begin
      // just go to debug even when we did not yet get a fetch enable
      // this means that the NPC will not be set yet
      ctrl_fsm_ns = DBG_SIGNAL;
    end
  end

  // copy boot address to instr fetch address
  BOOT_SET:
  begin
    instr_req_o   = 1'b1;
    pc_mux_o      = PC_BOOT;
    pc_set_o      = 1'b1;

    ctrl_fsm_ns = FIRST_FETCH;
  end
```

```
WAIT_SLEEP:
begin
  ctrl_busy_o   = 1'b0;
  instr_req_o   = 1'b0;
  halt_if_o     = 1'b1;
  halt_id_o     = 1'b1;
  ctrl_fsm_ns   = SLEEP;
end

// instruction in if_stage is already valid
SLEEP:
begin
  // we begin execution when either fetch_enable is high or an
  // interrupt has arrived
  ctrl_busy_o   = 1'b0;
  instr_req_o   = 1'b0;
  halt_if_o     = 1'b1;
  halt_id_o     = 1'b1;
  dbg_trap_o    = dbg_settings_i[DBG_SETS_SSTE];

  if (dbg_req_i) begin
    // debug request, now we need to check if we should stay sleeping or
    // go to normal processing later
    if (fetch_enable_i || irq_req_ctrl_i)
      ctrl_fsm_ns = DBG_SIGNAL;
    else
      ctrl_fsm_ns = DBG_SIGNAL_SLEEP;

  end else begin
    // no debug request incoming, normal execution flow
    if (fetch_enable_i || irq_req_ctrl_i)
    begin
      ctrl_fsm_ns  = FIRST_FETCH;
    end
  end
end

FIRST_FETCH:
begin
  first_fetch_o = 1'b1;
  // Stall because of IF miss
  if ((id_ready_i == 1'b1) && (dbg_stall_i == 1'b0))
  begin
    ctrl_fsm_ns = DECODE;
  end

  if (irq_req_ctrl_i & irq_enable_int) begin
    // This assumes that the pipeline is always flushed before
    // going to sleep.
    ctrl_fsm_ns = IRQ_TAKEN;
    halt_if_o   = 1'b1;
    halt_id_o   = 1'b1;
  end
end

DECODE:
begin
```

```
is_decoding_o = 1'b0;

  // decode and execute instructions only if the current conditional
  // branch in the EX stage is either not taken, or there is no
  // conditional branch in the EX stage
  if (instr_valid_i)
  begin // now analyze the current instruction in the ID stage
    is_decoding_o = 1'b1;

    unique case (1'b1)

      branch_set_i: begin
        pc_mux_o         = PC_JUMP;
        pc_set_o         = 1'b1;
        perf_tbranch_o    = 1'b1;
        dbg_trap_o       = dbg_settings_i[DBG_SETS_SSTE];
        if (dbg_req_i)
          ctrl_fsm_ns = DBG_SIGNAL;
      end
      jump_set_i: begin
        pc_mux_o         = PC_JUMP;
        pc_set_o         = 1'b1;
        perf_jump_o       = 1'b1;
        dbg_trap_o       = dbg_settings_i[DBG_SETS_SSTE];
      end
      mret_insn_i | ecall_insn_i | pipe_flush_i | ebrk_insn_i | illegal_insn_i | csr_status_i: begin
        ctrl_fsm_ns = FLUSH;
        halt_if_o   = 1'b1;
        halt_id_o   = 1'b1;
      end
      default: begin
        dbg_trap_o = dbg_settings_i[DBG_SETS_SSTE];

        unique case (1'b1)
          irq_req_ctrl_i & irq_enable_int & ~instr_multicyle_i & ~branch_in_id_i: begin
            ctrl_fsm_ns = IRQ_TAKEN;
            halt_if_o   = 1'b1;
            halt_id_o   = 1'b1;
          end
          dbg_req_i & ~branch_taken_ex_i: begin
            halt_if_o = 1'b1;
            if (id_ready_i) begin
              ctrl_fsm_ns = DBG_SIGNAL;
            end
          end
          default:
            exc_kill_o    = irq_req_ctrl_i & ~instr_multicyle_i & ~branch_in_id_i ? 1'b1 : 1'b0;
        endcase
      end
    endcase
  end
  else //~instr_valid_i
  begin
      if (irq_req_ctrl_i & irq_enable_int) begin
        ctrl_fsm_ns = IRQ_TAKEN;
        halt_if_o   = 1'b1;
        halt_id_o   = 1'b1;
```

```
      end
    end
end

// now we can signal to the debugger that our pipeline is empty and it
// can examine our current state
DBG_SIGNAL:
begin
  dbg_ack_o  = 1'b1;
  halt_if_o  = 1'b1;

  ctrl_fsm_ns = DBG_WAIT;
end

DBG_SIGNAL_SLEEP:
begin
  dbg_ack_o  = 1'b1;
  halt_if_o  = 1'b1;

  ctrl_fsm_ns = DBG_WAIT_SLEEP;
end

// The Debugger is active in this state
// we wait until it is done and go back to SLEEP
DBG_WAIT_SLEEP:
begin
  halt_if_o = 1'b1;

  if (dbg_jump_req_i) begin
    pc_mux_o     = PC_DBG_NPC;
    pc_set_o     = 1'b1;
    ctrl_fsm_ns  = DBG_WAIT;
  end

  if (dbg_stall_i == 1'b0) begin
    ctrl_fsm_ns = SLEEP;
  end
end

// The Debugger is active in this state
// we wait until it is done and go back to DECODE
DBG_WAIT:
begin
  halt_if_o = 1'b1;

  if (dbg_jump_req_i) begin
    pc_mux_o     = PC_DBG_NPC;
    pc_set_o     = 1'b1;
    ctrl_fsm_ns  = DBG_WAIT;
  end

  if (dbg_stall_i == 1'b0) begin
    ctrl_fsm_ns = DECODE;
  end
end

IRQ_TAKEN:
```

```
begin
  pc_mux_o         = PC_EXCEPTION;
  pc_set_o         = 1'b1;

  exc_pc_mux_o      = EXC_PC_IRQ;
  exc_cause_o       = {1'b0,irq_id_ctrl_i};

  csr_save_cause_o  = 1'b1;
  csr_cause_o       = {1'b1,irq_id_ctrl_i};

  csr_save_if_o     = 1'b1;

  irq_ack_o         = 1'b1;
  exc_ack_o         = 1'b1;

  ctrl_fsm_ns       = DECODE;
end

// flush the pipeline, insert NOP
FLUSH:
begin

  halt_if_o = fetch_enable_i ? dbg_req_i : 1'b1;
  halt_id_o = 1'b1;

  ctrl_fsm_ns = dbg_req_i ? DBG_SIGNAL : DECODE;

  unique case(1'b1)
    ecall_insn_i: begin
        //ecall
        pc_mux_o            = PC_EXCEPTION;
        pc_set_o            = 1'b1;
        csr_save_id_o        = 1'b1;
        csr_save_cause_o      = 1'b1;
        exc_pc_mux_o         = EXC_PC_ECALL;
        exc_cause_o          = EXC_CAUSE_ECALL_MMODE;
        csr_cause_o          = EXC_CAUSE_ECALL_MMODE;
        dbg_trap_o           = dbg_settings_i[DBG_SETS_ECALL] | dbg_settings_i[DBG_SETS_SSTE];
    end
    illegal_insn_i: begin
        //exceptions
        pc_mux_o            = PC_EXCEPTION;
        pc_set_o            = 1'b1;
        csr_save_id_o        = 1'b1;
        csr_save_cause_o      = 1'b1;
        exc_pc_mux_o         = EXC_PC_ILLINSN;
        exc_cause_o          = EXC_CAUSE_ILLEGAL_INSN;
        csr_cause_o          = EXC_CAUSE_ILLEGAL_INSN;
        dbg_trap_o           = dbg_settings_i[DBG_SETS_EILL] | dbg_settings_i[DBG_SETS_SSTE];
    end
    mret_insn_i: begin
        //mret
        pc_mux_o            = PC_ERET;
        pc_set_o            = 1'b1;
        csr_restore_mret_id_o = 1'b1;
        dbg_trap_o           = dbg_settings_i[DBG_SETS_SSTE];
    end
```

```
      ebrk_insn_i: begin
         dbg_trap_o    = dbg_settings_i[DBG_SETS_EBRK] | dbg_settings_i[DBG_SETS_SSTE];;
         exc_cause_o   = EXC_CAUSE_BREAKPOINT;
      end
      csr_status_i: begin
         dbg_trap_o    = dbg_settings_i[DBG_SETS_SSTE];
      end
      pipe_flush_i: begin
         dbg_trap_o    = dbg_settings_i[DBG_SETS_SSTE];
      end
      default:;
    endcase

    if(fetch_enable_i) begin
      if(dbg_req_i)
        ctrl_fsm_ns = DBG_SIGNAL;
      else
        ctrl_fsm_ns = DECODE;
    end else begin
      if(dbg_req_i)
        ctrl_fsm_ns = DBG_SIGNAL_SLEEP;
      else
        ctrl_fsm_ns = (mret_insn_i | pipe_flush_i) ? WAIT_SLEEP : DECODE;
    end
  end

  default: begin
    instr_req_o = 1'b0;
    ctrl_fsm_ns = RESET;
  end
 endcase
end

//////////////////////////////////////////////////////////////
//  ____  _      _ _   ____          _            _ //
// / ___|| |_ __ _| | |  / ___|___  _ __ | |_ _ __ ___ | | //
// \___ \| __/ _` | | | |   / _ \| '_ \| __| '__/ _ \| | //
//  ___) | || (_| | | | | |__| (_) | | | | |_| | | (_) | | //
// |____/ \__\__,_|_|_|_|  \____\___/|_| |_|\__|_|  \___/|_| //
//                                                          //
//////////////////////////////////////////////////////////////
always_comb
begin

 deassert_we_o  = 1'b0;

 // deassert WE when the core is not decoding instructions
 if (~is_decoding_o)
   deassert_we_o = 1'b1;

 // deassert WE in case of illegal instruction
 if (illegal_insn_i)
   deassert_we_o = 1'b1;
end

// Forwarding control unit
assign operand_a_fw_mux_sel_o = data_misaligned_i ? SEL_MISALIGNED : SEL_REGFILE;
```

```
  // update registers
  always_ff @(posedge clk , negedge rst_n)
  begin : UPDATE_REGS
    if ( rst_n == 1'b0 )
    begin
      ctrl_fsm_cs <= RESET;
      //jump_done_q <= 1'b0;
    end
    else
    begin
      ctrl_fsm_cs <= ctrl_fsm_ns;
      // clear when id is valid (no instruction incoming)
      //jump_done_q <= jump_done & (~id_ready_i);
    end
  end


  //-----------------------------------------------------------------------------
  // Assertions
  //-----------------------------------------------------------------------------
`ifndef VERILATOR
  assert property (
    @(posedge clk) (~(dbg_req_i & irq_req_ctrl_i)) ) else $warning("Both dbg_req_i and irq_req_ctrl_i
are active");
`endif
endmodule // controller

`include "zeroriscy_config.sv"

import zeroriscy_defines::*;

module zeroriscy_core
#(
  parameter N_EXT_PERF_COUNTERS = 0,
  parameter RV32E               = 0,
  parameter RV32M               = 1
)
(
  // Clock and Reset
  input  logic       clk_i,
  input  logic       rst_ni,

  input  logic       clock_en_i,    // enable clock, otherwise it is gated
  input  logic       test_en_i,     // enable all clock gates for testing

  // Core ID, Cluster ID and boot address are considered more or less static
  input  logic [ 3:0] core_id_i,
  input  logic [ 5:0] cluster_id_i,
  input  logic [31:0] boot_addr_i,

  // Instruction memory interface
  output logic                      instr_req_o,
  input  logic                      instr_gnt_i,
  input  logic                      instr_rvalid_i,
  output logic                [31:0] instr_addr_o,
  input  logic                [31:0] instr_rdata_i,
```

```
  // Data memory interface
  output logic       data_req_o,
  input  logic       data_gnt_i,
  input  logic       data_rvalid_i,
  output logic       data_we_o,
  output logic [3:0]  data_be_o,
  output logic [31:0] data_addr_o,
  output logic [31:0] data_wdata_o,
  input  logic [31:0] data_rdata_i,
  input  logic       data_err_i,

  // Interrupt inputs
  input  logic       irq_i,              // level sensitive IR lines
  input  logic [4:0]  irq_id_i,
  output logic       irq_ack_o,           // irq ack
  output logic [4:0]  irq_id_o,

  // Debug Interface
  input  logic       debug_req_i,
  output logic       debug_gnt_o,
  output logic       debug_rvalid_o,
  input  logic [14:0] debug_addr_i,
  input  logic       debug_we_i,
  input  logic [31:0] debug_wdata_i,
  output logic [31:0] debug_rdata_o,
  output logic       debug_halted_o,
  input  logic       debug_halt_i,
  input  logic       debug_resume_i,

  // CPU Control Signals
  input  logic       fetch_enable_i,
  output logic       core_busy_o,

  input  logic [N_EXT_PERF_COUNTERS-1:0] ext_perf_counters_i
);

  localparam N_HWLP      = 2;
  localparam N_HWLP_BITS = $clog2(N_HWLP);

  // IF/ID signals
  logic            instr_valid_id;
  logic [31:0]      instr_rdata_id;    // Instruction sampled inside IF stage
  logic            is_compressed_id;
  logic            illegal_c_insn_id; // Illegal compressed instruction sent to ID stage
  logic [31:0]      pc_if;            // Program counter in IF stage
  logic [31:0]      pc_id;            // Program counter in ID stage

  logic            clear_instr_valid;
  logic            pc_set;
  logic [2:0]       pc_mux_id;     // Mux selector for next PC
  logic [1:0]       exc_pc_mux_id;     // Mux selector for exception PC
  logic [5:0]       exc_cause;

  logic            lsu_load_err;
  logic            lsu_store_err;
```

```
// ID performance counter signals
logic       is_decoding;

logic       data_misaligned;
logic [31:0] misaligned_addr;

// Jump and branch target and decision (EX->IF)
logic [31:0] jump_target_ex;
logic       branch_in_ex;
logic       branch_decision;

logic       ctrl_busy;
logic       if_busy;
logic       lsu_busy;

// ALU Control
logic [ALU_OP_WIDTH-1:0] alu_operator_ex;
logic [31:0] alu_operand_a_ex;
logic [31:0] alu_operand_b_ex;

logic [31:0] alu_adder_result_ex; // Used to forward computed address to LSU
logic [31:0] regfile_wdata_ex;

// Multiplier Control
logic       mult_en_ex;
logic       div_en_ex;
logic [1:0]  multdiv_operator_ex;
logic [1:0]  multdiv_signed_mode_ex;
logic [31:0] multdiv_operand_a_ex;
logic [31:0] multdiv_operand_b_ex;

// CSR control
logic       csr_access_ex;
logic  [1:0] csr_op_ex;

logic       csr_access;
logic  [1:0] csr_op;
logic [11:0] csr_addr;
logic [11:0] csr_addr_int;
logic [31:0] csr_rdata;
logic [31:0] csr_wdata;

// Data Memory Control
logic       data_we_ex;
logic [1:0]  data_type_ex;
logic       data_sign_ext_ex;
logic [1:0]  data_reg_offset_ex;
logic       data_req_ex;
logic [31:0] data_wdata_ex;
logic       data_load_event_ex;
logic       data_misaligned_ex;
logic [31:0] regfile_wdata_lsu;

// stall control
logic       halt_if;
logic       id_ready;
logic       ex_ready;
```

```
logic       if_valid;
logic       id_valid;
logic       wb_valid;

logic       lsu_ready_ex;
logic       data_valid_lsu;

// Signals between instruction core interface and pipe (if and id stages)
logic       instr_req_int;    // Id stage asserts a req to instruction core interface

// Interrupts
logic       m_irq_enable;
logic [31:0] mepc;

logic       csr_save_cause;
logic       csr_save_if;
logic       csr_save_id;
logic [5:0]  csr_cause;
logic       csr_restore_mret_id;
logic       csr_restore_uret_id;

// Debug Unit
logic [DBG_SETS_W-1:0] dbg_settings;
logic       dbg_req;
logic       dbg_ack;
logic       dbg_stall;
logic       dbg_trap;

// Debug GPR Read Access
logic       dbg_reg_rreq;
logic [4:0]  dbg_reg_raddr;
logic [31:0] dbg_reg_rdata;

// Debug GPR Write Access
logic       dbg_reg_wreq;
logic [4:0]  dbg_reg_waddr;
logic [31:0] dbg_reg_wdata;

// Debug CSR Access
logic       dbg_csr_req;
logic [11:0] dbg_csr_addr;
logic       dbg_csr_we;
logic [31:0] dbg_csr_wdata;

logic [31:0] dbg_jump_addr;
logic       dbg_jump_req;

// Performance Counters
logic       perf_imiss;
logic       perf_jump;
logic       perf_branch;
logic       perf_tbranch;

//core busy signals
logic       core_ctrl_firstfetch, core_busy_int, core_busy_q;
```

```
////////////////////////////////////////////////////////////////////////////////////////////////
//   ____ _         _     __  __                                        _    //
//  / ___| | ___   ___| | __ |  \/  | __ _ _ __   __ _  __ _  ___ _ __ ___   ___ _ __ | |_  //
// | |   | |/ _ \ / __| |/ / | |\/| |/ _` | '_ \ / _` |/ _` |/ _ \ '_ `_ \ / _ \ '_ \| __| //
// | |___| | (_) | (__|   <  | |  | | (_| | | | | (_| | (_| |  __/ | | | | |  __/ | | | |_  //
// \____|_|\___/ \___|_|\_\ |_|  |_|\__,_|_| |_|\__,_|\__,_|\___|_| |_|\___|_| |_|\__| //
//                                   |___/                                    //
////////////////////////////////////////////////////////////////////////////////////////////////

  logic       clk;

  logic       clock_en;
  logic       dbg_busy;

  logic       sleeping;

  // if we are sleeping on a barrier let's just wait on the instruction
  // interface to finish loading instructions
  assign core_busy_int = (data_load_event_ex & data_req_o) ? if_busy : (if_busy | ctrl_busy |
lsu_busy);

  always_ff @(posedge clk, negedge rst_ni)
  begin
    if (rst_ni == 1'b0) begin
      core_busy_q <= 1'b0;
    end else begin
      core_busy_q <= core_busy_int;
    end
  end

  assign core_busy_o = core_ctrl_firstfetch ? 1'b1 : core_busy_q;

  assign dbg_busy    = dbg_req | dbg_csr_req | dbg_jump_req | dbg_reg_wreq | debug_req_i;

  assign clock_en    = clock_en_i | core_busy_o | dbg_busy;

  assign sleeping    = (~fetch_enable_i) & (~core_busy_o);


  // main clock gate of the core
  // generates all clocks except the one for the debug unit which is
  // independent
  cluster_clock_gating core_clock_gate_i
  (
    .clk_i     ( clk_i          ),
    .en_i      ( clock_en       ),
    .test_en_i ( test_en_i      ),
    .clk_o     ( clk            )
  );

  ////////////////////////////////////////////////////
  //   ___ _____   ____ _____  _   ____ _____   //
  //  |_ _|  ___| / ___|_   _|/ \  / ___| ____|  //
  //   | || |_    \___ \ | | / _ \| |  _|  _|    //
  //   | ||  _|    ___) || |/ ___ \ |_| | |___   //
  //  |___|_|     |____/ |_/_/   \_\____|_____|  //
  //                                            //
```

```
////////////////////////////////////////////////////
zeroriscy_if_stage if_stage_i
(
  .clk              ( clk             ),
  .rst_n            ( rst_ni          ),

  // boot address (trap vector location)
  .boot_addr_i      ( boot_addr_i     ),

  // instruction request control
  .req_i            ( instr_req_int   ),

  // instruction cache interface
  .instr_req_o      ( instr_req_o     ),
  .instr_addr_o     ( instr_addr_o    ),
  .instr_gnt_i      ( instr_gnt_i     ),
  .instr_rvalid_i   ( instr_rvalid_i  ),
  .instr_rdata_i    ( instr_rdata_i   ),

  // outputs to ID stage
  .instr_valid_id_o   ( instr_valid_id   ),
  .instr_rdata_id_o   ( instr_rdata_id   ),
  .is_compressed_id_o ( is_compressed_id ),
  .illegal_c_insn_id_o ( illegal_c_insn_id ),
  .pc_if_o            ( pc_if            ),
  .pc_id_o            ( pc_id            ),

  // control signals
  .clear_instr_valid_i ( clear_instr_valid ),
  .pc_set_i           ( pc_set           ),
  .exception_pc_reg_i ( mepc             ), // exception return address
  .pc_mux_i           ( pc_mux_id        ), // sel for pc multiplexer
  .exc_pc_mux_i       ( exc_pc_mux_id    ),
  .exc_vec_pc_mux_i   ( exc_cause[4:0]   ),


  // from debug unit
  .dbg_jump_addr_i    ( dbg_jump_addr    ),

  // Jump targets
  .jump_target_ex_i   ( jump_target_ex   ),

  // pipeline stalls
  .halt_if_i          ( halt_if          ),
  .id_ready_i         ( id_ready         ),
  .if_valid_o         ( if_valid         ),

  .if_busy_o          ( if_busy          ),
  .perf_imiss_o       ( perf_imiss       )
);


////////////////////////////////////////////////////
//   ___ ____    ____ _____  _    ____ _____   //
//  |_ _|  _ \  / ___|_   _|/ \  / ___| ____|  //
//   | || | | | \___ \ | | / _ \| |  _|  _|    //
//   | || |_| |  ___) || |/ ___ \ |_| | |___   //
```

```
//  |___|____/  |____/ |_/_/  \_\____|_____|  //
//                                            //
////////////////////////////////////////////////
zeroriscy_id_stage
#(
  .RV32E(RV32E),
  .RV32M(RV32M)
)
id_stage_i
(
  .clk                    ( clk               ),
  .rst_n                  ( rst_ni            ),

  .test_en_i              ( test_en_i         ),

  // Processor Enable
  .fetch_enable_i         ( fetch_enable_i    ),
  .ctrl_busy_o            ( ctrl_busy         ),
  .core_ctrl_firstfetch_o ( core_ctrl_firstfetch ),
  .is_decoding_o          ( is_decoding       ),

  // Interface to instruction memory
  .instr_valid_i          ( instr_valid_id    ),
  .instr_rdata_i          ( instr_rdata_id    ),
  .instr_req_o            ( instr_req_int     ),

  // Jumps and branches
  .branch_in_ex_o         ( branch_in_ex      ),
  .branch_decision_i      ( branch_decision   ),

  // IF and ID control signals
  .clear_instr_valid_o    ( clear_instr_valid ),
  .pc_set_o               ( pc_set            ),
  .pc_mux_o               ( pc_mux_id         ),
  .exc_pc_mux_o           ( exc_pc_mux_id     ),
  .exc_cause_o            ( exc_cause         ),

  .illegal_c_insn_i       ( illegal_c_insn_id ),
  .is_compressed_i        ( is_compressed_id  ),

  .pc_id_i                ( pc_id             ),

  // Stalls
  .halt_if_o              ( halt_if           ),

  .id_ready_o             ( id_ready          ),
  .ex_ready_i             ( ex_ready          ),

  .id_valid_o             ( id_valid          ),

  .alu_operator_ex_o      ( alu_operator_ex   ),
  .alu_operand_a_ex_o     ( alu_operand_a_ex  ),
  .alu_operand_b_ex_o     ( alu_operand_b_ex  ),

  .mult_en_ex_o           ( mult_en_ex        ),
  .div_en_ex_o            ( div_en_ex         ),
  .multdiv_operator_ex_o  ( multdiv_operator_ex ),
```

```
.multdiv_signed_mode_ex_o     ( multdiv_signed_mode_ex ),
.multdiv_operand_a_ex_o       ( multdiv_operand_a_ex   ),
.multdiv_operand_b_ex_o       ( multdiv_operand_b_ex   ),

// CSR ID/EX
.csr_access_ex_o              ( csr_access_ex       ),
.csr_op_ex_o                  ( csr_op_ex           ),
.csr_cause_o                  ( csr_cause           ),
.csr_save_if_o                ( csr_save_if         ), // control signal to save pc
.csr_save_id_o                ( csr_save_id         ), // control signal to save pc
.csr_restore_mret_id_o        ( csr_restore_mret_id  ), // control signal to restore pc
.csr_save_cause_o             ( csr_save_cause      ),

// LSU
.data_req_ex_o                ( data_req_ex         ), // to load store unit
.data_we_ex_o                 ( data_we_ex          ), // to load store unit
.data_type_ex_o               ( data_type_ex        ), // to load store unit
.data_sign_ext_ex_o           ( data_sign_ext_ex    ), // to load store unit
.data_reg_offset_ex_o         ( data_reg_offset_ex  ), // to load store unit
.data_load_event_ex_o         ( data_load_event_ex  ), // to load store unit
.data_wdata_ex_o              ( data_wdata_ex       ), // to load store unit

.data_misaligned_i            ( data_misaligned     ),
.misaligned_addr_i            ( misaligned_addr     ),

// Interrupt Signals
.irq_i                        ( irq_i               ), // incoming interrupts
.irq_id_i                     ( irq_id_i            ),
.m_irq_enable_i               ( m_irq_enable        ),
.irq_ack_o                    ( irq_ack_o           ),
.irq_id_o                     ( irq_id_o            ),

.lsu_load_err_i               ( lsu_load_err        ),
.lsu_store_err_i              ( lsu_store_err       ),

// Debug Unit Signals
.dbg_settings_i               ( dbg_settings        ),
.dbg_req_i                    ( dbg_req             ),
.dbg_ack_o                    ( dbg_ack             ),
.dbg_stall_i                  ( dbg_stall           ),
.dbg_trap_o                   ( dbg_trap            ),

.dbg_reg_rreq_i               ( dbg_reg_rreq        ),
.dbg_reg_raddr_i              ( dbg_reg_raddr       ),
.dbg_reg_rdata_o              ( dbg_reg_rdata       ),

.dbg_reg_wreq_i               ( dbg_reg_wreq        ),
.dbg_reg_waddr_i              ( dbg_reg_waddr       ),
.dbg_reg_wdata_i              ( dbg_reg_wdata       ),

.dbg_jump_req_i               ( dbg_jump_req        ),

// write data to commit in the register file
.regfile_wdata_lsu_i          ( regfile_wdata_lsu   ),
.regfile_wdata_ex_i           ( regfile_wdata_ex    ),
.csr_rdata_i                  ( csr_rdata           ),
```

```
  // Performance Counters
  .perf_jump_o              ( perf_jump          ),
  .perf_branch_o            ( perf_branch        ),
  .perf_tbranch_o           ( perf_tbranch       )

);


zeroriscy_ex_block
#(
  //change the localparam MULT_TYPE to 0 or 1
  //if you want a SLOW or FAST multiplier
  .RV32M(RV32M)
)
ex_block_i
(
  .clk                    ( clk                ),
  .rst_n                  ( rst_ni             ),
  // Alu signals from ID stage
  //TODO: hot encoding
  .alu_operator_i         ( alu_operator_ex    ),
  .multdiv_operator_i     ( multdiv_operator_ex  ),
  .alu_operand_a_i        ( alu_operand_a_ex   ),
  .alu_operand_b_i        ( alu_operand_b_ex   ),

  // Multipler
  .mult_en_i              ( mult_en_ex         ),
  .div_en_i               ( div_en_ex          ),
  .multdiv_signed_mode_i  ( multdiv_signed_mode_ex),
  .multdiv_operand_a_i    ( multdiv_operand_a_ex ),
  .multdiv_operand_b_i    ( multdiv_operand_b_ex ),
  .alu_adder_result_ex_o  ( alu_adder_result_ex  ), // from ALU to LSU
  .regfile_wdata_ex_o     ( regfile_wdata_ex   ),

  // To IF: Jump and branch target and decision
  .jump_target_o          ( jump_target_ex     ),
  .branch_decision_o      ( branch_decision    ),

  .lsu_en_i               ( data_req_ex        ),
  .lsu_ready_ex_i         ( data_valid_lsu     ),
  .ex_ready_o             ( ex_ready           )
);

//////////////////////////////////////////////////////////////////////////////////////////////
//   _    ___    _    ____    ____ _____ ___  ____  _____   _   _ _   _ ___ _____   //
//  | |  / _ \ / \  |  _ \ / ___|_   _/ _ \|  _ \| ____| | | | | \ | |_ _|_   _|  //
//  | | | | | |/ _ \ | | | | \___ \ | || | | | |_) |  _|   | | | | \| || | | |    //
//  | |__| |_| / ___ \| |_| |  ___) || || |_| |  _ <| |___  | |_| | |\  || | | |    //
//  |_____\___/_/   \_\____/  |____/ |_| \___/|_| \_\_____| \___/|_| \_|___| |_|    //
//                                                                                  //
//////////////////////////////////////////////////////////////////////////////////////////////

zeroriscy_load_store_unit  load_store_unit_i
(
  .clk                ( clk                ),
  .rst_n              ( rst_ni             ),
```

```
  //output to data memory
  .data_req_o          ( data_req_o        ),
  .data_gnt_i          ( data_gnt_i        ),
  .data_rvalid_i       ( data_rvalid_i     ),
  .data_err_i          ( data_err_i        ),

  .data_addr_o         ( data_addr_o       ),
  .data_we_o           ( data_we_o         ),
  .data_be_o           ( data_be_o         ),
  .data_wdata_o        ( data_wdata_o      ),
  .data_rdata_i        ( data_rdata_i      ),

  // signal from ex stage
  .data_we_ex_i        ( data_we_ex        ),
  .data_type_ex_i      ( data_type_ex      ),
  .data_wdata_ex_i     ( data_wdata_ex     ),
  .data_reg_offset_ex_i  ( data_reg_offset_ex ),
  .data_sign_ext_ex_i    ( data_sign_ext_ex   ),

  .data_rdata_ex_o     ( regfile_wdata_lsu  ),
  .data_req_ex_i       ( data_req_ex       ),

  .adder_result_ex_i    ( alu_adder_result_ex),

  .data_misaligned_o    ( data_misaligned    ),
  .misaligned_addr_o    ( misaligned_addr    ),

  // exception signals
  .load_err_o          ( lsu_load_err      ),
  .store_err_o         ( lsu_store_err     ),

  // control signals
  .data_valid_o        ( data_valid_lsu    ),
  .lsu_update_addr_o    (                  ),
  .busy_o              ( lsu_busy          )
);


////////////////////////////////////////
//       ____ ____  ____          //
//      / ___/ ___| |  _ \ ___     //
//      | |   \___ \| |_) / __|    //
//      | |___ ___) |  _ <\__ \    //
//      \____|____/|_| \_\___/    //
//                              //
//   Control and Status Registers   //
////////////////////////////////////////

zeroriscy_cs_registers
#(
  .N_EXT_CNT      ( N_EXT_PERF_COUNTERS   )
)
cs_registers_i
(
  .clk                  ( clk               ),
  .rst_n                ( rst_ni            ),
```

```
  // Core and Cluster ID from outside
  .core_id_i            ( core_id_i         ),
  .cluster_id_i         ( cluster_id_i      ),
  // boot address
  .boot_addr_i          ( boot_addr_i[31:8]  ),
  // Interface to CSRs (SRAM like)
  .csr_access_i         ( csr_access        ),
  .csr_addr_i           ( csr_addr          ),
  .csr_wdata_i          ( csr_wdata         ),
  .csr_op_i             ( csr_op            ),
  .csr_rdata_o          ( csr_rdata         ),

  // Interrupt related control signals
  .m_irq_enable_o       ( m_irq_enable      ),
  .mepc_o               ( mepc              ),

  .pc_if_i              ( pc_if             ),
  .pc_id_i              ( pc_id             ),

  .csr_save_if_i        ( csr_save_if       ),
  .csr_save_id_i        ( csr_save_id       ),
  .csr_restore_mret_i   ( csr_restore_mret_id ),
  .csr_cause_i          ( csr_cause         ),
  .csr_save_cause_i     ( csr_save_cause    ),


  // performance counter related signals
  .if_valid_i           ( if_valid          ),
  .id_valid_i           ( id_valid          ),
  .is_compressed_i      ( is_compressed_id  ),
  .is_decoding_i        ( is_decoding       ),

  .imiss_i              ( perf_imiss        ),
  .pc_set_i             ( pc_set            ),
  .jump_i               ( perf_jump         ),
  .branch_i             ( perf_branch       ),
  .branch_taken_i       ( perf_tbranch      ),
  .mem_load_i           ( data_req_o & data_gnt_i & (~data_we_o) ),
  .mem_store_i          ( data_req_o & data_gnt_i & data_we_o    ),

  .ext_counters_i       ( ext_perf_counters_i                    )
);

// Mux for CSR access through Debug Unit
assign csr_access   = (dbg_csr_req == 1'b0) ? csr_access_ex : 1'b1;
assign csr_addr     = (dbg_csr_req == 1'b0) ? csr_addr_int     : dbg_csr_addr;
assign csr_wdata    = (dbg_csr_req == 1'b0) ? alu_operand_a_ex : dbg_csr_wdata;
assign csr_op       = (dbg_csr_req == 1'b0) ? csr_op_ex
                                            : (dbg_csr_we == 1'b1 ? CSR_OP_WRITE
                                                                  : CSR_OP_NONE );
assign csr_addr_int = csr_access_ex ? alu_operand_b_ex[11:0] : '0;


//////////////////////////////////////////////////////////////
//  ____  _____ ____  _   _ ____   _   _ _   _ ___ _____  //
// |  _ \| ____| __ )| | | |/ ___| | | | | \ | |_ _|_   _| //
// | | | |  _| |  _ \| | | | _  | | | |  \| || |  | |   //
```

```
// | |_| | |___| |_) | |_| | |_| | | |_| | |\  || |  | |   //
// |____/|_____|____/ \___/ \____|  \___/|_| \_|___| |_|   //
//                                                          //
//////////////////////////////////////////////////////////////

zeroriscy_debug_unit debug_unit_i
(
  .clk            ( clk_i            ), // always-running clock for debug
  .rst_n          ( rst_ni           ),

  // Debug Interface
  .debug_req_i      ( debug_req_i       ),
  .debug_gnt_o      ( debug_gnt_o       ),
  .debug_rvalid_o   ( debug_rvalid_o    ),
  .debug_addr_i     ( debug_addr_i      ),
  .debug_we_i       ( debug_we_i        ),
  .debug_wdata_i    ( debug_wdata_i     ),
  .debug_rdata_o    ( debug_rdata_o     ),
  .debug_halt_i     ( debug_halt_i      ),
  .debug_resume_i   ( debug_resume_i    ),
  .debug_halted_o   ( debug_halted_o    ),

  // To/From Core
  .settings_o       ( dbg_settings      ),
  .trap_i           ( dbg_trap          ),
  .exc_cause_i      ( exc_cause         ),
  .stall_o          ( dbg_stall         ),
  .dbg_req_o        ( dbg_req           ),
  .dbg_ack_i        ( dbg_ack           ),

  // register file read port
  .regfile_rreq_o   ( dbg_reg_rreq      ),
  .regfile_raddr_o  ( dbg_reg_raddr     ),
  .regfile_rdata_i  ( dbg_reg_rdata     ),

  // register file write port
  .regfile_wreq_o   ( dbg_reg_wreq      ),
  .regfile_waddr_o  ( dbg_reg_waddr     ),
  .regfile_wdata_o  ( dbg_reg_wdata     ),

  // CSR read/write port
  .csr_req_o        ( dbg_csr_req       ),
  .csr_addr_o       ( dbg_csr_addr      ),
  .csr_we_o         ( dbg_csr_we        ),
  .csr_wdata_o      ( dbg_csr_wdata     ),
  .csr_rdata_i      ( csr_rdata         ),

  // signals for PPC and NPC
  .pc_if_i          ( pc_if             ), // from IF stage
  .pc_id_i          ( pc_id             ), // from ID stage
  .instr_valid_id_i ( instr_valid_id    ),

  .sleeping_i       ( sleeping          ),

  .jump_addr_o      ( dbg_jump_addr     ), // PC from debug unit
  .jump_req_o       ( dbg_jump_req      )  // set PC to new value
);
```

```
`ifndef VERILATOR
`ifdef TRACE_EXECUTION
 zeroriscy_tracer zeroriscy_tracer_i
 (
  .clk          ( clk_i                          ), // always-running clock for tracing
  .rst_n        ( rst_ni                         ),

  .fetch_enable   ( fetch_enable_i                 ),
  .core_id        ( core_id_i                      ),
  .cluster_id     ( cluster_id_i                   ),

  .pc           ( id_stage_i.pc_id_i               ),
  .instr        ( id_stage_i.instr                 ),
  .compressed     ( id_stage_i.is_compressed_i       ),
  .id_valid       ( id_stage_i.id_valid_o            ),
  .is_decoding    ( id_stage_i.is_decoding_o         ),
  .is_branch      ( id_stage_i.branch_in_id          ),
  .branch_taken   ( id_stage_i.branch_set_q          ),
  .pipe_flush     ( id_stage_i.controller_i.pipe_flush_i ),
  .mret_insn      ( id_stage_i.controller_i.mret_insn_i  ),
  .ecall_insn     ( id_stage_i.controller_i.ecall_insn_i ),
  .ebrk_insn      ( id_stage_i.controller_i.ebrk_insn_i  ),
  .csr_status     ( id_stage_i.controller_i.csr_status_i ),
  .rs1_value      ( id_stage_i.operand_a_fw_id       ),
  .rs2_value      ( id_stage_i.operand_b_fw_id       ),

  .lsu_value      ( data_wdata_ex                    ),

  .ex_reg_addr    ( id_stage_i.regfile_waddr_mux     ),
  .ex_reg_we      ( id_stage_i.regfile_we_mux        ),
  .ex_reg_wdata   ( id_stage_i.regfile_wdata_mux     ),
  .data_valid_lsu ( data_valid_lsu                   ),
  .ex_data_addr   ( data_addr_o                      ),
  .ex_data_req    ( data_req_o                       ),
  .ex_data_gnt    ( data_gnt_i                       ),
  .ex_data_we     ( data_we_o                        ),

  .ex_data_wdata  ( data_wdata_o                     ),

  .lsu_reg_wdata  ( regfile_wdata_lsu                ),

  .imm_u_type     ( id_stage_i.imm_u_type            ),
  .imm_uj_type    ( id_stage_i.imm_uj_type           ),
  .imm_i_type     ( id_stage_i.imm_i_type            ),
  .imm_iz_type    ( id_stage_i.imm_iz_type[11:0]     ),
  .imm_z_type     ( id_stage_i.imm_z_type            ),
  .imm_s_type     ( id_stage_i.imm_s_type            ),
  .imm_sb_type    ( id_stage_i.imm_sb_type           )
 );
`endif
`endif
Endmodule

`include "zeroriscy_config.sv"

import zeroriscy_defines::*;
```

```
`ifndef PULP_FPGA_EMUL
 `ifdef SYNTHESIS
  `define ASIC_SYNTHESIS
 `endif
`endif

module zeroriscy_cs_registers
#(
  parameter N_EXT_CNT    = 0
)
(
  // Clock and Reset
  input  logic       clk,
  input  logic       rst_n,

  // Core and Cluster ID
  input  logic  [3:0] core_id_i,
  input  logic  [5:0] cluster_id_i,

  // Used for boot address
  input  logic [23:0] boot_addr_i,

  // Interface to registers (SRAM like)
  input  logic       csr_access_i,
  input  logic [11:0] csr_addr_i,
  input  logic [31:0] csr_wdata_i,
  input  logic  [1:0] csr_op_i,
  output logic [31:0] csr_rdata_o,

  // Interrupts
  output logic       m_irq_enable_o,
  output logic [31:0] mepc_o,

  input  logic [31:0] pc_if_i,
  input  logic [31:0] pc_id_i,

  input  logic       csr_save_if_i,
  input  logic       csr_save_id_i,
  input  logic       csr_restore_mret_i,

  input  logic [5:0]  csr_cause_i,
  input  logic       csr_save_cause_i,


  // Performance Counters
  input  logic              if_valid_i,       // IF stage gives a new instruction
  input  logic              id_valid_i,       // ID stage is done
  input  logic              is_compressed_i,   // compressed instruction in ID
  input  logic              is_decoding_i,     // controller is in DECODE state

  input  logic              imiss_i,          // instruction fetch
  input  logic              pc_set_i,         // pc was set to a new value
  input  logic              jump_i,           // jump instruction seen   (j, jr, jal, jalr)
  input  logic              branch_i,         // branch instruction seen (bf, bnf)
  input  logic              branch_taken_i,    // branch was taken
  input  logic              mem_load_i,       // load from memory in this cycle
```

```
  input  logic              mem_store_i,      // store to memory in this cycle
  input  logic [N_EXT_CNT-1:0] ext_counters_i
);

  localparam N_PERF_COUNTERS = 11 + N_EXT_CNT;

`ifdef ASIC_SYNTHESIS
  localparam N_PERF_REGS     = 1;
`else
  localparam N_PERF_REGS     = N_PERF_COUNTERS;
`endif

  `define MSTATUS_UIE_BITS       0
  `define MSTATUS_SIE_BITS       1
  `define MSTATUS_MIE_BITS       3
  `define MSTATUS_UPIE_BITS      4
  `define MSTATUS_SPIE_BITS      5
  `define MSTATUS_MPIE_BITS      7
  `define MSTATUS_SPP_BITS       8
  `define MSTATUS_MPP_BITS    12:11

  typedef struct packed {
    //logic uie;      - unimplemented, hardwired to '0
    // logic sie;     - unimplemented, hardwired to '0
    // logic hie;     - unimplemented, hardwired to '0
    logic mie;
    //logic upie;    - unimplemented, hardwired to '0
    // logic spie;    - unimplemented, hardwired to '0
    // logic hpie;    - unimplemented, hardwired to '0
    logic mpie;
    // logic spp;     - unimplemented, hardwired to '0
    // logic[1:0] hpp; - unimplemented, hardwired to '0
    PrivLvl_t mpp;
  } Status_t;

  // Performance Counter Signals
  logic                      id_valid_q;
  logic [N_PERF_COUNTERS-1:0]    PCCR_in;  // input signals for each counter category
  logic [N_PERF_COUNTERS-1:0]    PCCR_inc, PCCR_inc_q; // should the counter be increased?

  logic [N_PERF_REGS-1:0] [31:0] PCCR_q, PCCR_n; // performance counters counter register
  logic [1:0]                PCMR_n, PCMR_q; // mode register, controls saturation and global enable
  logic [N_PERF_COUNTERS-1:0]    PCER_n, PCER_q; // selected counter input

  logic [31:0]                perf_rdata;
  logic [4:0]                 pccr_index;
  logic                      pccr_all_sel;
  logic                      is_pccr;
  logic                      is_pcer;
  logic                      is_pcmr;

  // CSR update logic
  logic [31:0] csr_wdata_int;
  logic [31:0] csr_rdata_int;
  logic       csr_we_int;

  // Interrupt control signals
```

```
logic [31:0] mepc_q, mepc_n;
logic [ 5:0] mcause_q, mcause_n;
Status_t mstatus_q, mstatus_n;

//////////////////////////////////////////////
//   ____ ____  ____    ____             //
//  / ___/ ___|| _ \  | _ \___  __ _   //
// | |   \___ \| |_) | | |_) / _ \/ _` |  //
// | |___ ___) |  _ <  |  _ <  __/ (_| |  //
//  \____|____/|_| \_\ |_| \_\___|\__, |  //
//                          |___/   //
//////////////////////////////////////////////

// read logic
always_comb
begin
            csr_rdata_int = '0;
  case (csr_addr_i)

    // mstatus: always M-mode, contains IE bit
    12'h300: csr_rdata_int = {
                              19'b0,
                              mstatus_q.mpp,
                              3'b0,
                              mstatus_q.mpie,
                              3'h0,
                              mstatus_q.mie,
                              3'h0
                             };
    // mtvec: machine trap-handler base address
    12'h305: csr_rdata_int = {boot_addr_i, 8'h0};
    // mepc: exception program counter
    12'h341: csr_rdata_int = mepc_q;
    // mcause: exception cause
    12'h342: csr_rdata_int = {mcause_q[5], 26'b0, mcause_q[4:0]};

    // mhartid: unique hardware thread id
    12'hF14: csr_rdata_int = {21'b0, cluster_id_i[5:0], 1'b0, core_id_i[3:0]};

                             default: ;
  endcase
end


// write logic
always_comb
begin
  mepc_n      = mepc_q;
  mstatus_n    = mstatus_q;
  mcause_n     = mcause_q;

  case (csr_addr_i)
    // mstatus: IE bit
    12'h300: if (csr_we_int) begin
        mstatus_n = '{
        mie:  csr_wdata_int[`MSTATUS_MIE_BITS],
        mpie: csr_wdata_int[`MSTATUS_MPIE_BITS],
```

```
      mpp:  PrivLvl_t'(PRIV_LVL_M)
     };
    end
    // mepc: exception program counter
    12'h341: if (csr_we_int) mepc_n = csr_wdata_int;
    // mcause
    12'h342: if (csr_we_int) mcause_n = {csr_wdata_int[31], csr_wdata_int[4:0]};
                        default: ;
  endcase

  // exception controller gets priority over other writes
  unique case (1'b1)

    csr_save_cause_i: begin
      unique case (1'b1)
        csr_save_if_i:
          mepc_n = pc_if_i;
        csr_save_id_i:
          mepc_n = pc_id_i;
        default:;
      endcase
      mstatus_n.mpie = mstatus_q.mie;
      mstatus_n.mie  = 1'b0;
      mcause_n       = csr_cause_i;
    end //csr_save_cause_i

    csr_restore_mret_i: begin //MRET
      mstatus_n.mie  = mstatus_q.mpie;
      mstatus_n.mpie = 1'b1;
    end //csr_restore_mret_i

    default:;

  endcase

end


// CSR operation logic
always_comb
begin
  csr_wdata_int = csr_wdata_i;
  csr_we_int    = 1'b1;

  unique case (csr_op_i)
    CSR_OP_WRITE: csr_wdata_int = csr_wdata_i;
    CSR_OP_SET:   csr_wdata_int = csr_wdata_i | csr_rdata_o;
    CSR_OP_CLEAR: csr_wdata_int = (~csr_wdata_i) & csr_rdata_o;

    CSR_OP_NONE: begin
      csr_wdata_int = csr_wdata_i;
      csr_we_int    = 1'b0;
    end

    default:;
  endcase
end
```

```
// output mux
always_comb
begin
  csr_rdata_o = csr_rdata_int;

  // performance counters
  if (is_pccr || is_pcer || is_pcmr)
    csr_rdata_o = perf_rdata;
end


// directly output some registers
assign m_irq_enable_o  = mstatus_q.mie;
assign mepc_o          = mepc_q;


// actual registers
always_ff @(posedge clk, negedge rst_n)
begin
  if (rst_n == 1'b0)
  begin
    mstatus_q  <= '{
          mie:  1'b0,
          mpie: 1'b0,
          mpp:  PRIV_LVL_M
        };
    mepc_q     <= '0;
    mcause_q   <= '0;
  end
  else
  begin
    // update CSRs
      mstatus_q  <= '{
            mie:  mstatus_n.mie,
            mpie: mstatus_n.mpie,
            mpp:  PRIV_LVL_M
          };
    mepc_q     <= mepc_n;
    mcause_q   <= mcause_n;
  end
end

//////////////////////////////////////////////////////////////////
//   ____          __    ____              _          //
// |  _ \ ___ _ __ / _|   / ___|___  _   _ _ __ | |_ ___ _ __  //
// | |_) / _ \ '__| |_   | |   / _ \| | | | '_ \| __/ _ \ '__| //
// |  __/  __/ |  |  _|  | |__| (_) | |_| | | | | ||  __/ |    //
// |_|   \___|_|  |_|(_)  \____\___/ \__,_|_| |_|\__\___|_|    //
//                                                  //
//////////////////////////////////////////////////////////////////

assign PCCR_in[0]  = 1'b1;                    // cycle counter
assign PCCR_in[1]  = if_valid_i;                // instruction counter
assign PCCR_in[2]  = 1'b0;                    // Reserved
assign PCCR_in[3]  = 1'b0;                    // Reserved
```

```
  assign PCCR_in[4]  = imiss_i & (~pc_set_i);        // cycles waiting for instruction fetches, excluding
jumps and branches
  assign PCCR_in[5]  = mem_load_i;                  // nr of loads
  assign PCCR_in[6]  = mem_store_i;                 // nr of stores
  assign PCCR_in[7]  = jump_i;                     // nr of jumps (unconditional)
  assign PCCR_in[8]  = branch_i;                    // nr of branches (conditional)
  assign PCCR_in[9]  = branch_taken_i;              // nr of taken branches (conditional)
  assign PCCR_in[10] = id_valid_i & is_decoding_i & is_compressed_i; // compressed instruction
counter

  // assign external performance counters
  generate
    genvar i;
    for (i = 0; i < N_EXT_CNT; i++)
    begin : g_extcounters
      assign PCCR_in[N_PERF_COUNTERS - N_EXT_CNT + i] = ext_counters_i[i];
    end
  endgenerate

  // address decoder for performance counter registers
  always_comb
  begin
    is_pccr      = 1'b0;
    is_pcmr      = 1'b0;
    is_pcer      = 1'b0;
    pccr_all_sel = 1'b0;
    pccr_index   = '0;
    perf_rdata   = '0;

    // only perform csr access if we actually care about the read data
    if (csr_access_i) begin
      unique case (csr_addr_i)
        12'h7A0: begin
          is_pcer = 1'b1;
          perf_rdata[15:0] = PCER_q;
        end
        12'h7A1: begin
          is_pcmr = 1'b1;
          perf_rdata[1:0] = PCMR_q;
        end
        12'h79F: begin
          is_pccr = 1'b1;
          pccr_all_sel = 1'b1;
        end
        default:;
      endcase

      // look for 780 to 79F, Performance Counter Counter Registers
      if (csr_addr_i[11:5] == 7'b0111100) begin
        is_pccr     = 1'b1;

        pccr_index = csr_addr_i[4:0];
`ifdef  ASIC_SYNTHESIS
        perf_rdata = PCCR_q[0];
`else
        perf_rdata = PCCR_q[csr_addr_i[4:0]];
`endif
```

```
    end
   end
  end


  // performance counter counter update logic
`ifdef ASIC_SYNTHESIS
  // for synthesis we just have one performance counter register
  assign PCCR_inc[0] = (|(PCCR_in & PCER_q)) & PCMR_q[0];

  always_comb
  begin
   PCCR_n[0]   = PCCR_q[0];

   if ((PCCR_inc_q[0] == 1'b1) && ((PCCR_q[0] != 32'hFFFFFFFF) || (PCMR_q[1] == 1'b0)))
    PCCR_n[0] = PCCR_q[0] + 1;

   if (is_pccr == 1'b1) begin
    unique case (csr_op_i)
     CSR_OP_NONE:   ;
     CSR_OP_WRITE:  PCCR_n[0] = csr_wdata_i;
     CSR_OP_SET:    PCCR_n[0] = csr_wdata_i | PCCR_q[0];
     CSR_OP_CLEAR:  PCCR_n[0] = csr_wdata_i & ~(PCCR_q[0]);
    endcase
   end
  end
`else
  always_comb
  begin
   for(int i = 0; i < N_PERF_COUNTERS; i++)
   begin : PERF_CNT_INC
    PCCR_inc[i] = PCCR_in[i] & PCER_q[i] & PCMR_q[0];

    PCCR_n[i]   = PCCR_q[i];

    if ((PCCR_inc_q[i] == 1'b1) && ((PCCR_q[i] != 32'hFFFFFFFF) || (PCMR_q[1] == 1'b0)))
     PCCR_n[i] = PCCR_q[i] + 1;

    if (is_pccr == 1'b1 && (pccr_all_sel == 1'b1 || pccr_index == i)) begin
     unique case (csr_op_i)
      CSR_OP_NONE:   ;
      CSR_OP_WRITE:  PCCR_n[i] = csr_wdata_i;
      CSR_OP_SET:    PCCR_n[i] = csr_wdata_i | PCCR_q[i];
      CSR_OP_CLEAR:  PCCR_n[i] = csr_wdata_i & ~(PCCR_q[i]);
     endcase
    end
   end
  end
`endif

  // update PCMR and PCER
  always_comb
  begin
   PCMR_n = PCMR_q;
   PCER_n = PCER_q;

   if (is_pcmr) begin
```

```
    unique case (csr_op_i)
      CSR_OP_NONE:   ;
      CSR_OP_WRITE:  PCMR_n = csr_wdata_i[1:0];
      CSR_OP_SET:    PCMR_n = csr_wdata_i[1:0] | PCMR_q;
      CSR_OP_CLEAR:  PCMR_n = csr_wdata_i[1:0] & ~(PCMR_q);
    endcase
  end

  if (is_pcer) begin
    unique case (csr_op_i)
      CSR_OP_NONE:   ;
      CSR_OP_WRITE:  PCER_n = csr_wdata_i[N_PERF_COUNTERS-1:0];
      CSR_OP_SET:    PCER_n = csr_wdata_i[N_PERF_COUNTERS-1:0] | PCER_q;
      CSR_OP_CLEAR:  PCER_n = csr_wdata_i[N_PERF_COUNTERS-1:0] & ~(PCER_q);
    endcase
  end
end

// Performance Counter Registers
always_ff @(posedge clk, negedge rst_n)
begin
  if (rst_n == 1'b0)
  begin
    id_valid_q <= 1'b0;

    PCER_q <= '0;
    PCMR_q <= 2'h3;

    for(int i = 0; i < N_PERF_REGS; i++)
    begin
      PCCR_q[i]     <= '0;
      PCCR_inc_q[i] <= '0;
    end
  end
  else
  begin
    id_valid_q <= id_valid_i;

    PCER_q <= PCER_n;
    PCMR_q <= PCMR_n;

    for(int i = 0; i < N_PERF_REGS; i++)
    begin
      PCCR_q[i]     <= PCCR_n[i];
      PCCR_inc_q[i] <= PCCR_inc[i];
    end

  end
end

Endmodule

`include "zeroriscy_config.sv"

import zeroriscy_defines::*;

module zeroriscy_debug_unit
```

```
#(
   parameter REG_ADDR_WIDTH      = 5
)
(
  input logic        clk,
  input logic        rst_n,

  // Debug Interface
  input  logic       debug_req_i,
  output logic       debug_gnt_o,
  output logic       debug_rvalid_o,
  input  logic [14:0] debug_addr_i,
  input  logic       debug_we_i,
  input  logic [31:0] debug_wdata_i,
  output logic [31:0] debug_rdata_o,

  output logic       debug_halted_o,
  input  logic       debug_halt_i,
  input  logic       debug_resume_i,

  // signals to core
  output logic [DBG_SETS_W-1:0] settings_o,

  input  logic       trap_i,      // trap found, need to stop the core now
  input  logic [5:0]  exc_cause_i, // if it was a trap, then the exception controller knows more
  output logic       stall_o,     // after we got control, we control the stall signal
  output logic       dbg_req_o,
  input  logic       dbg_ack_i,

  // register file read port
  output logic       regfile_rreq_o,
  output logic [(REG_ADDR_WIDTH-1):0] regfile_raddr_o,
  input  logic [31:0] regfile_rdata_i,

  // register file write port
  output logic       regfile_wreq_o,
  output logic [(REG_ADDR_WIDTH-1):0] regfile_waddr_o,
  output logic [31:0] regfile_wdata_o,

  // CSR read/write port
  output logic       csr_req_o,
  output logic [11:0] csr_addr_o,
  output logic       csr_we_o,
  output logic [31:0] csr_wdata_o,
  input  logic [31:0] csr_rdata_i,

  // Signals for PPC & NPC register
  input  logic [31:0] pc_if_i,
  input  logic [31:0] pc_id_i,

  input  logic       instr_valid_id_i,

  input  logic       sleeping_i,

  output logic       jump_req_o,
  output logic [31:0] jump_addr_o
);
```

```
enum logic [2:0] {RD_NONE, RD_CSR, RD_GPR, RD_DBGA, RD_DBGS} rdata_sel_q, rdata_sel_n;

enum logic [0:0] {FIRST, SECOND} state_q, state_n;

logic [DBG_SETS_W-1:0] settings_q, settings_n;

// for timing critical register file access we need to keep those in FFs
logic [14:0] addr_q;
logic [31:0] wdata_q; // mainly for jumps
logic       regfile_rreq_q, regfile_rreq_n;
logic       jump_req_q, jump_req_n;

// not timing critical
logic       csr_req_q, csr_req_n;
logic       regfile_wreq;


enum logic [1:0] {RUNNING, HALT_REQ, HALT} stall_cs, stall_ns;
logic [31:0] dbg_rdata;
logic       dbg_resume;
logic       dbg_halt;
logic [5:0]  dbg_cause_q, dbg_cause_n;
logic       dbg_ssth_q,  dbg_ssth_n;

logic       ssth_clear;


// ppc/npc tracking
logic [31:0] ppc_int, npc_int;


// address decoding, write and read controller
always_comb
begin
  rdata_sel_n    = RD_NONE;
  state_n        = FIRST;

  debug_gnt_o    = 1'b0;

  regfile_rreq_n = 1'b0;
  regfile_wreq   = 1'b0;
  csr_req_n      = 1'b0;
  csr_we_o       = 1'b0;
  jump_req_n     = 1'b0;

  dbg_resume     = 1'b0;
  dbg_halt       = 1'b0;
  settings_n     = settings_q;

  ssth_clear     = 1'b0;

  if (debug_req_i) begin
    if (debug_we_i) begin
      //-----------------------------------------------------------------------------
      // write access
      //-----------------------------------------------------------------------------
```

```
if (debug_addr_i[14]) begin
  // CSR access
  if (state_q == FIRST) begin
    // only grant in second cycle, address and data have been latched by then
    debug_gnt_o = 1'b0;
    state_n     = SECOND;

    if (debug_halted_o) begin
      // access to internal registers are only allowed when the core is halted
      csr_req_n = 1'b1;
    end
  end else begin
    debug_gnt_o = 1'b1; // grant it even when invalid access to not block
    state_n     = FIRST;
    csr_we_o    = 1'b1;
  end
end else begin
  // non-CSR access
  unique case (debug_addr_i[13:8])
    6'b00_0000: begin // Debug Registers, always accessible
      debug_gnt_o = 1'b1;

      unique case (debug_addr_i[6:2])
        5'b0_0000: begin // DBG_CTRL
          if (debug_wdata_i[16]) begin
            // HALT set
            if (~debug_halted_o) begin
              // not halt, so STOP
              dbg_halt = 1'b1;
            end
          end else begin
            // RESUME set
            if (debug_halted_o) begin
              dbg_resume = 1'b1;
            end
          end

          settings_n[DBG_SETS_SSTE] = debug_wdata_i[0];
        end
        5'b0_0001: begin // DBG_HIT
          ssth_clear = debug_wdata_i[0];
        end
        5'b0_0010: begin // DBG_IE
          settings_n[DBG_SETS_ECALL] = debug_wdata_i[11];
          settings_n[DBG_SETS_ELSU]  = debug_wdata_i[7] | debug_wdata_i[5];
          settings_n[DBG_SETS_EBRK]  = debug_wdata_i[3];
          settings_n[DBG_SETS_EILL]  = debug_wdata_i[2];
        end
        default:;
      endcase
    end

    6'b10_0000: begin // Debug Registers, only accessible when in debug
      debug_gnt_o = 1'b1; // grant it even when invalid access to not block

      if (debug_halted_o) begin
        unique case (debug_addr_i[6:2])
```

```
          5'b0_0000: jump_req_n = 1'b1; // DNPC
          default:;
        endcase
      end
    end

    6'b00_0100: begin // General-Purpose Registers
      debug_gnt_o = 1'b1; // grant it even when invalid access to not block

      if (debug_halted_o) begin
        regfile_wreq = 1'b1;
      end
    end

    default: debug_gnt_o = 1'b1; // grant it even when invalid access to not block
  endcase
 end
end else begin
 //-----------------------------------------------------------------------------
 // read access
 //-----------------------------------------------------------------------------
 if (debug_addr_i[14]) begin
   debug_gnt_o = 1'b1; // grant it even when invalid access to not block

   // CSR access
   if (debug_halted_o) begin
     // access to internal registers are only allowed when the core is halted
     csr_req_n   = 1'b1;
     rdata_sel_n = RD_CSR;
   end
 end else begin
   // non-CSR access
   unique case (debug_addr_i[13:8])
     6'b00_0000: begin // Debug Registers, always accessible
       debug_gnt_o = 1'b1;

       rdata_sel_n = RD_DBGA;
     end

     6'b10_0000: begin // Debug Registers, only accessible when in debug
       debug_gnt_o = 1'b1; // grant it even when invalid access to not block

       rdata_sel_n = RD_DBGS;
     end

     6'b00_0100: begin // General-Purpose Registers
       debug_gnt_o = 1'b1; // grant it even when invalid access to not block

       if (debug_halted_o) begin
         regfile_rreq_n = 1'b1;
         rdata_sel_n    = RD_GPR;
       end
     end

     default: debug_gnt_o = 1'b1; // grant it even when invalid access to not block
   endcase
 end
```

```
    end
  end
end

//-----------------------------------------------------------------------------
// debug register read access
//
// Since those are combinational, we can do it in the cycle where we set
// rvalid. The address has been latched into addr_q
//-----------------------------------------------------------------------------
always_comb
begin
  dbg_rdata = '0;

  case (rdata_sel_q)
    RD_DBGA: begin
      unique case (addr_q[6:2])
        5'h00: dbg_rdata[31:0] = {15'b0, debug_halted_o, 15'b0, settings_q[DBG_SETS_SSTE]}; //
DBG_CTRL
        5'h01: dbg_rdata[31:0] = {15'b0, sleeping_i, 15'b0, dbg_ssth_q}; // DBG_HIT
        5'h02: begin // DBG_IE
          dbg_rdata[31:16] = '0;
          dbg_rdata[15:12] = '0;
          dbg_rdata[11]    = settings_q[DBG_SETS_ECALL];
          dbg_rdata[10: 8] = '0;
          dbg_rdata[ 7]    = settings_q[DBG_SETS_ELSU];
          dbg_rdata[ 6]    = 1'b0;
          dbg_rdata[ 5]    = settings_q[DBG_SETS_ELSU];
          dbg_rdata[ 4]    = 1'b0;
          dbg_rdata[ 3]    = settings_q[DBG_SETS_EBRK];
          dbg_rdata[ 2]    = settings_q[DBG_SETS_EILL];
          dbg_rdata[ 1: 0] = '0;
        end
        5'h03: dbg_rdata = {dbg_cause_q[5], 26'b0, dbg_cause_q[4:0]}; // DBG_CAUSE
        5'h10: dbg_rdata = '0; // DBG_BPCTRL0
        5'h12: dbg_rdata = '0; // DBG_BPCTRL1
        5'h14: dbg_rdata = '0; // DBG_BPCTRL2
        5'h16: dbg_rdata = '0; // DBG_BPCTRL3
        5'h18: dbg_rdata = '0; // DBG_BPCTRL4
        5'h1A: dbg_rdata = '0; // DBG_BPCTRL5
        5'h1C: dbg_rdata = '0; // DBG_BPCTRL6
        5'h1E: dbg_rdata = '0; // DBG_BPCTRL7
        default:;
      endcase
    end

    RD_DBGS: begin
      unique case (addr_q[2:2])
        1'b0: dbg_rdata = npc_int; // DBG_NPC
        1'b1: dbg_rdata = ppc_int; // DBG_PPC
        default:;
      endcase
    end

    default:;
  endcase
end
```

```
//-----------------------------------------------------------------------------
// read data mux
//-----------------------------------------------------------------------------
always_comb
begin
  debug_rdata_o = '0;

  case (rdata_sel_q)
    RD_CSR:  debug_rdata_o = csr_rdata_i;
    RD_GPR:  debug_rdata_o = regfile_rdata_i;
    RD_DBGA: debug_rdata_o = dbg_rdata;
    RD_DBGS: debug_rdata_o = dbg_rdata;
                          default: ;
  endcase
end

//-----------------------------------------------------------------------------
// rvalid generation
//-----------------------------------------------------------------------------
always_ff @(posedge clk, negedge rst_n)
begin
  if (~rst_n) begin
    debug_rvalid_o <= 1'b0;
  end else begin
    debug_rvalid_o <= debug_gnt_o; // always give the rvalid one cycle after gnt
  end
end

//-----------------------------------------------------------------------------
// stall control
//-----------------------------------------------------------------------------
always_comb
begin
  stall_ns       = stall_cs;
  dbg_req_o      = 1'b0;
  stall_o        = 1'b0;
  debug_halted_o = 1'b0;
  dbg_cause_n    = dbg_cause_q;
  dbg_ssth_n     = dbg_ssth_q;

  case (stall_cs)
    RUNNING: begin
      dbg_ssth_n = 1'b0;

      if (dbg_halt | debug_halt_i | trap_i) begin
        dbg_req_o   = 1'b1;
        stall_ns    = HALT_REQ;

        if (trap_i) begin
          if (settings_q[DBG_SETS_SSTE])
            dbg_ssth_n = 1'b1;

          dbg_cause_n = exc_cause_i;
        end else begin
          dbg_cause_n = DBG_CAUSE_HALT;
        end
```

```
      end
    end

    HALT_REQ: begin
      dbg_req_o = 1'b1;

      if (dbg_ack_i)
        stall_ns = HALT;

      if (dbg_resume | debug_resume_i)
        stall_ns = RUNNING;
    end

    HALT: begin
      stall_o       = 1'b1;
      debug_halted_o = 1'b1;

      if (dbg_resume | debug_resume_i) begin
        stall_ns = RUNNING;
        stall_o  = 1'b0;
      end
    end
  endcase

  if (ssth_clear)
    dbg_ssth_n = 1'b0;
end

always_ff @(posedge clk, negedge rst_n)
begin
  if (~rst_n) begin
    stall_cs    <= RUNNING;
    dbg_cause_q <= DBG_CAUSE_HALT;
    dbg_ssth_q  <= 1'b0;
  end else begin
    stall_cs    <= stall_ns;
    dbg_cause_q <= dbg_cause_n;
    dbg_ssth_q  <= dbg_ssth_n;
  end
end

//-----------------------------------------------------------------------------
// NPC/PPC selection
//-----------------------------------------------------------------------------

assign ppc_int = pc_id_i;
assign npc_int = pc_if_i;

always_ff @(posedge clk, negedge rst_n)
begin
  if (~rst_n) begin

    addr_q            <= '0;
    wdata_q           <= '0;
    state_q           <= FIRST;
    rdata_sel_q       <= RD_NONE;
    regfile_rreq_q    <= 1'b0;
```

```
    csr_req_q         <= 1'b0;
    jump_req_q        <= 1'b0;

    settings_q        <= 1'b0;
  end else begin

    // settings
    settings_q        <= settings_n;

    // for the actual interface
    if (debug_req_i) begin
      addr_q  <= debug_addr_i;
      wdata_q <= debug_wdata_i;
      state_q <= state_n;
    end

    if (debug_req_i | debug_rvalid_o) begin
      // wait for either req or rvalid to set those FFs
      // This makes sure that they are only triggered once when there is
      // only one request, and the FFs can be properly clock gated otherwise
      regfile_rreq_q <= regfile_rreq_n;
      csr_req_q      <= csr_req_n;
      jump_req_q     <= jump_req_n;
      rdata_sel_q    <= rdata_sel_n;
    end
  end
end

assign regfile_rreq_o  = regfile_rreq_q;
assign regfile_raddr_o = addr_q[6:2];

assign regfile_wreq_o  = regfile_wreq;
assign regfile_waddr_o = debug_addr_i[6:2];
assign regfile_wdata_o = debug_wdata_i;

assign csr_req_o       = csr_req_q;
assign csr_addr_o      = addr_q[13:2];
assign csr_wdata_o     = wdata_q;

assign jump_req_o      = jump_req_q;
assign jump_addr_o     = wdata_q;

assign settings_o      = settings_q;

//-----------------------------------------------------------------------------
// Assertions
//-----------------------------------------------------------------------------
`ifndef VERILATOR
// check that no registers are accessed when we are not in debug mode
assert property (
  @(posedge clk) (debug_req_i) |-> ((debug_halted_o == 1'b1) ||
                                    ((debug_addr_i[14] != 1'b1) &&
                                     (debug_addr_i[13:7] != 5'b0_1001)  &&
                                     (debug_addr_i[13:7] != 5'b0_1000)) ) )
  else $warning("Trying to access internal debug registers while core is not stalled");

// check that all accesses are word-aligned
```

```
  assert property (
    @(posedge clk) (debug_req_i) |-> (debug_addr_i[1:0] == 2'b00) );
`endif
endmodule // debug_unit

`include "zeroriscy_config.sv"

import zeroriscy_defines::*;

module zeroriscy_decoder
#(
  parameter RV32M      = 1
)
(
  // singals running to/from controller
  input  logic       deassert_we_i,          // deassert we, we are stalled or not active
  input  logic       data_misaligned_i,      // misaligned data load/store in progress
  input  logic       branch_mux_i,
  input  logic       jump_mux_i,
  output logic       illegal_insn_o,         // illegal instruction encountered
  output logic       ebrk_insn_o,            // trap instruction encountered
  output logic       mret_insn_o,            // return from exception instruction encountered
  output logic       ecall_insn_o,           // environment call (syscall) instruction encountered
  output logic       pipe_flush_o,           // pipeline flush is requested

  // from IF/ID pipeline
  input  logic [31:0] instr_rdata_i,         // instruction read from instr memory/cache
  input  logic       illegal_c_insn_i,       // compressed instruction decode failed

  // ALU signals
  output logic [ALU_OP_WIDTH-1:0] alu_operator_o, // ALU operation selection
  output logic [2:0]  alu_op_a_mux_sel_o,     // operand a selection: reg value, PC, immediate or zero
  output logic [2:0]  alu_op_b_mux_sel_o,     // oNOperand b selection: reg value or immediate

  output logic [0:0]  imm_a_mux_sel_o,        // immediate selection for operand a
  output logic [3:0]  imm_b_mux_sel_o,        // immediate selection for operand b

  // MUL, DIV related control signals
  output logic       mult_int_en_o,          // perform integer multiplication
  output logic       div_int_en_o,           // perform integer division or reminder
  output logic [1:0]  multdiv_operator_o,
  output logic [1:0]  multdiv_signed_mode_o,
  // register file related signals
  output logic       regfile_we_o,           // write enable for regfile

  // CSR manipulation
  output logic       csr_access_o,           // access to CSR
  output logic [1:0]  csr_op_o,              // operation to perform on CSR
  output logic       csr_status_o,           // access to xstatus CSR

  // LD/ST unit signals
  output logic       data_req_o,             // start transaction to data memory
  output logic       data_we_o,              // data memory write enable
  output logic [1:0]  data_type_o,           // data type on data memory: byte, half word or word
  output logic       data_sign_extension_o,  // sign extension on read data from data memory
  output logic [1:0]  data_reg_offset_o,     // offset in byte inside register for stores
  output logic       data_load_event_o,      // data request is in the special event range
```

```
  // jump/branches
  output logic       jump_in_id_o,          // jump is being calculated in ALU
  output logic       branch_in_id_o
);

  // write enable/request control
  logic      regfile_we;
  logic      data_req;

  logic      mult_int_en;
  logic      div_int_en;
  logic      branch_in_id;
  logic      jump_in_id;

  logic [1:0] csr_op;
  logic      csr_illegal;

  ////////////////////////////////////////////////
  //   ____                  _          //
  //  |  _ \ ___  ___ ___   __| | ___ _ __  //
  //  | | | |/ _ \/ __/ _ \ / _` |/ _ \ '__| //
  //  | |_| |  __/ (_| (_) | (_| |  __/ |    //
  //  |____/ \___|\___\___/ \__,_|\___|_|    //
  //                                  //
  ////////////////////////////////////////////////

  always_comb
  begin
    jump_in_id                = 1'b0;
    branch_in_id              = 1'b0;
    alu_operator_o            = ALU_SLTU;
    alu_op_a_mux_sel_o        = OP_A_REGA_OR_FWD;
    alu_op_b_mux_sel_o        = OP_B_REGB_OR_FWD;

    imm_a_mux_sel_o           = IMMA_ZERO;
    imm_b_mux_sel_o           = IMMB_I;

    mult_int_en               = 1'b0;
    div_int_en                = 1'b0;
    multdiv_operator_o        = MD_OP_MULL;
    multdiv_signed_mode_o     = 2'b00;

    regfile_we                = 1'b0;

    csr_access_o              = 1'b0;
    csr_status_o              = 1'b0;
    csr_illegal               = 1'b0;
    csr_op                    = CSR_OP_NONE;

    data_we_o                 = 1'b0;
    data_type_o               = 2'b00;
    data_sign_extension_o     = 1'b0;
    data_reg_offset_o         = 2'b00;
    data_req                  = 1'b0;
    data_load_event_o         = 1'b0;
```

```
illegal_insn_o          = 1'b0;
ebrk_insn_o             = 1'b0;
mret_insn_o             = 1'b0;
ecall_insn_o            = 1'b0;
pipe_flush_o            = 1'b0;

unique case (instr_rdata_i[6:0])

  ////////////////////////////////////////
  //      _ _   _ __  __ ____  ____    //
  //     | | | | |  \/  |  _ \/ ___|  //
  //  _  | | | | | |\/| | |_) \___ \  //
  // | |_| | |_| | |  | |  __/ ___) | //
  //  \___/ \___/|_|  |_|_|   |____/  //
  //                                  //
  ////////////////////////////////////////

  OPCODE_JAL: begin   // Jump and Link
    jump_in_id          = 1'b1;
    if(jump_mux_i) begin
      // Calculate jump target
      alu_op_a_mux_sel_o  = OP_A_CURRPC;
      alu_op_b_mux_sel_o  = OP_B_IMM;
      imm_b_mux_sel_o     = IMMB_UJ;
      alu_operator_o      = ALU_ADD;
      regfile_we          = 1'b0;
    end else begin
      // Calculate and store PC+4
      alu_op_a_mux_sel_o  = OP_A_CURRPC;
      alu_op_b_mux_sel_o  = OP_B_IMM;
      imm_b_mux_sel_o     = IMMB_PCINCR;
      alu_operator_o      = ALU_ADD;
      regfile_we          = 1'b1;
    end
  end

  OPCODE_JALR: begin  // Jump and Link Register
    jump_in_id          = 1'b1;
    if(jump_mux_i) begin
      // Calculate jump target
      alu_op_a_mux_sel_o  = OP_A_REGA_OR_FWD;
      alu_op_b_mux_sel_o  = OP_B_IMM;
      imm_b_mux_sel_o     = IMMB_I;
      alu_operator_o      = ALU_ADD;
      regfile_we          = 1'b0;
    end else begin
      // Calculate and store PC+4
      alu_op_a_mux_sel_o  = OP_A_CURRPC;
      alu_op_b_mux_sel_o  = OP_B_IMM;
      imm_b_mux_sel_o     = IMMB_PCINCR;
      alu_operator_o      = ALU_ADD;
      regfile_we          = 1'b1;
    end
    if (instr_rdata_i[14:12] != 3'b0) begin
      jump_in_id      = 1'b0;
      regfile_we      = 1'b0;
```

```
    illegal_insn_o   = 1'b1;
  end

end

OPCODE_BRANCH: begin // Branch

  branch_in_id         = 1'b1;

  if (branch_mux_i)
  begin
    unique case (instr_rdata_i[14:12])
      3'b000: alu_operator_o = ALU_EQ;
      3'b001: alu_operator_o = ALU_NE;
      3'b100: alu_operator_o = ALU_LTS;
      3'b101: alu_operator_o = ALU_GES;
      3'b110: alu_operator_o = ALU_LTU;
      3'b111: alu_operator_o = ALU_GEU;
      default: begin
        illegal_insn_o = 1'b1;
      end
    endcase
  end
  else begin
    // Calculate jump target in EX
    alu_op_a_mux_sel_o  = OP_A_CURRPC;
    alu_op_b_mux_sel_o  = OP_B_IMM;
    imm_b_mux_sel_o     = IMMB_SB;
    alu_operator_o      = ALU_ADD;
    regfile_we          = 1'b0;
  end

end


//////////////////////////////////
//  _     ____    ______ _____  //
// | |   |  _ \  / / ___|_   _| //
// | |   | | | |/ /\___ \ | |   //
// | |___| |_| / /  ___) | | |   //
// |_____|____/_/  |____/ |_|   //
//                              //
//////////////////////////////////

OPCODE_STORE: begin
  data_req       = 1'b1;
  data_we_o      = 1'b1;
  alu_operator_o = ALU_ADD;

  if (instr_rdata_i[14] == 1'b0) begin
    // offset from immediate
    imm_b_mux_sel_o     = IMMB_S;
    alu_op_b_mux_sel_o  = OP_B_IMM;
  end
  // Register offset is illegal since no register c available
  else begin
    data_req       = 1'b0;
```

```
    data_we_o      = 1'b0;
    illegal_insn_o = 1'b1;
  end

  // store size
  unique case (instr_rdata_i[13:12])
    2'b00: data_type_o = 2'b10; // SB
    2'b01: data_type_o = 2'b01; // SH
    2'b10: data_type_o = 2'b00; // SW
    default: begin
      data_req       = 1'b0;
      data_we_o      = 1'b0;
      illegal_insn_o = 1'b1;
    end
  endcase
end

OPCODE_LOAD: begin
  data_req        = 1'b1;
  regfile_we      = 1'b1;
  data_type_o     = 2'b00;

  // offset from immediate
  alu_operator_o      = ALU_ADD;
  alu_op_b_mux_sel_o  = OP_B_IMM;
  imm_b_mux_sel_o     = IMMB_I;


  // sign/zero extension
  data_sign_extension_o = ~instr_rdata_i[14];

  // load size
  unique case (instr_rdata_i[13:12])
    2'b00:   data_type_o = 2'b10; // LB
    2'b01:   data_type_o = 2'b01; // LH
    2'b10:   data_type_o = 2'b00; // LW
    default: data_type_o = 2'b00; // illegal or reg-reg
  endcase

  // reg-reg load (different encoding)
  if (instr_rdata_i[14:12] == 3'b111) begin
    // offset from RS2
    alu_op_b_mux_sel_o = OP_B_REGB_OR_FWD;

    // sign/zero extension
    data_sign_extension_o = ~instr_rdata_i[30];

    // load size
    unique case (instr_rdata_i[31:25])
      7'b0000_000,
      7'b0100_000: data_type_o = 2'b10; // LB, LBU
      7'b0001_000,
      7'b0101_000: data_type_o = 2'b01; // LH, LHU
      7'b0010_000: data_type_o = 2'b00; // LW
      default: begin
        illegal_insn_o = 1'b1;
      end
```

```
    endcase
  end

  // special p.elw (event load)
  if (instr_rdata_i[14:12] == 3'b110)
    data_load_event_o = 1'b1;

  if (instr_rdata_i[14:12] == 3'b011) begin
    // LD -> RV64 only
    illegal_insn_o = 1'b1;
  end
end


//////////////////////////
//     _    _    _   _  //
//    / \  | |  | | | | //
//   / _ \ | |  | | | | //
//  / ___ \| |__| |_| | //
// /_/   \_\_____\___/  //
//                      //
//////////////////////////

OPCODE_LUI: begin  // Load Upper Immediate
  alu_op_a_mux_sel_o  = OP_A_IMM;
  alu_op_b_mux_sel_o  = OP_B_IMM;
  imm_a_mux_sel_o     = IMMA_ZERO;
  imm_b_mux_sel_o     = IMMB_U;
  alu_operator_o      = ALU_ADD;
  regfile_we          = 1'b1;
end

OPCODE_AUIPC: begin  // Add Upper Immediate to PC
  alu_op_a_mux_sel_o  = OP_A_CURRPC;
  alu_op_b_mux_sel_o  = OP_B_IMM;
  imm_b_mux_sel_o     = IMMB_U;
  alu_operator_o      = ALU_ADD;
  regfile_we          = 1'b1;
end

OPCODE_OPIMM: begin // Register-Immediate ALU Operations
  alu_op_b_mux_sel_o  = OP_B_IMM;
  imm_b_mux_sel_o     = IMMB_I;
  regfile_we          = 1'b1;

  unique case (instr_rdata_i[14:12])
    3'b000: alu_operator_o = ALU_ADD;  // Add Immediate
    3'b010: alu_operator_o = ALU_SLTS; // Set to one if Lower Than Immediate
    3'b011: alu_operator_o = ALU_SLTU; // Set to one if Lower Than Immediate Unsigned
    3'b100: alu_operator_o = ALU_XOR;  // Exclusive Or with Immediate
    3'b110: alu_operator_o = ALU_OR;   // Or with Immediate
    3'b111: alu_operator_o = ALU_AND;  // And with Immediate

    3'b001: begin
      alu_operator_o = ALU_SLL;  // Shift Left Logical by Immediate
      if (instr_rdata_i[31:25] != 7'b0)
        illegal_insn_o = 1'b1;
```

```
    end

    3'b101: begin
      if (instr_rdata_i[31:25] == 7'b0)
        alu_operator_o = ALU_SRL;  // Shift Right Logical by Immediate
      else if (instr_rdata_i[31:25] == 7'b010_0000)
        alu_operator_o = ALU_SRA;  // Shift Right Arithmetically by Immediate
      else
        illegal_insn_o = 1'b1;
    end

    default: illegal_insn_o = 1'b1;
  endcase
end

OPCODE_OP: begin  // Register-Register ALU operation
  regfile_we     = 1'b1;

  if (instr_rdata_i[31]) begin
    illegal_insn_o = 1'b1;
  end
  else
  begin // non bit-manipulation instructions

    if (~instr_rdata_i[28])

    unique case ({instr_rdata_i[30:25], instr_rdata_i[14:12]})
      // RV32I ALU operations
      {6'b00_0000, 3'b000}: alu_operator_o = ALU_ADD;   // Add
      {6'b10_0000, 3'b000}: alu_operator_o = ALU_SUB;   // Sub
      {6'b00_0000, 3'b010}: alu_operator_o = ALU_SLTS;  // Set Lower Than
      {6'b00_0000, 3'b011}: alu_operator_o = ALU_SLTU;  // Set Lower Than Unsigned
      {6'b00_0000, 3'b100}: alu_operator_o = ALU_XOR;   // Xor
      {6'b00_0000, 3'b110}: alu_operator_o = ALU_OR;    // Or
      {6'b00_0000, 3'b111}: alu_operator_o = ALU_AND;   // And
      {6'b00_0000, 3'b001}: alu_operator_o = ALU_SLL;   // Shift Left Logical
      {6'b00_0000, 3'b101}: alu_operator_o = ALU_SRL;   // Shift Right Logical
      {6'b10_0000, 3'b101}: alu_operator_o = ALU_SRA;   // Shift Right Arithmetic

      // supported RV32M instructions
      {6'b00_0001, 3'b000}: begin // mul
         alu_operator_o       = ALU_ADD;
         multdiv_operator_o    = MD_OP_MULL;
         mult_int_en          = 1'b1;
         multdiv_signed_mode_o = 2'b00;
         illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      {6'b00_0001, 3'b001}: begin // mulh
         alu_operator_o       = ALU_ADD;
         multdiv_operator_o    = MD_OP_MULH;
         mult_int_en          = 1'b1;
         multdiv_signed_mode_o = 2'b11;
         illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      {6'b00_0001, 3'b010}: begin // mulhsu
         alu_operator_o       = ALU_ADD;
         multdiv_operator_o    = MD_OP_MULH;
```

```
        mult_int_en          = 1'b1;
        multdiv_signed_mode_o = 2'b01;
        illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      {6'b00_0001, 3'b011}: begin // mulhu
        alu_operator_o       = ALU_ADD;
        multdiv_operator_o    = MD_OP_MULH;
        mult_int_en          = 1'b1;
        multdiv_signed_mode_o = 2'b00;
        illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      {6'b00_0001, 3'b100}: begin // div
       alu_operator_o       = ALU_ADD;
       multdiv_operator_o    = MD_OP_DIV;
       div_int_en           = 1'b1;
       multdiv_signed_mode_o = 2'b11;
       illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      {6'b00_0001, 3'b101}: begin // divu
       alu_operator_o       = ALU_ADD;
       multdiv_operator_o    = MD_OP_DIV;
       div_int_en           = 1'b1;
       multdiv_signed_mode_o = 2'b00;
       illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      {6'b00_0001, 3'b110}: begin // rem
       alu_operator_o       = ALU_ADD;
       multdiv_operator_o    = MD_OP_REM;
       div_int_en           = 1'b1;
       multdiv_signed_mode_o = 2'b11;
       illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      {6'b00_0001, 3'b111}: begin // remu
       alu_operator_o       = ALU_ADD;
       multdiv_operator_o    = MD_OP_REM;
       div_int_en           = 1'b1;
       multdiv_signed_mode_o = 2'b00;
       illegal_insn_o       = RV32M ? 1'b0 : 1'b1;
      end
      default: begin
       illegal_insn_o = 1'b1;
      end
    endcase
  end
end
```

```
/////////////////////////////////////////////////
//  ____  ____  _____ ____ ___   _   _     //
// / ___|| _ \| ____/ ___|_ _|  / \  | |    //
// \___ \| |_) |  _|| |    | |  / _ \ | |    //
//  ___) |  __/| |__| |___ | | / ___ \| |___  //
// |____/|_|   |_____\____|___/_/   \_\_____| //
//                                           //
/////////////////////////////////////////////////
```

```
OPCODE_SYSTEM: begin
 if (instr_rdata_i[14:12] == 3'b000)
 begin
  // non CSR related SYSTEM instructions
  unique case (instr_rdata_i[31:20])
   12'h000:  // ECALL
   begin
    // environment (system) call
    ecall_insn_o = 1'b1;
   end

   12'h001:  // ebreak
   begin
    // debugger trap
    ebrk_insn_o = 1'b1;
   end

   12'h302:  // mret
   begin
    mret_insn_o = 1'b1;
   end

   12'h105:  // wfi
   begin
    // flush pipeline
    pipe_flush_o = 1'b1;
   end

   default:
   begin
    illegal_insn_o = 1'b1;
   end
  endcase
 end
 else
 begin
  // instruction to read/modify CSR
  csr_access_o       = 1'b1;
  regfile_we         = 1'b1;
  alu_op_b_mux_sel_o  = OP_B_IMM;
  imm_a_mux_sel_o     = IMMA_Z;
  imm_b_mux_sel_o     = IMMB_I;    // CSR address is encoded in I imm

  if (instr_rdata_i[14] == 1'b1) begin
   // rs1 field is used as immediate
   alu_op_a_mux_sel_o = OP_A_IMM;
  end else begin
   alu_op_a_mux_sel_o = OP_A_REGA_OR_FWD;
  end

  unique case (instr_rdata_i[13:12])
   2'b01:   csr_op   = CSR_OP_WRITE;
   2'b10:   csr_op   = CSR_OP_SET;
   2'b11:   csr_op   = CSR_OP_CLEAR;
   default: csr_illegal = 1'b1;
  endcase
```

```
        if(~csr_illegal)
          if (instr_rdata_i[31:20] == 12'h300)
            //access to mstatus
            csr_status_o = 1'b1;

        illegal_insn_o = csr_illegal;

      end

    end
    default: begin
      illegal_insn_o = 1'b1;
    end
  endcase

  // make sure invalid compressed instruction causes an exception
  if (illegal_c_insn_i) begin
    illegal_insn_o = 1'b1;
  end

  // misaligned access was detected by the LSU
  // TODO: this section should eventually be moved out of the decoder
  if (data_misaligned_i == 1'b1)
  begin
    // only part of the pipeline is unstalled, make sure that the
    // correct operands are sent to the AGU
    alu_op_a_mux_sel_o  = OP_A_REGA_OR_FWD;
    alu_op_b_mux_sel_o  = OP_B_IMM;
    imm_b_mux_sel_o     = IMMB_PCINCR;

    // if prepost increments are used, we do not write back the
    // second address since the first calculated address was
    // the correct one
    regfile_we = 1'b0;

  end
end




// deassert we signals (in case of stalls)
assign regfile_we_o      = (deassert_we_i) ? 1'b0         : regfile_we;
assign mult_int_en_o     = RV32M ? ((deassert_we_i) ? 1'b0 : mult_int_en) : 1'b0;
assign div_int_en_o      = RV32M ? ((deassert_we_i) ? 1'b0 : div_int_en ) : 1'b0;
assign data_req_o        = (deassert_we_i) ? 1'b0         : data_req;
assign csr_op_o          = (deassert_we_i) ? CSR_OP_NONE   : csr_op;
assign jump_in_id_o      = (deassert_we_i) ? 1'b0         : jump_in_id;
assign branch_in_id_o    = (deassert_we_i) ? 1'b0         : branch_in_id;

endmodule // controller

`include "zeroriscy_config.sv"
```

```
import zeroriscy_defines::*;

module zeroriscy_ex_block
#(
  parameter RV32M      = 1
)
(

  input  logic       clk,
  input  logic       rst_n,

  // ALU signals from ID stage
  input  logic [ALU_OP_WIDTH-1:0] alu_operator_i,
  input  logic [1:0]            multdiv_operator_i,
  input  logic                  mult_en_i,
  input  logic                  div_en_i,

  input  logic [31:0]           alu_operand_a_i,
  input  logic [31:0]           alu_operand_b_i,

  input  logic  [1:0]           multdiv_signed_mode_i,
  input  logic [31:0]           multdiv_operand_a_i,
  input  logic [31:0]           multdiv_operand_b_i,

  output logic [31:0]           alu_adder_result_ex_o,
  output logic [31:0]           regfile_wdata_ex_o,

  // To IF: Jump and branch target and decision
  output logic [31:0]           jump_target_o,
  output logic                  branch_decision_o,

  input  logic                  lsu_en_i,

  // Stall Control
  input  logic                  lsu_ready_ex_i, // LSU is done
  output logic                  ex_ready_o      // EX stage gets new data
);

  localparam MULT_TYPE = 1; //0 is SLOW

  logic [31:0] alu_result, multdiv_result;

  logic [32:0] multdiv_alu_operand_b, multdiv_alu_operand_a;
  logic [33:0] alu_adder_result_ext;
  logic       alu_cmp_result, alu_is_equal_result;
  logic       multdiv_ready, multdiv_en_sel;
  logic       multdiv_en;

/*
  The multdiv_i output is never selected if RV32M=0
  At synthesis time, all the combinational and sequential logic
  from the multdiv_i module are eliminated
*/
generate
if (RV32M) begin
  assign multdiv_en_sel     = MULT_TYPE == 0 ? mult_en_i | div_en_i : div_en_i;
  assign multdiv_en         = mult_en_i | div_en_i;
```

```
end else begin
 assign multdiv_en_sel     = 1'b0;
 assign multdiv_en         = 1'b0;
end
endgenerate

 assign regfile_wdata_ex_o = multdiv_en ? multdiv_result : alu_result;

 // branch handling
 assign branch_decision_o  = alu_cmp_result;
 assign jump_target_o      = alu_adder_result_ex_o;

 ////////////////////////////
 //     _    _    _   _    //
 //    / \  | |  | | | |   //
 //   / _ \ | |  | | | |   //
 //  / ___ \| |__| |_| |   //
 // /_/   \_\_____\___/    //
 //                        //
 ////////////////////////////

 zeroriscy_alu alu_i
 (
  .operator_i         ( alu_operator_i           ),
  .operand_a_i        ( alu_operand_a_i          ),
  .operand_b_i        ( alu_operand_b_i          ),
  .multdiv_operand_a_i ( multdiv_alu_operand_a     ),
  .multdiv_operand_b_i ( multdiv_alu_operand_b     ),
  .multdiv_en_i       ( multdiv_en_sel           ),
  .adder_result_o     ( alu_adder_result_ex_o    ),
  .adder_result_ext_o  ( alu_adder_result_ext     ),
  .result_o           ( alu_result               ),
  .comparison_result_o ( alu_cmp_result           ),
  .is_equal_result_o   ( alu_is_equal_result       )
 );

 ////////////////////////////////////////////////////////////////
 //  __  __ _   _ _   _____ ___ ____  _    ___ _____ ____     //
 // |  \/  | | | | | |_   _|_ _|  _ \| |   |_ _| ____|  _ \    //
 // | |\/| | | | | |   | |  | || |_) | |    | ||  _| | |_) |   //
 // | |  | | |_| | |___| |  | ||  __/| |___ | || |___|  _ <    //
 // |_|  |_|\___/|_____|_| |___|_|   |_____|___|_____|_| \_\   //
 //                                                            //
 ////////////////////////////////////////////////////////////////

 generate
 if (MULT_TYPE == 0) begin : multdiv_slow
   zeroriscy_multdiv_slow multdiv_i
   (
    .clk              ( clk                    ),
    .rst_n            ( rst_n                  ),
    .mult_en_i        ( mult_en_i              ),
    .div_en_i         ( div_en_i               ),
    .operator_i       ( multdiv_operator_i     ),
    .signed_mode_i    ( multdiv_signed_mode_i ),
    .op_a_i           ( multdiv_operand_a_i    ),
    .op_b_i           ( multdiv_operand_b_i    ),
```

```
    .alu_adder_ext_i    ( alu_adder_result_ext  ),
    .alu_adder_i        ( alu_adder_result_ex_o ),
    .equal_to_zero      ( alu_is_equal_result   ),
    .ready_o            ( multdiv_ready         ),
    .alu_operand_a_o    ( multdiv_alu_operand_a ),
    .alu_operand_b_o    ( multdiv_alu_operand_b ),
    .multdiv_result_o   ( multdiv_result        )
   );
  end else begin: multdiv_fast
   zeroriscy_multdiv_fast multdiv_i
    (
    .clk                ( clk                   ),
    .rst_n              ( rst_n                 ),
    .mult_en_i          ( mult_en_i             ),
    .div_en_i           ( div_en_i              ),
    .operator_i         ( multdiv_operator_i    ),
    .signed_mode_i      ( multdiv_signed_mode_i ),
    .op_a_i             ( multdiv_operand_a_i   ),
    .op_b_i             ( multdiv_operand_b_i   ),
    .alu_operand_a_o    ( multdiv_alu_operand_a ),
    .alu_operand_b_o    ( multdiv_alu_operand_b ),
    .alu_adder_ext_i    ( alu_adder_result_ext  ),
    .alu_adder_i        ( alu_adder_result_ex_o ),
    .equal_to_zero      ( alu_is_equal_result   ),
    .ready_o            ( multdiv_ready         ),
    .multdiv_result_o   ( multdiv_result        )
   );
  end
  endgenerate

  always_comb
  begin
     unique case (1'b1)
       multdiv_en:
         ex_ready_o = multdiv_ready;
       lsu_en_i:
         ex_ready_o = lsu_ready_ex_i;
       default:
         //1 Cycle case
         ex_ready_o = 1'b1;
     endcase
  end

Endmodule

`include "zeroriscy_config.sv"

// input port: send address one cycle before the data
// clear_i clears the FIFO for the following cycle. in_addr_i can be sent in
// this cycle already
module zeroriscy_fetch_fifo
(
   input  logic       clk,
   input  logic       rst_n,

   // control signals
   input  logic       clear_i,         // clears the contents of the fifo
```

```
  // input port
  input  logic [31:0] in_addr_i,
  input  logic [31:0] in_rdata_i,
  input  logic       in_valid_i,
  output logic       in_ready_o,


  // output port
  output logic       out_valid_o,
  input  logic       out_ready_i,
  output logic [31:0] out_rdata_o,
  output logic [31:0] out_addr_o,

  output logic       out_valid_stored_o // same as out_valid_o, except that if something is incoming
now it is not included. This signal is available immediately as it comes directly out of FFs
 );

 localparam DEPTH = 3; // must be 3 or greater

 // index 0 is used for output
 logic [DEPTH-1:0] [31:0]  addr_n,   addr_int,   addr_Q;
 logic [DEPTH-1:0] [31:0]  rdata_n,  rdata_int,  rdata_Q;
 logic [DEPTH-1:0]         valid_n,  valid_int,  valid_Q;

 logic          [31:0]  addr_next;
 logic          [31:0]  rdata, rdata_unaligned;
 logic                  valid, valid_unaligned;

 logic                  aligned_is_compressed, unaligned_is_compressed;
 logic                  aligned_is_compressed_st, unaligned_is_compressed_st;

 ////////////////////////////////////////////////////////////////////////////////
 // output port
 ////////////////////////////////////////////////////////////////////////////////


 assign rdata = (valid_Q[0]) ? rdata_Q[0] : in_rdata_i;
 assign valid = valid_Q[0] || in_valid_i;

 assign rdata_unaligned = (valid_Q[1]) ? {rdata_Q[1][15:0], rdata[31:16]} : {in_rdata_i[15:0],
rdata[31:16]};
 // it is implied that rdata_valid_Q[0] is set
 assign valid_unaligned = (valid_Q[1] || (valid_Q[0] && in_valid_i));

 assign unaligned_is_compressed    = rdata[17:16] != 2'b11;
 assign aligned_is_compressed      = rdata[1:0] != 2'b11;
 assign unaligned_is_compressed_st = rdata_Q[0][17:16] != 2'b11;
 assign aligned_is_compressed_st   = rdata_Q[0][1:0] != 2'b11;

 ////////////////////////////////////////////////////////////////////////////////
 // instruction aligner (if unaligned)
 ////////////////////////////////////////////////////////////////////////////////

 always_comb
 begin
  // serve the aligned case even though the output address is unaligned when
```

```
  // the next instruction will be from a hardware loop target
  // in this case the current instruction is already prealigned in element 0
  if (out_addr_o[1]) begin
    // unaligned case
    out_rdata_o = rdata_unaligned;

    if (unaligned_is_compressed)
      out_valid_o = valid;
    else
      out_valid_o = valid_unaligned;
  end else begin
    // aligned case
    out_rdata_o = rdata;
    out_valid_o = valid;
  end
end

assign out_addr_o    = (valid_Q[0]) ? addr_Q[0] : in_addr_i;

// this valid signal must not depend on signals from outside!
always_comb
begin
  out_valid_stored_o = 1'b1;

  if (out_addr_o[1]) begin
    if (unaligned_is_compressed_st)
      out_valid_stored_o = 1'b1;
    else
      out_valid_stored_o = valid_Q[1];
  end else begin
    out_valid_stored_o = valid_Q[0];
  end
end


////////////////////////////////////////////////////////////////////////////////
// input port
////////////////////////////////////////////////////////////////////////////////

// we accept data as long as our fifo is not full
// we don't care about clear here as the data will be received one cycle
// later anyway
assign in_ready_o = ~valid_Q[DEPTH-2];


////////////////////////////////////////////////////////////////////////////////
// FIFO management
////////////////////////////////////////////////////////////////////////////////

always_comb
begin
  int j;

            addr_int    = addr_Q;
  rdata_int   = rdata_Q;
  valid_int   = valid_Q;
```

```
  if (in_valid_i) begin
    for(j = 0; j < DEPTH; j++) begin
      if (~valid_Q[j]) begin
        addr_int[j]  = in_addr_i;
        rdata_int[j] = in_rdata_i;
        valid_int[j] = 1'b1;

        break;
      end
    end

  end
end

assign addr_next = {addr_int[0][31:2], 2'b00} + 32'h4;

// move everything by one step
always_comb
begin
  addr_n     = addr_int;
  rdata_n    = rdata_int;
  valid_n    = valid_int;

  if (out_ready_i && out_valid_o) begin
    begin
      if (addr_int[0][1]) begin
        // unaligned case
        if (unaligned_is_compressed) begin
          addr_n[0] = {addr_next[31:2], 2'b00};
        end else begin
          addr_n[0] = {addr_next[31:2], 2'b10};
        end

        rdata_n  = {32'b0, rdata_int[DEPTH-1:1]};
        valid_n  = {1'b0,  valid_int[DEPTH-1:1]};
      end else begin

        if (aligned_is_compressed) begin
          // just increase address, do not move to next entry in FIFO
          addr_n[0] = {addr_int[0][31:2], 2'b10};
        end else begin
          // move to next entry in FIFO
          addr_n[0] = {addr_next[31:2], 2'b00};
          rdata_n   = {32'b0, rdata_int[DEPTH-1:1]};
          valid_n   = {1'b0,  valid_int[DEPTH-1:1]};
        end
      end
    end
  end
end

////////////////////////////////////////////////////////////////////////////////
// registers
////////////////////////////////////////////////////////////////////////////////

always_ff @(posedge clk, negedge rst_n)
```

```
  begin
    if(rst_n == 1'b0)
    begin
      addr_Q    <= '{default: '0};
      rdata_Q   <= '{default: '0};
      valid_Q   <= '0;
    end
    else
    begin
      // on a clear signal from outside we invalidate the content of the FIFO
      // completely and start from an empty state
      if (clear_i) begin
        valid_Q    <= '0;
      end else begin
        addr_Q    <= addr_n;
        rdata_Q   <= rdata_n;
        valid_Q   <= valid_n;
      end
    end
  end

  //-----------------------------------------------------------------------------
  // Assertions
  //-----------------------------------------------------------------------------
`ifndef VERILATOR
  assert property (
    @(posedge clk) (in_valid_i) |-> ((valid_Q[DEPTH-1] == 1'b0) || (clear_i == 1'b1)) );
`endif
Endmodule

`include "zeroriscy_config.sv"

// input port: send address one cycle before the data
// clear_i clears the FIFO for the following cycle. in_addr_i can be sent in
// this cycle already
module zeroriscy_fetch_fifo
(
    input  logic       clk,
    input  logic       rst_n,

    // control signals
    input  logic       clear_i,         // clears the contents of the fifo

    // input port
    input  logic [31:0] in_addr_i,
    input  logic [31:0] in_rdata_i,
    input  logic       in_valid_i,
    output logic       in_ready_o,


    // output port
    output logic       out_valid_o,
    input  logic       out_ready_i,
    output logic [31:0] out_rdata_o,
    output logic [31:0] out_addr_o,
```

```
  output logic       out_valid_stored_o // same as out_valid_o, except that if something is incoming
now it is not included. This signal is available immediately as it comes directly out of FFs
 );

 localparam DEPTH = 3; // must be 3 or greater

 // index 0 is used for output
 logic [DEPTH-1:0] [31:0]  addr_n,    addr_int,    addr_Q;
 logic [DEPTH-1:0] [31:0]  rdata_n,   rdata_int,   rdata_Q;
 logic [DEPTH-1:0]         valid_n,   valid_int,   valid_Q;

 logic           [31:0]  addr_next;
 logic           [31:0]  rdata, rdata_unaligned;
 logic                   valid, valid_unaligned;

 logic                   aligned_is_compressed, unaligned_is_compressed;
 logic                   aligned_is_compressed_st, unaligned_is_compressed_st;

 ////////////////////////////////////////////////////////////////////////////
 // output port
 ////////////////////////////////////////////////////////////////////////////


 assign rdata = (valid_Q[0]) ? rdata_Q[0] : in_rdata_i;
 assign valid = valid_Q[0] || in_valid_i;

 assign rdata_unaligned = (valid_Q[1]) ? {rdata_Q[1][15:0], rdata[31:16]} : {in_rdata_i[15:0],
rdata[31:16]};
 // it is implied that rdata_valid_Q[0] is set
 assign valid_unaligned = (valid_Q[1] || (valid_Q[0] && in_valid_i));

 assign unaligned_is_compressed    = rdata[17:16] != 2'b11;
 assign aligned_is_compressed      = rdata[1:0] != 2'b11;
 assign unaligned_is_compressed_st = rdata_Q[0][17:16] != 2'b11;
 assign aligned_is_compressed_st   = rdata_Q[0][1:0] != 2'b11;

 ////////////////////////////////////////////////////////////////////////////
 // instruction aligner (if unaligned)
 ////////////////////////////////////////////////////////////////////////////

 always_comb
 begin
  // serve the aligned case even though the output address is unaligned when
  // the next instruction will be from a hardware loop target
  // in this case the current instruction is already prealigned in element 0
  if (out_addr_o[1]) begin
    // unaligned case
    out_rdata_o = rdata_unaligned;

    if (unaligned_is_compressed)
      out_valid_o = valid;
    else
      out_valid_o = valid_unaligned;
  end else begin
    // aligned case
    out_rdata_o = rdata;
    out_valid_o = valid;
```

```
  end
end

assign out_addr_o    = (valid_Q[0]) ? addr_Q[0] : in_addr_i;

// this valid signal must not depend on signals from outside!
always_comb
begin
  out_valid_stored_o = 1'b1;

  if (out_addr_o[1]) begin
    if (unaligned_is_compressed_st)
      out_valid_stored_o = 1'b1;
    else
      out_valid_stored_o = valid_Q[1];
  end else begin
    out_valid_stored_o = valid_Q[0];
  end
end


////////////////////////////////////////////////////////////////////////////////
// input port
////////////////////////////////////////////////////////////////////////////////

// we accept data as long as our fifo is not full
// we don't care about clear here as the data will be received one cycle
// later anyway
assign in_ready_o = ~valid_Q[DEPTH-2];


////////////////////////////////////////////////////////////////////////////////
// FIFO management
////////////////////////////////////////////////////////////////////////////////

always_comb
begin
  int j;

            addr_int    = addr_Q;
  rdata_int   = rdata_Q;
  valid_int   = valid_Q;


  if (in_valid_i) begin
    for(j = 0; j < DEPTH; j++) begin
      if (~valid_Q[j]) begin
        addr_int[j]  = in_addr_i;
        rdata_int[j] = in_rdata_i;
        valid_int[j] = 1'b1;

        break;
      end
    end

  end
end
```

```
assign addr_next = {addr_int[0][31:2], 2'b00} + 32'h4;

// move everything by one step
always_comb
begin
  addr_n     = addr_int;
  rdata_n    = rdata_int;
  valid_n    = valid_int;

  if (out_ready_i && out_valid_o) begin
    begin
      if (addr_int[0][1]) begin
        // unaligned case
        if (unaligned_is_compressed) begin
          addr_n[0] = {addr_next[31:2], 2'b00};
        end else begin
          addr_n[0] = {addr_next[31:2], 2'b10};
        end

        rdata_n  = {32'b0, rdata_int[DEPTH-1:1]};
        valid_n  = {1'b0,  valid_int[DEPTH-1:1]};
      end else begin

        if (aligned_is_compressed) begin
          // just increase address, do not move to next entry in FIFO
          addr_n[0] = {addr_int[0][31:2], 2'b10};
        end else begin
          // move to next entry in FIFO
          addr_n[0] = {addr_next[31:2], 2'b00};
          rdata_n   = {32'b0, rdata_int[DEPTH-1:1]};
          valid_n   = {1'b0,  valid_int[DEPTH-1:1]};
        end
      end
    end
  end
end

////////////////////////////////////////////////////////////////////////////////
// registers
////////////////////////////////////////////////////////////////////////////////

always_ff @(posedge clk, negedge rst_n)
begin
  if(rst_n == 1'b0)
  begin
    addr_Q    <= '{default: '0};
    rdata_Q   <= '{default: '0};
    valid_Q   <= '0;
  end
  else
  begin
    // on a clear signal from outside we invalidate the content of the FIFO
    // completely and start from an empty state
    if (clear_i) begin
      valid_Q    <= '0;
    end else begin
```

```
      addr_Q    <= addr_n;
      rdata_Q   <= rdata_n;
      valid_Q   <= valid_n;
    end
  end
 end

 //-----------------------------------------------------------------------------
 // Assertions
 //-----------------------------------------------------------------------------
`ifndef VERILATOR
 assert property (
   @(posedge clk) (in_valid_i) |-> ((valid_Q[DEPTH-1] == 1'b0) || (clear_i == 1'b1)) );
`endif
Endmodule

`include "zeroriscy_config.sv"

import zeroriscy_defines::*;


// Source/Destination register instruction index
`define REG_S1 19:15
`define REG_S2 24:20
`define REG_D  11:07


module zeroriscy_id_stage
#(
 parameter RV32M      = 1,
 parameter RV32E      = 0
)
(
   input  logic       clk,
   input  logic       rst_n,

   input  logic       test_en_i,

   input  logic       fetch_enable_i,
   output logic       ctrl_busy_o,
   output logic       core_ctrl_firstfetch_o,
   output logic       is_decoding_o,

   // Interface to IF stage
   input  logic            instr_valid_i,
   input  logic      [31:0] instr_rdata_i,     // comes from pipeline of IF stage
   output logic            instr_req_o,

   // Jumps and branches
   output logic       branch_in_ex_o,
   input  logic       branch_decision_i,

   // IF and ID stage signals
   output logic       clear_instr_valid_o,
   output logic       pc_set_o,
   output logic [2:0]  pc_mux_o,
   output logic [1:0]  exc_pc_mux_o,
```

```
input  logic       illegal_c_insn_i,
input  logic       is_compressed_i,

input  logic [31:0] pc_id_i,

// Stalls
output logic       halt_if_o,     // controller requests a halt of the IF stage
output logic       id_ready_o,     // ID stage is ready for the next instruction
input  logic       ex_ready_i,
output logic       id_valid_o,     // ID stage is done

// ALU
output logic [ALU_OP_WIDTH-1:0] alu_operator_ex_o,
output logic [31:0] alu_operand_a_ex_o,
output logic [31:0] alu_operand_b_ex_o,

// MUL, DIV
output logic       mult_en_ex_o,
output logic       div_en_ex_o,
output logic  [1:0] multdiv_operator_ex_o,
output logic  [1:0] multdiv_signed_mode_ex_o,
output logic [31:0] multdiv_operand_a_ex_o,
output logic [31:0] multdiv_operand_b_ex_o,

// CSR
output logic       csr_access_ex_o,
output logic [1:0]  csr_op_ex_o,
output logic [5:0]  csr_cause_o,
output logic       csr_save_if_o,
output logic       csr_save_id_o,
output logic       csr_restore_mret_id_o,
output logic       csr_save_cause_o,

// Interface to load store unit
output logic       data_req_ex_o,
output logic       data_we_ex_o,
output logic [1:0]  data_type_ex_o,
output logic       data_sign_ext_ex_o,
output logic [1:0]  data_reg_offset_ex_o,
output logic       data_load_event_ex_o,
output logic [31:0] data_wdata_ex_o,

input  logic       data_misaligned_i,
input  logic [31:0] misaligned_addr_i,

// Interrupt signals
input  logic       irq_i,
input  logic [4:0]  irq_id_i,
input  logic       m_irq_enable_i,
output logic       irq_ack_o,
output logic [4:0]  irq_id_o,
output logic [5:0]  exc_cause_o,

input  logic       lsu_load_err_i,
input  logic       lsu_store_err_i,
```

```
  // Debug Unit Signals
  input  logic [DBG_SETS_W-1:0] dbg_settings_i,
  input  logic       dbg_req_i,
  output logic       dbg_ack_o,
  input  logic       dbg_stall_i,
  output logic       dbg_trap_o,

  input  logic       dbg_reg_rreq_i,
  input  logic [4:0]  dbg_reg_raddr_i,
  output logic [31:0] dbg_reg_rdata_o,

  input  logic       dbg_reg_wreq_i,
  input  logic [4:0]  dbg_reg_waddr_i,
  input  logic [31:0] dbg_reg_wdata_i,

  input  logic       dbg_jump_req_i,

  // Write back signal
  input  logic [31:0] regfile_wdata_lsu_i,
  input  logic [31:0] regfile_wdata_ex_i,
  input  logic [31:0] csr_rdata_i,

  // Performance Counters
  output logic       perf_jump_o,         // we are executing a jump instruction
  output logic       perf_branch_o,       // we are executing a branch instruction
  output logic       perf_tbranch_o       // we are executing a taken branch instruction
);

 logic [31:0] instr;

 // Decoder/Controller ID stage internal signals
 logic       deassert_we;

 logic       illegal_insn_dec;
 logic       illegal_reg_rv32e;
 logic       ebrk_insn;
 logic       mret_insn_dec;
 logic       ecall_insn_dec;
 logic       pipe_flush_dec;

 logic       branch_taken_ex;
 logic       branch_in_id;
 logic       branch_set_n;
 logic       branch_set_q;
 logic       branch_mux_dec;
 logic       jump_set;
 logic       jump_mux_dec;
 logic       jump_in_id;

 logic       instr_multicyle;
 logic       load_stall;
 logic       multdiv_stall;
 logic       branch_stall;
 logic       jump_stall;

 logic       halt_id;
 //FSM signals to write back multi cycles instructions
```

```
logic       regfile_we;
enum logic {RF_LSU, RF_EX} select_data_rf;

// Immediate decoding and sign extension
logic [31:0] imm_i_type;
logic [31:0] imm_iz_type;
logic [31:0] imm_s_type;
logic [31:0] imm_sb_type;
logic [31:0] imm_u_type;
logic [31:0] imm_uj_type;
logic [31:0] imm_z_type;
logic [31:0] imm_s2_type;
logic [31:0] imm_bi_type;
logic [31:0] imm_s3_type;
logic [31:0] imm_vs_type;
logic [31:0] imm_vu_type;

logic [31:0] imm_a;      // contains the immediate for operand b
logic [31:0] imm_b;      // contains the immediate for operand b


// Signals running between controller and exception controller
logic      irq_req_ctrl;
logic [4:0] irq_id_ctrl;
logic      exc_ack, exc_kill;// handshake

// Register file interface
logic [4:0]  regfile_addr_ra_id;
logic [4:0]  regfile_addr_rb_id;

logic [4:0]  regfile_alu_waddr_id;
logic       regfile_we_id;

logic [31:0] regfile_data_ra_id;
logic [31:0] regfile_data_rb_id;

// ALU Control
logic [ALU_OP_WIDTH-1:0] alu_operator;
logic [2:0]  alu_op_a_mux_sel;
logic [2:0]  alu_op_b_mux_sel;

logic [0:0]  imm_a_mux_sel;
logic [3:0]  imm_b_mux_sel;

// Multiplier Control
logic       mult_int_en;     // use integer multiplier
logic       div_int_en;     // use integer division or reminder
logic       multdiv_int_en;
logic [1:0]  multdiv_operator;
logic [1:0]  multdiv_signed_mode;

// Data Memory Control
logic       data_we_id;
logic [1:0]  data_type_id;
logic       data_sign_ext_id;
logic [1:0]  data_reg_offset_id;
logic       data_req_id;
```

```
  logic       data_load_event_id;

  // CSR control
  logic       csr_access;
  logic [1:0]  csr_op;
  logic       csr_status;

  // Forwarding
  logic [1:0]  operand_a_fw_mux_sel;

  logic [31:0] operand_a_fw_id;
  logic [31:0] operand_b_fw_id;

  logic [31:0] operand_b;

  logic [31:0] alu_operand_a;
  logic [31:0] alu_operand_b;

  assign instr = instr_rdata_i;

  // immediate extraction and sign extension
  assign imm_i_type  = { {20 {instr[31]}}, instr[31:20] };
  assign imm_iz_type = {           20'b0, instr[31:20] };
  assign imm_s_type  = { {20 {instr[31]}}, instr[31:25], instr[11:7] };
  assign imm_sb_type = { {19 {instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0 };
  assign imm_u_type  = { instr[31:12], 12'b0 };
  assign imm_uj_type = { {12 {instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0 };

  // immediate for CSR manipulatin (zero extended)
  assign imm_z_type = { 27'b0, instr[`REG_S1] };

  assign imm_s2_type = { 27'b0, instr[24:20] };
  assign imm_bi_type = { {27{instr[24]}}, instr[24:20] };
  assign imm_s3_type = { 27'b0, instr[29:25] };
  assign imm_vs_type = { {26 {instr[24]}}, instr[24:20], instr[25] };
  assign imm_vu_type = { 26'b0, instr[24:20], instr[25] };

  //---------------------------------------------------------------------------
  // source register selection
  //---------------------------------------------------------------------------
  assign regfile_addr_ra_id = instr[`REG_S1];
  assign regfile_addr_rb_id = instr[`REG_S2];

  //---------------------------------------------------------------------------
  // destination registers
  //---------------------------------------------------------------------------
  assign regfile_alu_waddr_id = instr[`REG_D];

//if(RV32E)
//  assign illegal_reg_rv32e = (regfile_addr_ra_id[4] | regfile_addr_rb_id[4] | regfile_alu_waddr_id[4]);
//else
  assign illegal_reg_rv32e = 1'b0;

  // kill instruction in the IF/ID stage by setting the instr_valid_id control
  // signal to 0 for instructions that are done
  assign clear_instr_valid_o = id_ready_o | halt_id;
```

```
assign branch_taken_ex = branch_in_id & branch_decision_i;

/////////////////////////////////////////////////////////
//   ___                          _     _    //
//  / _ \ _ __   ___ _ __ __ _ _ __   __| |    / \    //
// | | | | '_ \/ _ \ '__/ _` | '_ \/ _` |   / _ \   //
// | |_| | | |_) |  __/ | | (_| | | | | (_| |  / ___ \ //
//  \___/| .__/ \___|_|  \__,_|_| |_|\__,_| /_/   \_\//
//       |_|                                //
/////////////////////////////////////////////////////////

// ALU_Op_a Mux
always_comb
begin : alu_operand_a_mux
  case (alu_op_a_mux_sel)
    OP_A_REGA_OR_FWD:  alu_operand_a = operand_a_fw_id;
    OP_A_CURRPC:       alu_operand_a = pc_id_i;
    OP_A_IMM:          alu_operand_a = imm_a;
    default:           alu_operand_a = operand_a_fw_id;
  endcase; // case (alu_op_a_mux_sel)
end

always_comb
  begin : immediate_a_mux
    unique case (imm_a_mux_sel)
      IMMA_Z:      imm_a = imm_z_type;
      IMMA_ZERO:   imm_a = '0;
      default:     imm_a = '0;
    endcase
  end

// Operand a forwarding mux used with LSU instructions
always_comb
  begin : operand_a_fw_mux
    case (operand_a_fw_mux_sel)
      SEL_MISALIGNED:    operand_a_fw_id = misaligned_addr_i;
      SEL_REGFILE:       operand_a_fw_id = regfile_data_ra_id;
      default:           operand_a_fw_id = regfile_data_ra_id;
    endcase; // case (operand_a_fw_mux_sel)
  end


/////////////////////////////////////////////////////////
//   ___                          _   ____   //
//  / _ \ _ __   ___ _ __ __ _ _ __   __| | | __ )  //
// | | | | '_ \/ _ \ '__/ _` | '_ \/ _` | |  _ \ //
// | |_| | | |_) |  __/ | | (_| | | | | (_| | | |_) | //
//  \___/| .__/ \___|_|  \__,_|_| |_|\__,_| |____/  //
//       |_|                                //
/////////////////////////////////////////////////////////

// Immediate Mux for operand B
always_comb
  begin : immediate_b_mux
    unique case (imm_b_mux_sel)
      IMMB_I:      imm_b = imm_i_type;
      IMMB_S:      imm_b = imm_s_type;
```

```
    IMMB_U:      imm_b = imm_u_type;
    IMMB_PCINCR: imm_b = (is_compressed_i && (~data_misaligned_i)) ? 32'h2 : 32'h4;
    IMMB_S2:     imm_b = imm_s2_type;
    IMMB_BI:     imm_b = imm_bi_type;
    IMMB_S3:     imm_b = imm_s3_type;
    IMMB_VS:     imm_b = imm_vs_type;
    IMMB_VU:     imm_b = imm_vu_type;
    IMMB_UJ:     imm_b = imm_uj_type;
    IMMB_SB:     imm_b = imm_sb_type;
    default:     imm_b = imm_i_type;
   endcase
  end

// ALU_Op_b Mux
always_comb
  begin : alu_operand_b_mux
   case (alu_op_b_mux_sel)
    OP_B_REGB_OR_FWD:  operand_b = regfile_data_rb_id;
    OP_B_IMM:          operand_b = imm_b;
    default:           operand_b = regfile_data_rb_id;
   endcase // case (alu_op_b_mux_sel)
  end

assign alu_operand_b   = operand_b;
assign operand_b_fw_id = regfile_data_rb_id;

/////////////////////////////////////////////////////////
//  ____  _____ ____ ___ ____ _____ _____ ____  ____   //
// |  _ \| ____/ ___|_ _/ ___|_   _| ____|  _ \/ ___|  //
// | |_) |  _| | |  _ | |\___ \ | | |  _| | |_) \___ \ //
// |  _ <| |__| |_| || | ___) || | | |___|  _ < ___) | //
// |_| \_\_____\____|___|____/ |_| |_____|_| \_\____/  //
//                                                     //
/////////////////////////////////////////////////////////

logic [31:0] regfile_wdata_mux;
logic        regfile_we_mux;
logic  [4:0] regfile_waddr_mux;
//TODO: add assertion
// Register File mux
always_comb
begin
  if(dbg_reg_wreq_i) begin
   regfile_wdata_mux   = dbg_reg_wdata_i;
   regfile_waddr_mux   = dbg_reg_waddr_i;
   regfile_we_mux      = 1'b1;
  end else begin
   regfile_we_mux      = regfile_we;
   regfile_waddr_mux   = regfile_alu_waddr_id;
   if (select_data_rf == RF_LSU)
    regfile_wdata_mux = regfile_wdata_lsu_i;
   else
    if (csr_access)
     regfile_wdata_mux = csr_rdata_i;
    else
     regfile_wdata_mux = regfile_wdata_ex_i;
  end
```

```
end

zeroriscy_register_file
#(
  .RV32E(RV32E)
)
registers_i
(
  .clk         ( clk              ),
  .rst_n       ( rst_n            ),

  .test_en_i    ( test_en_i         ),

  // Read port a
  .raddr_a_i    ( regfile_addr_ra_id ),
  .rdata_a_o    ( regfile_data_ra_id ),
  // Read port b
  .raddr_b_i    ( (dbg_reg_rreq_i == 1'b0) ? regfile_addr_rb_id : dbg_reg_raddr_i ),
  .rdata_b_o    ( regfile_data_rb_id ),
  // write port
  .waddr_a_i    ( regfile_waddr_mux ),
  .wdata_a_i    ( regfile_wdata_mux ),
  .we_a_i       ( regfile_we_mux    )
);

assign dbg_reg_rdata_o = regfile_data_rb_id;

assign multdiv_int_en  = mult_int_en | div_int_en;

/////////////////////////////////////////////////
//  ____  _____ ____ ___  ____  _____ ____   //
// |  _ \| ____/ ___/ _ \|  _ \| ____|  _ \  //
// | | | |  _|| |  | | | | | | |  _| | |_) | //
// | |_| | |__| |__| |_| | |_| | |___|  _ <  //
// |____/|_____\____\___/|____/|_____|_| \_\ //
//                                           //
/////////////////////////////////////////////////

zeroriscy_decoder
#(
   .RV32M(RV32M)
)
decoder_i
(
  // controller related signals
  .deassert_we_i                ( deassert_we             ),
  .data_misaligned_i              ( data_misaligned_i         ),
  .branch_mux_i                  ( branch_mux_dec           ),
  .jump_mux_i                    ( jump_mux_dec             ),

  .illegal_insn_o                ( illegal_insn_dec         ),
  .ebrk_insn_o                   ( ebrk_insn                ),
  .mret_insn_o                   ( mret_insn_dec            ),
  .ecall_insn_o                  ( ecall_insn_dec           ),
  .pipe_flush_o                  ( pipe_flush_dec           ),

  // from IF/ID pipeline
```

```
  .instr_rdata_i                ( instr                  ),
  .illegal_c_insn_i             ( illegal_c_insn_i       ),

  // ALU signals
  .alu_operator_o               ( alu_operator           ),
  .alu_op_a_mux_sel_o           ( alu_op_a_mux_sel       ),
  .alu_op_b_mux_sel_o           ( alu_op_b_mux_sel       ),

  .imm_a_mux_sel_o              ( imm_a_mux_sel          ),
  .imm_b_mux_sel_o              ( imm_b_mux_sel          ),

  .mult_int_en_o                ( mult_int_en            ),
  .div_int_en_o                 ( div_int_en             ),
  .multdiv_operator_o           ( multdiv_operator       ),
  .multdiv_signed_mode_o        ( multdiv_signed_mode    ),
  // Register file control signals
  .regfile_we_o                 ( regfile_we_id          ),

  // CSR control signals
  .csr_access_o                 ( csr_access             ),
  .csr_op_o                     ( csr_op                 ),
  .csr_status_o                 ( csr_status             ),

  // Data bus interface
  .data_req_o                   ( data_req_id            ),
  .data_we_o                    ( data_we_id             ),
  .data_type_o                  ( data_type_id           ),
  .data_sign_extension_o        ( data_sign_ext_id       ),
  .data_reg_offset_o            ( data_reg_offset_id     ),
  .data_load_event_o            ( data_load_event_id     ),

  // jump/branches
  .jump_in_id_o                 ( jump_in_id             ),
  .branch_in_id_o               ( branch_in_id           )
);

//////////////////////////////////////////////////////////////////////
//   ____ ___  _   _ _____ ____   ___  _    _    _____ ____    //
//  / ___/ _ \| \ | |_   _|  _ \ / _ \| |   | |   | ____|  _ \  //
// | |  | | | |  \| | | | | |_) | | | | |   | |   |  _| | |_) | //
// | |__| |_| | |\  | | | |  _ <| |_| | |___| |___| |___|  _ <  //
//  \____\___/|_| \_| |_| |_| \_\\___/|_____|_____|_____|_| \_\ //
//                                                              //
//////////////////////////////////////////////////////////////////////

zeroriscy_controller controller_i
(
  .clk                      ( clk                    ),
  .rst_n                    ( rst_n                  ),

  .fetch_enable_i           ( fetch_enable_i         ),
  .ctrl_busy_o              ( ctrl_busy_o            ),
  .first_fetch_o            ( core_ctrl_firstfetch_o ),
  .is_decoding_o            ( is_decoding_o          ),

  // decoder related signals
  .deassert_we_o            ( deassert_we            ),
```

```
.illegal_insn_i          ( illegal_insn_dec | illegal_reg_rv32e ),
.ecall_insn_i            ( ecall_insn_dec       ),
.mret_insn_i             ( mret_insn_dec        ),
.pipe_flush_i            ( pipe_flush_dec       ),
.ebrk_insn_i             ( ebrk_insn            ),
.csr_status_i            ( csr_status           ),

// from IF/ID pipeline
.instr_valid_i           ( instr_valid_i        ),

// from prefetcher
.instr_req_o             ( instr_req_o          ),

// to prefetcher
.pc_set_o                ( pc_set_o             ),
.pc_mux_o                ( pc_mux_o             ),
.exc_pc_mux_o            ( exc_pc_mux_o         ),
.exc_cause_o             ( exc_cause_o          ),

// LSU
.data_misaligned_i       ( data_misaligned_i    ),

// jump/branch control
.branch_in_id_i          ( branch_in_id         ),
.branch_taken_ex_i       ( branch_taken_ex      ),
.branch_set_i            ( branch_set_q         ),
.jump_set_i              ( jump_set             ),

.instr_multicyle_i       ( instr_multicyle      ),

// Interrupt Controller Signals
.irq_req_ctrl_i          ( irq_req_ctrl         ),
.irq_id_ctrl_i           ( irq_id_ctrl          ),
.m_IE_i                  ( m_irq_enable_i       ),

.irq_ack_o               ( irq_ack_o            ),
.irq_id_o                ( irq_id_o             ),

.exc_ack_o               ( exc_ack              ),
.exc_kill_o              ( exc_kill             ),

// CSR Controller Signals
.csr_save_cause_o        ( csr_save_cause_o     ),
.csr_cause_o             ( csr_cause_o          ),
.csr_save_if_o           ( csr_save_if_o        ),
.csr_save_id_o           ( csr_save_id_o        ),
.csr_restore_mret_id_o   ( csr_restore_mret_id_o ),

// Debug Unit Signals
.dbg_req_i               ( dbg_req_i            ),
.dbg_ack_o               ( dbg_ack_o            ),
.dbg_stall_i             ( dbg_stall_i          ),
.dbg_jump_req_i          ( dbg_jump_req_i       ),
.dbg_settings_i          ( dbg_settings_i       ),
.dbg_trap_o              ( dbg_trap_o           ),

// Forwarding signals
```

```
  .operand_a_fw_mux_sel_o        ( operand_a_fw_mux_sel   ),

  // Stall signals
  .halt_if_o                     ( halt_if_o              ),
  .halt_id_o                     ( halt_id                ),

  .id_ready_i                    ( id_ready_o             ),

  // Performance Counters
  .perf_jump_o                   ( perf_jump_o            ),
  .perf_tbranch_o                ( perf_tbranch_o         )
);

////////////////////////////////////////////////////////////////////////////////
//  _____     _      _____          _          _ _          //
// |_   _|    | |    /  __ \         | |         | | |         //
//   | | _ __ | |_    | / \/ ___  _ __ | |_ _ __ ___ | | | ___ _ __  //
//   | || '_ \| __|   | |    / _ \| '_ \| __| '__/ _ \| | |/ _ \ '__| //
//  _| || | | | |_ _  | \__/\ (_) | | | | |_| | | (_) | | |  __/ |    //
// \___/_| |_|\__(_)  \____/\___/|_| |_|\__|_|  \___/|_|_|\___|_|    //
//                                                          //
////////////////////////////////////////////////////////////////////////////////

zeroriscy_int_controller int_controller_i
(
  .clk                ( clk              ),
  .rst_n              ( rst_n            ),

  // to controller
  .irq_req_ctrl_o     ( irq_req_ctrl     ),
  .irq_id_ctrl_o      ( irq_id_ctrl      ),

  .ctrl_ack_i         ( exc_ack          ),
  .ctrl_kill_i        ( exc_kill         ),

  // Interrupt signals
  .irq_i              ( irq_i            ),
  .irq_id_i           ( irq_id_i         ),

  .m_IE_i             ( m_irq_enable_i   )
);

////////////////////////////////////////
//   ___ ____       _______ __   //
//  |_ _|  _ \     | ____\ \/ /   //
//   | || | | |_____|  _|  \ /    //
//   | || |_| |_____| |___ / \    //
//  |___|____/     |_____/_/\_\   //
//                          //
////////////////////////////////////////

assign data_we_ex_o              = data_we_id;
assign data_type_ex_o            = data_type_id;
assign data_sign_ext_ex_o        = data_sign_ext_id;
assign data_wdata_ex_o           = regfile_data_rb_id;
assign data_req_ex_o             = data_req_id;
assign data_reg_offset_ex_o      = data_reg_offset_id;
```

```
assign data_load_event_ex_o       = data_load_event_id;

assign alu_operator_ex_o          = alu_operator;
assign alu_operand_a_ex_o         = alu_operand_a;
assign alu_operand_b_ex_o         = alu_operand_b;

assign csr_access_ex_o            = csr_access;
assign csr_op_ex_o                = csr_op;

assign branch_in_ex_o             = branch_in_id;

assign mult_en_ex_o               = mult_int_en;
assign div_en_ex_o                = div_int_en;

assign multdiv_operator_ex_o      = multdiv_operator;
assign multdiv_signed_mode_ex_o   = multdiv_signed_mode;
assign multdiv_operand_a_ex_o     = regfile_data_ra_id;
assign multdiv_operand_b_ex_o     = regfile_data_rb_id;

enum logic { IDLE, WAIT_MULTICYCLE } id_wb_fsm_cs, id_wb_fsm_ns;

//////////////////////////////////////
// ID-EX/WB Pipeline Register       //
//////////////////////////////////////
always_ff @(posedge clk, negedge rst_n)
begin : EX_WB_Pipeline_Register
  if (~rst_n)
  begin
    id_wb_fsm_cs  <= IDLE;
    branch_set_q  <= 1'b0;
  end
  else begin
    id_wb_fsm_cs  <= id_wb_fsm_ns;
    branch_set_q  <= branch_set_n;
  end
end

//////////////////////////////////////
// ID-EX/WB FMS                     //
//////////////////////////////////////

always_comb
begin
  id_wb_fsm_ns    = id_wb_fsm_cs;
  regfile_we      = regfile_we_id;
  load_stall      = 1'b0;
  multdiv_stall   = 1'b0;
  jump_stall      = 1'b0;
  branch_stall    = 1'b0;
  select_data_rf  = RF_EX;
  instr_multicyle = 1'b0;
  branch_set_n    = 1'b0;
  branch_mux_dec  = 1'b0;
  jump_set        = 1'b0;
  jump_mux_dec    = 1'b0;
  perf_branch_o   = 1'b0;
```

```
unique case (id_wb_fsm_cs)

 IDLE:
 begin
  jump_mux_dec   = 1'b1;
  branch_mux_dec = 1'b1;
  unique case (1'b1)
    data_req_id: begin
      //LSU operation
      regfile_we      = 1'b0;
      id_wb_fsm_ns    = WAIT_MULTICYCLE;
      load_stall      = 1'b1;
      instr_multicyle = 1'b1;
    end
    branch_in_id: begin
      //Cond Branch operation
      id_wb_fsm_ns    = branch_decision_i ? WAIT_MULTICYCLE : IDLE;
      branch_stall    = branch_decision_i;
      instr_multicyle = branch_decision_i;
      branch_set_n    = branch_decision_i;
      perf_branch_o   = 1'b1;
    end
    multdiv_int_en: begin
      //MUL or DIV operation
      regfile_we      = 1'b0;
      id_wb_fsm_ns    = WAIT_MULTICYCLE;
      multdiv_stall   = 1'b1;
      instr_multicyle = 1'b1;
    end
    jump_in_id: begin
      //UnCond Branch operation
      regfile_we      = 1'b0;
      id_wb_fsm_ns    = WAIT_MULTICYCLE;
      jump_stall      = 1'b1;
      instr_multicyle = 1'b1;
      jump_set        = 1'b1;
    end
    default:;
  endcase
 end

 WAIT_MULTICYCLE:
 begin
  if(ex_ready_i) begin
    regfile_we     = regfile_we_id;
    id_wb_fsm_ns   = IDLE;
    load_stall     = 1'b0;
    multdiv_stall  = 1'b0;
    select_data_rf = data_req_id ? RF_LSU : RF_EX;
  end else begin
    regfile_we      = 1'b0;
    instr_multicyle = 1'b1;
    unique case (1'b1)
      data_req_id:
        load_stall    = 1'b1;
      multdiv_int_en:
        multdiv_stall = 1'b1;
```

```
          default:;
        endcase
      end
    end

    default:;
  endcase
end

// stall control
assign id_ready_o = (~load_stall) & (~branch_stall) & (~jump_stall) & (~multdiv_stall);

assign id_valid_o = (~halt_id) & id_ready_o;


//----------------------------------------------------------------------------
// Assertions
//----------------------------------------------------------------------------
`ifndef VERILATOR
// make sure that branch decision is valid when jumping
assert property (
  @(posedge clk) (branch_decision_i !== 1'bx || branch_in_id == 1'b0) ) else begin $display("Branch
decision is X"); $stop; end

`ifdef CHECK_MISALIGNED
assert property (
  @(posedge clk) (~data_misaligned_i) ) else $display("Misaligned memory access at %x",pc_id_i);
`endif

// the instruction delivered to the ID stage should always be valid
assert property (
  @(posedge clk) (instr_valid_i & (~illegal_c_insn_i)) |-> (!$isunknown(instr_rdata_i)) ) else
$display("Instruction is valid, but has at least one X");

// make sure multicycles enable signals are unique
assert property (
  @(posedge clk) ~(data_req_ex_o & multdiv_int_en )) else $display("Multicycles enable signals are
not unique");

`endif

Endmodule

`include "zeroriscy_config.sv"

import zeroriscy_defines::*;

module zeroriscy_if_stage
(
    input  logic       clk,
    input  logic       rst_n,
    // the boot address is used to calculate the exception offsets
    input  logic [31:0] boot_addr_i,
    // instruction request control
    input  logic       req_i,
    // instruction cache interface
    output logic                instr_req_o,
```

```
    output logic          [31:0] instr_addr_o,
    input  logic               instr_gnt_i,
    input  logic               instr_rvalid_i,
    input  logic          [31:0] instr_rdata_i,
    // Output of IF Pipeline stage
    output logic            instr_valid_id_o,     // instruction in IF/ID pipeline is valid
    output logic      [31:0] instr_rdata_id_o,     // read instruction is sampled and sent to ID stage for
decoding
    output logic            is_compressed_id_o,   // compressed decoder thinks this is a compressed
instruction
    output logic            illegal_c_insn_id_o,  // compressed decoder thinks this is an invalid
instruction
    output logic      [31:0] pc_if_o,
    output logic      [31:0] pc_id_o,
    // Forwarding ports - control signals
    input  logic       clear_instr_valid_i,  // clear instruction valid bit in IF/ID pipe
    input  logic       pc_set_i,            // set the program counter to a new value
    input  logic [31:0] exception_pc_reg_i,   // address used to restore PC when the
interrupt/exception is served
    input  logic  [2:0] pc_mux_i,            // sel for pc multiplexer
    input  logic  [1:0] exc_pc_mux_i,         // selects ISR address
    input  logic  [4:0] exc_vec_pc_mux_i,     // selects ISR address for vectorized interrupt lines

    // jump and branch target and decision
    input  logic [31:0] jump_target_ex_i,     // jump target address
    // from debug unit
    input  logic [31:0] dbg_jump_addr_i,
    // pipeline stall
    input  logic       halt_if_i,
    input  logic       id_ready_i,
    output logic       if_valid_o,
    // misc signals
    output logic       if_busy_o,           // is the IF stage busy fetching instructions?
    output logic       perf_imiss_o          // Instruction Fetch Miss
  );

    // offset FSM
    enum logic[0:0] {WAIT, IDLE } offset_fsm_cs, offset_fsm_ns;

    logic            valid;
    logic            if_ready;
    // prefetch buffer related signals
    logic            prefetch_busy;
    logic            branch_req;
    logic      [31:0] fetch_addr_n;

    logic            fetch_valid;
    logic            fetch_ready;
    logic      [31:0] fetch_rdata;
    logic      [31:0] fetch_addr;

    logic      [31:0] exc_pc;



    // exception PC selection mux
    always_comb
```

```
    begin : EXC_PC_MUX
      exc_pc = '0;

      unique case (exc_pc_mux_i)
        EXC_PC_ILLINSN: exc_pc = { boot_addr_i[31:8], EXC_OFF_ILLINSN };
        EXC_PC_ECALL:   exc_pc = { boot_addr_i[31:8], EXC_OFF_ECALL   };
        EXC_PC_IRQ:     exc_pc = { boot_addr_i[31:8], 1'b0, exc_vec_pc_mux_i[4:0], 2'b0 };
        // TODO: Add case for EXC_PC_STORE and EXC_PC_LOAD as soon as they are supported
        default:;
      endcase
    end

    // fetch address selection
    always_comb
    begin
      fetch_addr_n = '0;

      unique case (pc_mux_i)
        PC_BOOT:      fetch_addr_n = {boot_addr_i[31:8], EXC_OFF_RST};
        PC_JUMP:      fetch_addr_n = jump_target_ex_i;
        PC_EXCEPTION: fetch_addr_n = exc_pc;            // set PC to exception handler
        PC_ERET:      fetch_addr_n = exception_pc_reg_i; // PC is restored when returning from
IRQ/exception
        PC_DBG_NPC:   fetch_addr_n = dbg_jump_addr_i;    // PC is taken from debug unit

        default:;
      endcase
    end

    // prefetch buffer, caches a fixed number of instructions
    zeroriscy_prefetch_buffer prefetch_buffer_i
      (
        .clk            ( clk                    ),
        .rst_n          ( rst_n                  ),

        .req_i          ( req_i                  ),

        .branch_i       ( branch_req             ),
        .addr_i         ( {fetch_addr_n[31:1], 1'b0}  ),

        .ready_i        ( fetch_ready            ),
        .valid_o        ( fetch_valid            ),
        .rdata_o        ( fetch_rdata            ),
        .addr_o         ( fetch_addr             ),

        // goes to instruction memory / instruction cache
        .instr_req_o      ( instr_req_o            ),
        .instr_addr_o     ( instr_addr_o           ),
        .instr_gnt_i      ( instr_gnt_i            ),
        .instr_rvalid_i   ( instr_rvalid_i         ),
        .instr_rdata_i    ( instr_rdata_i          ),

        // Prefetch Buffer Status
        .busy_o         ( prefetch_busy          )
      );
```

```
// offset FSM state
always_ff @(posedge clk, negedge rst_n)
begin
  if (rst_n == 1'b0) begin
    offset_fsm_cs     <= IDLE;
  end else begin
    offset_fsm_cs     <= offset_fsm_ns;
  end
end

// offset FSM state transition logic
always_comb
begin
  offset_fsm_ns = offset_fsm_cs;

  fetch_ready   = 1'b0;
  branch_req    = 1'b0;
  valid         = 1'b0;

  unique case (offset_fsm_cs)
    // no valid instruction data for ID stage
    // assume aligned
    IDLE: begin
      if (req_i) begin
        branch_req    = 1'b1;
        offset_fsm_ns = WAIT;
      end
    end

    // serving aligned 32 bit or 16 bit instruction, we don't know yet
    WAIT: begin
      if (fetch_valid) begin
        valid   = 1'b1; // an instruction is ready for ID stage

        if (req_i && if_valid_o) begin
          fetch_ready   = 1'b1;
          offset_fsm_ns = WAIT;
        end
      end
    end

    default: begin
      offset_fsm_ns = IDLE;
    end
  endcase


  // take care of jumps and branches
  if (pc_set_i) begin
    valid = 1'b0;

    // switch to new PC from ID stage
    branch_req = 1'b1;
    offset_fsm_ns = WAIT;
  end
end
```

```
assign pc_if_o        = fetch_addr;

assign if_busy_o      = prefetch_busy;

assign perf_imiss_o    = (~fetch_valid) | branch_req;


// compressed instruction decoding, or more precisely compressed instruction
// expander
//
// since it does not matter where we decompress instructions, we do it here
// to ease timing closure
logic [31:0] instr_decompressed;
logic       illegal_c_insn;
logic       instr_compressed_int;

zeroriscy_compressed_decoder compressed_decoder_i
  (
    .instr_i        ( fetch_rdata         ),
    .instr_o        ( instr_decompressed   ),
    .is_compressed_o ( instr_compressed_int ),
    .illegal_instr_o ( illegal_c_insn       )
  );

// IF-ID pipeline registers, frozen when the ID stage is stalled
always_ff @(posedge clk, negedge rst_n)
begin : IF_ID_PIPE_REGISTERS
  if (rst_n == 1'b0)
    begin
      instr_valid_id_o      <= 1'b0;
      instr_rdata_id_o      <= '0;
      illegal_c_insn_id_o   <= 1'b0;
      is_compressed_id_o    <= 1'b0;
      pc_id_o               <= '0;
    end
  else
    begin

      if (if_valid_o)
      begin
         instr_valid_id_o    <= 1'b1;
         instr_rdata_id_o    <= instr_decompressed;
         illegal_c_insn_id_o <= illegal_c_insn;
         is_compressed_id_o  <= instr_compressed_int;
         pc_id_o             <= pc_if_o;
      end else if (clear_instr_valid_i) begin
        instr_valid_id_o    <= 1'b0;
      end

    end
end


assign if_ready = valid & id_ready_i;
```

```
      assign if_valid_o = (~halt_if_i) & if_ready;

      //-----------------------------------------------------------------------------
      // Assertions
      //-----------------------------------------------------------------------------
`ifndef VERILATOR
      // there should never be a grant when there is no request
      assert property (
        @(posedge clk) (instr_gnt_i) |-> (instr_req_o) )
      else $warning("There was a grant without a request");
`endif

      Endmodule

import zeroriscy_defines::*;

module zeroriscy_int_controller
(
  input  logic       clk,
  input  logic       rst_n,

  // irq_req for controller
  output logic       irq_req_ctrl_o,
  output logic  [4:0] irq_id_ctrl_o,

  // handshake signals to controller
  input  logic       ctrl_ack_i,
  input  logic       ctrl_kill_i,

  // external interrupt lines
  input  logic       irq_i,         // level-triggered interrupt inputs
  input  logic  [4:0] irq_id_i,      // interrupt id [0,1,....31]

  input  logic       m_IE_i         // interrupt enable bit from CSR (M mode)
);

  enum logic [1:0] { IDLE, IRQ_PENDING, IRQ_DONE} exc_ctrl_cs, exc_ctrl_ns;

  logic irq_enable_ext;
  logic [4:0] irq_id_q;

  assign irq_enable_ext =  m_IE_i;
  assign irq_req_ctrl_o = exc_ctrl_cs == IRQ_PENDING;
  assign irq_id_ctrl_o  = irq_id_q;

  always_ff @(posedge clk, negedge rst_n)
  begin
    if (rst_n == 1'b0) begin

      irq_id_q    <= '0;
      exc_ctrl_cs <= IDLE;

    end else begin

      unique case (exc_ctrl_cs)

        IDLE:
```

```
      begin
        if(irq_enable_ext & irq_i) begin
          exc_ctrl_cs <= IRQ_PENDING;
          irq_id_q    <= irq_id_i;
        end
      end

      IRQ_PENDING:
      begin
        unique case(1'b1)
          ctrl_ack_i:
            exc_ctrl_cs <= IRQ_DONE;
          ctrl_kill_i:
            exc_ctrl_cs <= IDLE;
          default:
            exc_ctrl_cs <= IRQ_PENDING;
        endcase
      end

      IRQ_DONE:
      begin
        exc_ctrl_cs <= IDLE;
      end

    endcase

  end
 end

`ifndef SYNTHESIS
 // synopsys translate_off
 // evaluate at falling edge to avoid duplicates during glitches
 // Removed this message as it pollutes too much the output and makes tests fail
 //always_ff @(negedge clk)
 //begin
 //  if (rst_n && exc_ctrl_cs == IRQ_DONE)
 //    $display("%t: Entering interrupt service routine. [%m]", $time);
 //end
 // synopsys translate_on
`endif

Endmodule

`include "zeroriscy_config.sv"

module zeroriscy_load_store_unit
(
   input  logic        clk,
   input  logic        rst_n,

   // output to data memory
   output logic        data_req_o,
   input  logic        data_gnt_i,
   input  logic        data_rvalid_i,
   input  logic        data_err_i,

   output logic [31:0]  data_addr_o,
```

```
  output logic        data_we_o,
  output logic [3:0]   data_be_o,
  output logic [31:0]  data_wdata_o,
  input  logic [31:0]  data_rdata_i,

  // signals from ex stage
  input  logic        data_we_ex_i,        // write enable                    -> from ex stage
  input  logic [1:0]   data_type_ex_i,      // Data type word, halfword, byte    -> from ex stage
  input  logic [31:0]  data_wdata_ex_i,     // data to write to memory          -> from ex stage
  input  logic [1:0]   data_reg_offset_ex_i, // offset inside register for stores -> from ex stage
  input  logic        data_sign_ext_ex_i,   // sign extension                  -> from ex stage

  output logic [31:0]  data_rdata_ex_o,     // requested data                  -> to ex stage
  input  logic        data_req_ex_i,       // data request                    -> from ex stage

  input  logic [31:0]  adder_result_ex_i,

  output logic        data_misaligned_o,    // misaligned access was detected    -> to controller
  output logic [31:0]  misaligned_addr_o,

  // exception signals
  output logic        load_err_o,
  output logic        store_err_o,

  // stall signal
  output logic        lsu_update_addr_o, // LSU ready for new data in EX stage
  output logic        data_valid_o,

  output logic        busy_o
);

 logic [31:0]  data_addr_int;

 // registers for data_rdata alignment and sign extension
 logic [1:0]   data_type_q;
 logic [1:0]   rdata_offset_q;
 logic        data_sign_ext_q;
 logic        data_we_q;

 logic [1:0]   wdata_offset;   // mux control for data to be written to memory

 logic [3:0]   data_be;
 logic [31:0]  data_wdata;

 logic        misaligned_st;   // high if we are currently performing the second part of a misaligned store
 logic        data_misaligned, data_misaligned_q;
 logic        increase_address;

 enum logic [2:0]  { IDLE, WAIT_GNT_MIS, WAIT_RVALID_MIS, WAIT_GNT, WAIT_RVALID } CS, NS;

 logic [31:0]  rdata_q;

 ////////////////////////////////// BE generation ////////////////////////////////
 always_comb
 begin
  case (data_type_ex_i) // Data type 00 Word, 01 Half word, 11,10 byte
    2'b00:
```

```
    begin // Writing a word
      if (misaligned_st == 1'b0)
      begin // non-misaligned case
        case (data_addr_int[1:0])
          2'b00: data_be = 4'b1111;
          2'b01: data_be = 4'b1110;
          2'b10: data_be = 4'b1100;
          2'b11: data_be = 4'b1000;
        endcase; // case (data_addr_int[1:0])
      end
      else
      begin // misaligned case
        case (data_addr_int[1:0])
          2'b00: data_be = 4'b0000; // this is not used, but included for completeness
          2'b01: data_be = 4'b0001;
          2'b10: data_be = 4'b0011;
          2'b11: data_be = 4'b0111;
        endcase; // case (data_addr_int[1:0])
      end
    end

    2'b01:
    begin // Writing a half word
      if (misaligned_st == 1'b0)
      begin // non-misaligned case
        case (data_addr_int[1:0])
          2'b00: data_be = 4'b0011;
          2'b01: data_be = 4'b0110;
          2'b10: data_be = 4'b1100;
          2'b11: data_be = 4'b1000;
        endcase; // case (data_addr_int[1:0])
      end
      else
      begin // misaligned case
        data_be = 4'b0001;
      end
    end

    2'b10,
    2'b11: begin // Writing a byte
      case (data_addr_int[1:0])
        2'b00: data_be = 4'b0001;
        2'b01: data_be = 4'b0010;
        2'b10: data_be = 4'b0100;
        2'b11: data_be = 4'b1000;
      endcase; // case (data_addr_int[1:0])
    end
  endcase; // case (data_type_ex_i)
end

// prepare data to be written to the memory
// we handle misaligned accesses, half word and byte accesses and
// register offsets here
assign wdata_offset = data_addr_int[1:0] - data_reg_offset_ex_i[1:0];
always_comb
begin
  case (wdata_offset)
```

```
    2'b00: data_wdata = data_wdata_ex_i[31:0];
    2'b01: data_wdata = {data_wdata_ex_i[23:0], data_wdata_ex_i[31:24]};
    2'b10: data_wdata = {data_wdata_ex_i[15:0], data_wdata_ex_i[31:16]};
    2'b11: data_wdata = {data_wdata_ex_i[ 7:0], data_wdata_ex_i[31: 8]};
  endcase; // case (wdata_offset)
end


// FF for rdata alignment and sign-extension
always_ff @(posedge clk, negedge rst_n)
begin
  if(rst_n == 1'b0)
  begin
    data_type_q     <= '0;
    rdata_offset_q  <= '0;
    data_sign_ext_q <= '0;
    data_we_q       <= 1'b0;
  end
  else if (data_gnt_i == 1'b1) // request was granted, we wait for rvalid and can continue to WB
  begin
    data_type_q     <= data_type_ex_i;
    rdata_offset_q  <= data_addr_int[1:0];
    data_sign_ext_q <= data_sign_ext_ex_i;
    data_we_q       <= data_we_ex_i;
  end
end


////////////////////////////////////////////////////////////////////////////////
//  ____  _            _____     _             _            //
// / ___|(_) __ _ _ __   | ____|_  _| |_ ___ _ __  ___(_) ___  _ __   //
// \___ \| |/ _` | '_ \  |  _| \ \/ / __/ _ \ '_ \/ __| |/ _ \| '_ \  //
//  ___) | | (_| | | | | | |___ >  <| ||  __/ | | \__ \ | (_) | | | | //
// |____/|_|\__, |_| |_| |_____/_/\_\\__\___|_| |_|___/_|\___/|_| |_| //
//         |___/                                          //
////////////////////////////////////////////////////////////////////////////////

logic [31:0] data_rdata_ext;

logic [31:0] rdata_w_ext; // sign extension for words, actually only misaligned assembly
logic [31:0] rdata_h_ext; // sign extension for half words
logic [31:0] rdata_b_ext; // sign extension for bytes

// take care of misaligned words
always_comb
begin
  case (rdata_offset_q)
    2'b00: rdata_w_ext = data_rdata_i[31:0];
    2'b01: rdata_w_ext = {data_rdata_i[ 7:0], rdata_q[31:8]};
    2'b10: rdata_w_ext = {data_rdata_i[15:0], rdata_q[31:16]};
    2'b11: rdata_w_ext = {data_rdata_i[23:0], rdata_q[31:24]};
  endcase
end

// sign extension for half words
always_comb
begin
```

```
  case (rdata_offset_q)
    2'b00:
    begin
      if (data_sign_ext_q == 1'b0)
        rdata_h_ext = {16'h0000, data_rdata_i[15:0]};
      else
        rdata_h_ext = {{16{data_rdata_i[15]}}, data_rdata_i[15:0]};
    end

    2'b01:
    begin
      if (data_sign_ext_q == 1'b0)
        rdata_h_ext = {16'h0000, data_rdata_i[23:8]};
      else
        rdata_h_ext = {{16{data_rdata_i[23]}}, data_rdata_i[23:8]};
    end

    2'b10:
    begin
      if (data_sign_ext_q == 1'b0)
        rdata_h_ext = {16'h0000, data_rdata_i[31:16]};
      else
        rdata_h_ext = {{16{data_rdata_i[31]}}, data_rdata_i[31:16]};
    end

    2'b11:
    begin
      if (data_sign_ext_q == 1'b0)
        rdata_h_ext = {16'h0000, data_rdata_i[7:0], rdata_q[31:24]};
      else
        rdata_h_ext = {{16{data_rdata_i[7]}}, data_rdata_i[7:0], rdata_q[31:24]};
    end
  endcase // case (rdata_offset_q)
end

// sign extension for bytes
always_comb
begin
  case (rdata_offset_q)
    2'b00:
    begin
      if (data_sign_ext_q == 1'b0)
        rdata_b_ext = {24'h00_0000, data_rdata_i[7:0]};
      else
        rdata_b_ext = {{24{data_rdata_i[7]}}, data_rdata_i[7:0]};
    end

    2'b01: begin
      if (data_sign_ext_q == 1'b0)
        rdata_b_ext = {24'h00_0000, data_rdata_i[15:8]};
      else
        rdata_b_ext = {{24{data_rdata_i[15]}}, data_rdata_i[15:8]};
    end

    2'b10:
    begin
      if (data_sign_ext_q == 1'b0)
```

```
      rdata_b_ext = {24'h00_0000, data_rdata_i[23:16]};
     else
      rdata_b_ext = {{24{data_rdata_i[23]}}, data_rdata_i[23:16]};
    end

    2'b11:
    begin
     if (data_sign_ext_q == 1'b0)
      rdata_b_ext = {24'h00_0000, data_rdata_i[31:24]};
     else
      rdata_b_ext = {{24{data_rdata_i[31]}}, data_rdata_i[31:24]};
    end
  endcase // case (rdata_offset_q)
end

// select word, half word or byte sign extended version
always_comb
begin
  case (data_type_q)
    2'b00:      data_rdata_ext = rdata_w_ext;
    2'b01:      data_rdata_ext = rdata_h_ext;
    2'b10,2'b11: data_rdata_ext = rdata_b_ext;
  endcase //~case(rdata_type_q)
end



always_ff @(posedge clk, negedge rst_n)
begin
  if(rst_n == 1'b0)
  begin
    CS          <= IDLE;
    rdata_q      <= '0;
    data_misaligned_q <= '0;
    misaligned_addr_o <= 32'b0;
  end
  else
  begin
    CS          <= NS;
    if (lsu_update_addr_o) begin
      data_misaligned_q <= data_misaligned;
      if(increase_address) begin
        misaligned_addr_o <= data_addr_int;
      end
    end
    if (data_rvalid_i && (~data_we_q))
    begin
      // if we have detected a misaligned access, and we are
      // currently doing the first part of this access, then
      // store the data coming from memory in rdata_q.
      // In all other cases, rdata_q gets the value that we are
      // writing to the register file

      if ((data_misaligned_q == 1'b1) || (data_misaligned == 1'b1))
        rdata_q  <= data_rdata_i;
      else
        rdata_q  <= data_rdata_ext;
```

```
    end
  end
end

// output to register file
assign data_rdata_ex_o = (data_rvalid_i == 1'b1) ? data_rdata_ext : rdata_q;

// output to data interface
assign data_addr_o   = data_addr_int;
assign data_wdata_o  = data_wdata;
assign data_we_o     = data_we_ex_i;
assign data_be_o     = data_be;

assign misaligned_st = data_misaligned_q;

assign load_err_o    = 1'b0;
assign store_err_o   = 1'b0;

// FSM
always_comb
begin
  NS           = CS;

  data_req_o     = 1'b0;

  lsu_update_addr_o   = 1'b0;

  data_valid_o     = 1'b0;
  increase_address = 1'b0;
  data_misaligned_o = 1'b0;

  case(CS)
    // starts from not active and stays in IDLE until request was granted
    IDLE:
    begin
      if (data_req_ex_i) begin
          data_req_o     = data_req_ex_i;
          if(data_gnt_i) begin
            lsu_update_addr_o   = 1'b1;
            increase_address = data_misaligned;
            NS = data_misaligned ? WAIT_RVALID_MIS : WAIT_RVALID;
          end
          else begin
            NS = data_misaligned ? WAIT_GNT_MIS    : WAIT_GNT;
          end
      end
    end //~ IDLE

    WAIT_GNT_MIS:
    begin
      data_req_o = 1'b1;
      if(data_gnt_i) begin
        lsu_update_addr_o = 1'b1;
        increase_address  = data_misaligned;
        NS = WAIT_RVALID_MIS;
      end
    end //~ WAIT_GNT_MIS
```

```
// wait for rvalid in WB stage and send a new request if there is any
WAIT_RVALID_MIS:
begin
  //increase_address goes down, we already have the proper address
  increase_address  = 1'b0;
  //tell the controller to update the address
  data_misaligned_o = 1'b1;
  data_req_o        = 1'b0;
  lsu_update_addr_o = data_gnt_i;

  if(data_rvalid_i) begin
    //if first part rvalid is received
    data_req_o        = 1'b1;
    if(data_gnt_i) begin
      //second grant is received
      NS            = WAIT_RVALID;
      //in this stage we already received the first valid but no the second one
      //it differes from WAIT_RVALID_MIS because we do not send other requests
    end
    else begin
      //second grant is NOT received, but first rvalid yes
      //lsu_update_addr_o is 0 so data_misaligned_q stays high in WAIT_GNT
      //increase address stays the same as well
      NS             = WAIT_GNT; //  [1]
    end
  end
  else begin
    //if first part rvalid is NOT received
    //the second grand is not received either by protocol.
    //stay here
    NS               = WAIT_RVALID_MIS;
  end
end

WAIT_GNT:
begin
  data_misaligned_o = data_misaligned_q;
  //useful in case [1]
  data_req_o        = 1'b1;
  if(data_gnt_i) begin
    lsu_update_addr_o = 1'b1;
    NS = WAIT_RVALID;
  end
end //~ WAIT_GNT

WAIT_RVALID:
begin
  data_req_o        = 1'b0;

  if(data_rvalid_i) begin
    data_valid_o = 1'b1;
    NS           = IDLE;
  end
  else begin
    NS           = WAIT_RVALID;
  end
```

```
    end //~ WAIT_RVALID


    default: begin
      NS = IDLE;
    end
  endcase
end

// check for misaligned accesses that need a second memory access
// If one is detected, this is signaled with data_misaligned_o to
// the controller which selectively stalls the pipeline
always_comb
begin
  data_misaligned = 1'b0;

  if((data_req_ex_i == 1'b1) && (data_misaligned_q == 1'b0))
  begin
    case (data_type_ex_i)
      2'b00: // word
      begin
        if(data_addr_int[1:0] != 2'b00)
          data_misaligned = 1'b1;
      end
      2'b01: // half word
      begin
        if(data_addr_int[1:0] == 2'b11)
          data_misaligned = 1'b1;
      end
                          default: ;
    endcase // case (data_type_ex_i)
  end
end

assign data_addr_int = adder_result_ex_i;

assign busy_o = (CS == WAIT_RVALID) || (data_req_o == 1'b1);

////////////////////////////////////////////////////////////////////////////////
// Assertions
////////////////////////////////////////////////////////////////////////////////
`ifndef VERILATOR
// make sure there is no new request when the old one is not yet completely done
// i.e. it should not be possible to get a grant without an rvalid for the
// last request
assert property (
  @(posedge clk) ((CS == WAIT_RVALID) && (data_gnt_i == 1'b1)) |-> (data_rvalid_i == 1'b1) );

// there should be no rvalid when we are in IDLE
assert property (
  @(posedge clk) (CS == IDLE) |-> (data_rvalid_i == 1'b0) );

// assert that errors are only sent at the same time as grant
assert property ( @(posedge clk) (data_err_i) |-> (data_gnt_i) );

// assert that the address does not contain X when request is sent
assert property ( @(posedge clk) (data_req_o) |-> (!$isunknown(data_addr_o)) );
```

```
`endif
Endmodule

import zeroriscy_defines::*;

`define OP_L 15:0
`define OP_H 31:16

module zeroriscy_multdiv_fast
(
  input  logic       clk,
  input  logic       rst_n,
  input  logic       mult_en_i,
  input  logic       div_en_i,
  input  logic  [1:0] operator_i,
  input  logic  [1:0] signed_mode_i,
  input  logic [31:0] op_a_i,
  input  logic [31:0] op_b_i,
  input  logic [33:0] alu_adder_ext_i,
  input  logic [31:0] alu_adder_i,
  input  logic       equal_to_zero,

  output logic [32:0] alu_operand_a_o,
  output logic [32:0] alu_operand_b_o,

  output logic [31:0] multdiv_result_o,
  output logic       ready_o
);

  logic [ 4:0] div_counter_q, div_counter_n;
  enum logic [1:0] {ALBL, ALBH, AHBL, AHBH } mult_state_q, mult_state_n;
  enum logic [2:0] { MD_IDLE, MD_ABS_A, MD_ABS_B, MD_COMP, MD_LAST, MD_CHANGE_SIGN,
MD_FINISH } divcurr_state_q, divcurr_state_n;

  logic [34:0] mac_res_ext;
  logic [33:0] mac_res_q, mac_res_n, mac_res, op_reminder_n;
  logic [15:0] mult_op_a;
  logic [15:0] mult_op_b;
  logic [33:0] accum;
  logic       sign_a, sign_b;
  logic       div_sign_a, div_sign_b;
  logic       signed_mult;
  logic       is_greater_equal;
  logic       div_change_sign, rem_change_sign;
  logic [31:0] one_shift;
  logic [31:0] op_denominator_q;
  logic [31:0] op_numerator_q;
  logic [31:0] op_quotient_q;
  logic [31:0] op_denominator_n;
  logic [31:0] op_numerator_n;
  logic [31:0] op_quotient_n;
  logic [32:0] next_reminder, next_quotient;
  logic [32:0] res_adder_h;
  logic       mult_is_ready;

  always_ff @(posedge clk or negedge rst_n) begin : proc_mult_state_q
    if(~rst_n) begin
```

```
    mult_state_q      <= ALBL;
    mac_res_q         <= '0;
    div_counter_q     <= '0;
    divcurr_state_q   <= MD_IDLE;
    op_denominator_q  <= '0;
    op_numerator_q    <= '0;
    op_quotient_q     <= '0;
  end else begin

    if(mult_en_i) begin
      mult_state_q <= mult_state_n;
    end

    if(div_en_i) begin
      div_counter_q     <= div_counter_n;
      op_denominator_q  <= op_denominator_n  ;
      op_numerator_q    <= op_numerator_n    ;
      op_quotient_q     <= op_quotient_n     ;
      divcurr_state_q   <= divcurr_state_n;
    end

    unique case(1'b1)
      mult_en_i:
        mac_res_q <= mac_res_n;
      div_en_i:
        mac_res_q <= op_reminder_n;
      default:
        mac_res_q <= mac_res_q;
     endcase

  end
end

assign signed_mult    = (signed_mode_i != 2'b00);

assign multdiv_result_o = div_en_i ? mac_res_q[31:0] : mac_res_n[31:0];

assign mac_res_ext = $signed({sign_a, mult_op_a})*$signed({sign_b, mult_op_b}) + $signed(accum);
assign mac_res     = mac_res_ext[33:0];

assign res_adder_h   = alu_adder_ext_i[33:1];

assign next_reminder = is_greater_equal ? res_adder_h                  : mac_res_q[32:0];
assign next_quotient = is_greater_equal ? op_quotient_q | one_shift     : op_quotient_q;

assign one_shift     = {31'b0, 1'b1} << div_counter_q;

/*
   The adder in the ALU computes alu_operand_a_o + alu_operand_b_o which means
   Reminder - Divisor. If Reminder - Divisor >= 0, is_greater_equal is equal to 1,
   the next Reminder is Reminder - Divisor contained in res_adder_h and the
*/

always_comb
begin
  if ((mac_res_q[31] ^ op_denominator_q[31]) == 0)
    is_greater_equal = (res_adder_h[31] == 0);
```

```
  else
    is_greater_equal = mac_res_q[31];
end

assign div_sign_a   = op_a_i[31] & signed_mode_i[0];
assign div_sign_b   = op_b_i[31] & signed_mode_i[1];
assign div_change_sign  = div_sign_a ^ div_sign_b;
assign rem_change_sign  = div_sign_a;


always_comb
begin : div_fsm
    div_counter_n    = div_counter_q - 1;
    op_reminder_n    = mac_res_q;
    op_quotient_n    = op_quotient_q;
    divcurr_state_n  = divcurr_state_q;
    op_numerator_n   = op_numerator_q;
    op_denominator_n = op_denominator_q;
    alu_operand_a_o  = {32'h0  , 1'b1};
    alu_operand_b_o  = {~op_b_i, 1'b1};

    unique case(divcurr_state_q)
        MD_IDLE: begin
            unique case(operator_i)
              MD_OP_DIV: begin
                //Check if the Denominator is 0
                //quotient for division by 0
                op_reminder_n    = '1;
                divcurr_state_n  = equal_to_zero ? MD_FINISH : MD_ABS_A;
              end
              default: begin
                //Check if the Denominator is 0
                //reminder for division by 0
                op_reminder_n     = {2'b0, op_a_i};
                divcurr_state_n    = equal_to_zero ? MD_FINISH : MD_ABS_A;
              end
            endcase
            //0 - B = 0 iff B == 0
            alu_operand_a_o     = {32'h0  , 1'b1};
            alu_operand_b_o     = {~op_b_i, 1'b1};
            div_counter_n       = 5'd31;
        end

        MD_ABS_A: begin
            //quotient
            op_quotient_n  = '0;
            //A abs value
            op_numerator_n = div_sign_a ? alu_adder_i : op_a_i;
            divcurr_state_n   = MD_ABS_B;
            div_counter_n     = 5'd31;
            //ABS(A) = 0 - A
            alu_operand_a_o     = {32'h0  , 1'b1};
            alu_operand_b_o     = {~op_a_i, 1'b1};
        end

        MD_ABS_B: begin
            //reminder
```

```
        op_reminder_n    = { 33'h0, op_numerator_q[31]};
        //B abs value
        op_denominator_n  = div_sign_b ? alu_adder_i : op_b_i;
        divcurr_state_n   = MD_COMP;
        div_counter_n     = 5'd31;
        //ABS(B) = 0 - B
        alu_operand_a_o     = {32'h0  , 1'b1};
        alu_operand_b_o     = {~op_b_i, 1'b1};
      end

      MD_COMP: begin

       op_reminder_n     = {1'b0, next_reminder[31:0], op_numerator_q[div_counter_n]};
       op_quotient_n     = next_quotient;
       if(div_counter_q == 5'd1)
          divcurr_state_n = MD_LAST;
       else
          divcurr_state_n = MD_COMP;
       //Division
       alu_operand_a_o     = {mac_res_q[31:0], 1'b1}; //it contains the reminder
       alu_operand_b_o     = {~op_denominator_q[31:0], 1'b1}; //denominator negated + 1 to do -
denominator
      end

      MD_LAST: begin

       unique case(operator_i)
        MD_OP_DIV:
          //this time we save the quotient in op_reminder_n (i.e. mac_res_q) since we do not need
anymore the reminder
          op_reminder_n  = {1'b0, next_quotient};
        default:
        //this time we do not save the quotient anymore since we need only the reminder
          op_reminder_n  = {2'b0, next_reminder[31:0]};
       endcase
       //Division
       alu_operand_a_o     = {mac_res_q[31:0], 1'b1}; //it contains the reminder
       alu_operand_b_o     = {~op_denominator_q[31:0], 1'b1}; //denominator negated + 1 to do -
denominator

       divcurr_state_n = MD_CHANGE_SIGN;
      end

      MD_CHANGE_SIGN: begin
        divcurr_state_n  = MD_FINISH;
        unique case(operator_i)
         MD_OP_DIV:
           op_reminder_n = (div_change_sign) ? alu_adder_i : mac_res_q;
         default:
           op_reminder_n = (rem_change_sign) ? alu_adder_i : mac_res_q;
        endcase
        //ABS(Quotient) = 0 - Quotient (or Reminder)
        alu_operand_a_o     = {32'h0  , 1'b1};
        alu_operand_b_o     = {~mac_res_q[31:0], 1'b1};
     end

     MD_FINISH: begin
```

```
        divcurr_state_n = MD_IDLE;
      end

      default:;
    endcase // divcurr_state_q
end

assign ready_o  = mult_is_ready | (divcurr_state_q == MD_FINISH);
always_comb
begin : mult_fsm
    mult_op_a    = op_a_i[`OP_L];
    mult_op_b    = op_b_i[`OP_L];
    sign_a       = 1'b0;
    sign_b       = 1'b0;
    accum        = mac_res_q;
    mac_res_n    = mac_res;
    mult_state_n = mult_state_q;
    mult_is_ready = 1'b0;

    unique case (mult_state_q)

      ALBL: begin
        //al*bl
        mult_op_a = op_a_i[`OP_L];
        mult_op_b = op_b_i[`OP_L];
        sign_a    = 1'b0;
        sign_b    = 1'b0;
        accum     = '0;
        mac_res_n = mac_res;
        mult_state_n = ALBH;
      end

      ALBH: begin
        //al*bh<<16
        mult_op_a = op_a_i[`OP_L];
        mult_op_b = op_b_i[`OP_H];
        sign_a    = 1'b0;
        sign_b    = signed_mode_i[1] & op_b_i[31];
        //result of AL*BL (in mac_res_q) always unsigned with no carry, so carries_q always 00
        accum     = {18'b0,mac_res_q[31:16]};
        unique case(operator_i)
          MD_OP_MULL: begin
            mac_res_n = {2'b0,mac_res[`OP_L],mac_res_q[`OP_L]};
          end
          default: begin
          //MD_OP_MULH
            mac_res_n = mac_res;
          end
        endcase

        mult_state_n = AHBL;
      end

      AHBL: begin
        //ah*bl<<16
        mult_op_a = op_a_i[`OP_H];
        mult_op_b = op_b_i[`OP_L];
```

```
        sign_a    = signed_mode_i[0] & op_a_i[31];
        sign_b    = 1'b0;
        unique case(operator_i)
          MD_OP_MULL: begin
            accum       = {18'b0,mac_res_q[31:16]};
            mac_res_n    = {2'b0,mac_res[15:0],mac_res_q[15:0]};
            mult_is_ready = 1'b1;
            mult_state_n = ALBL;
          end
          default: begin
            accum       = mac_res_q;
            mac_res_n    = mac_res;
            mult_state_n = AHBH;
          end
        endcase
      end

      AHBH: begin
      //only MD_OP_MULH here
        //ah*bh
        mult_op_a = op_a_i[`OP_H];
        mult_op_b = op_b_i[`OP_H];
        sign_a    = signed_mode_i[0] & op_a_i[31];
        sign_b    = signed_mode_i[1] & op_b_i[31];
        accum[17:0 ]  = mac_res_q[33:16];
        accum[33:18]  = {18{signed_mult & mac_res_q[33]}};
        //result of AH*BL is not signed only if signed_mode_i == 2'b00
        mac_res_n    = mac_res;
        mult_state_n = ALBL;
        mult_is_ready = 1'b1;

      end

      default:;
    endcase // mult_state_q
  end


endmodule // zeroriscy_mult

import zeroriscy_defines::*;


module zeroriscy_multdiv_slow
(
  input  logic       clk,
  input  logic       rst_n,
  input  logic       mult_en_i,
  input  logic       div_en_i,
  input  logic  [1:0] operator_i,
  input  logic  [1:0] signed_mode_i,
  input  logic [31:0] op_a_i,
  input  logic [31:0] op_b_i,
  input  logic [33:0] alu_adder_ext_i,
  input  logic [31:0] alu_adder_i,
  input  logic       equal_to_zero,
```

```
  output logic [32:0] alu_operand_a_o,
  output logic [32:0] alu_operand_b_o,
  output logic [31:0] multdiv_result_o,

  output logic       ready_o
);

  logic [ 4:0] multdiv_state_q, multdiv_state_n;
  enum logic [2:0] { MD_IDLE, MD_ABS_A, MD_ABS_B, MD_COMP, MD_LAST, MD_CHANGE_SIGN,
MD_FINISH } curr_state_q;

  logic [32:0] accum_window_q;

  logic [32:0] res_adder_l;
  logic [32:0] res_adder_h;

  logic [32:0] op_b_shift_q;
  logic [32:0] op_a_shift_q;
  logic [32:0] op_a_ext, op_b_ext;
  logic [32:0] one_shift;
  logic [32:0] op_a_bw_pp, op_a_bw_last_pp;
  logic [31:0] b_0;
  logic       sign_a, sign_b;
  logic [32:0] next_reminder, next_quotient;
  logic [32:0] op_remainder;
  logic [31:0] op_numerator_q;
  logic       is_greater_equal;
  logic       div_change_sign, rem_change_sign;


   //(accum_window_q + op_a_shift_q)
  assign res_adder_l      = alu_adder_ext_i[32:0];
   //(accum_window_q + op_a_shift_q)>>1
  assign res_adder_h      = alu_adder_ext_i[33:1];

  always_comb
  begin

    alu_operand_a_o   = accum_window_q;
    multdiv_result_o  = div_en_i ? accum_window_q[31:0] : res_adder_l;

    unique case(operator_i)

      MD_OP_MULL: begin
        alu_operand_b_o   = op_a_bw_pp;
      end

      MD_OP_MULH: begin
        if(curr_state_q == MD_LAST)
          alu_operand_b_o   = op_a_bw_last_pp;
        else
          alu_operand_b_o   = op_a_bw_pp;
      end
      default: begin
        unique case(curr_state_q)
          MD_IDLE: begin
            //0 - B = 0 iff B == 0
```

```
          alu_operand_a_o     = {32'h0  , 1'b1};
          alu_operand_b_o     = {~op_b_i, 1'b1};
        end
        MD_ABS_A: begin
          //ABS(A) = 0 - A
          alu_operand_a_o     = {32'h0  , 1'b1};
          alu_operand_b_o     = {~op_a_i, 1'b1};
        end
        MD_ABS_B: begin
          //ABS(B) = 0 - B
          alu_operand_a_o     = {32'h0  , 1'b1};
          alu_operand_b_o     = {~op_b_i, 1'b1};
        end
        MD_CHANGE_SIGN: begin
          //ABS(Quotient) = 0 - Quotient (or Reminder)
          alu_operand_a_o     = {32'h0  , 1'b1};
          alu_operand_b_o     = {~accum_window_q[31:0], 1'b1};
        end
        default: begin
          //Division
          alu_operand_a_o     = {accum_window_q[31:0], 1'b1}; //it contains the reminder
          alu_operand_b_o     = {~op_b_shift_q[31:0], 1'b1}; //denominator negated + 1 to do -
denominator
        end
      endcase
    end

  endcase

end

/*
  The adder in the ALU computes alu_operand_a_o + alu_operand_b_o which means
  Reminder - Divisor. If Reminder - Divisor >= 0, is_greater_equal is equal to 1,
  the next Reminder is Reminder - Divisor contained in res_adder_h and the
  Quotient multdiv_state_q-th bit is set to 1 using the shift register op_b_shift_q.
  The
*/

always_comb
begin
  if ((accum_window_q[31] ^ op_b_shift_q[31]) == 0)
    is_greater_equal = (res_adder_h[31] == 0);
  else
    is_greater_equal = accum_window_q[31];
end

assign one_shift     = {32'b0, 1'b1} << multdiv_state_q;

assign next_reminder = is_greater_equal ? res_adder_h                  : op_remainder;
assign next_quotient = is_greater_equal ? op_a_shift_q | one_shift      : op_a_shift_q;

assign b_0           = {32{op_b_shift_q[0]}};

//build the partial product
assign op_a_bw_pp      = { ~(op_a_shift_q[32] & op_b_shift_q[0]), op_a_shift_q[31:0] & b_0 };
```

```
assign op_a_bw_last_pp  = { op_a_shift_q[32] & op_b_shift_q[0], ~(op_a_shift_q[31:0] & b_0) };

assign sign_a   = op_a_i[31] & signed_mode_i[0];
assign sign_b   = op_b_i[31] & signed_mode_i[1];

assign op_a_ext = {sign_a, op_a_i};
assign op_b_ext = {sign_b, op_b_i};

//division
assign op_remainder = accum_window_q[32:0];

assign multdiv_state_n     = multdiv_state_q - 1;
assign div_change_sign  = sign_a ^ sign_b;
assign rem_change_sign  = sign_a;

always_ff @(posedge clk or negedge rst_n) begin : proc_multdiv_state_q
  if(~rst_n) begin
    multdiv_state_q     <= '0;
    accum_window_q   <= '0;
    op_b_shift_q     <= '0;
    op_a_shift_q     <= '0;
    curr_state_q     <= MD_IDLE;
    op_numerator_q   <= '0;
  end else begin
    if(mult_en_i | div_en_i) begin
        unique case(curr_state_q)

          MD_IDLE: begin
            unique case(operator_i)
              MD_OP_MULL: begin
                op_a_shift_q   <= op_a_ext << 1;
                accum_window_q <= {  ~(op_a_ext[32] & op_b_i[0]),  op_a_ext[31:0] &
{32{op_b_i[0]}}  };
                op_b_shift_q   <= op_b_ext >> 1;
                curr_state_q   <= MD_COMP;
              end
              MD_OP_MULH: begin
                op_a_shift_q   <= op_a_ext;
                accum_window_q <= { 1'b1, ~(op_a_ext[32] & op_b_i[0]),  op_a_ext[31:1] &
{31{op_b_i[0]}}  };
                op_b_shift_q   <= op_b_ext >> 1;
                curr_state_q   <= MD_COMP;
              end
              MD_OP_DIV: begin
                //Check if the Denominator is 0
                //quotient for division by 0
                accum_window_q <= '1;
                curr_state_q   <= equal_to_zero ? MD_FINISH : MD_ABS_A;
              end
              default: begin
                //Check if the Denominator is 0
                //reminder for division by 0
                accum_window_q <= op_a_ext;
                curr_state_q   <= equal_to_zero ? MD_FINISH : MD_ABS_A;
              end
            endcase
            multdiv_state_q   <= 5'd31;
```

```
end

MD_ABS_A: begin
   //quotient
   op_a_shift_q   <= '0;
   //A abs value
   op_numerator_q <= sign_a ? alu_adder_i : op_a_i;
   curr_state_q   <= MD_ABS_B;
end

MD_ABS_B: begin
   //reminder
   accum_window_q   <= { 32'h0, op_numerator_q[31]};
   //B abs value
   op_b_shift_q     <= sign_b ? alu_adder_i : op_b_i;
   curr_state_q     <= MD_COMP;
end

MD_COMP: begin

   multdiv_state_q   <= multdiv_state_n;

   unique case(operator_i)
     MD_OP_MULL: begin
       accum_window_q <= res_adder_l;
       op_a_shift_q   <= op_a_shift_q << 1;
       op_b_shift_q   <= op_b_shift_q >> 1;
     end
     MD_OP_MULH: begin
       accum_window_q <= res_adder_h;
       op_a_shift_q   <= op_a_shift_q;
       op_b_shift_q   <= op_b_shift_q >> 1;
     end
     default: begin
       accum_window_q <= {next_reminder[31:0], op_numerator_q[multdiv_state_n]};
       op_a_shift_q   <= next_quotient;
     end
   endcase

   if(multdiv_state_q == 5'd1)
       curr_state_q <= MD_LAST;
   else
       curr_state_q <= MD_COMP;
end

MD_LAST: begin

   unique case(operator_i)
     MD_OP_MULL: begin
       accum_window_q <= res_adder_l;
       curr_state_q   <= MD_IDLE;
     end
     MD_OP_MULH: begin
       accum_window_q <= res_adder_l;
       curr_state_q   <= MD_IDLE;
     end
     MD_OP_DIV: begin
```

```
                //this time we save the quotient in accum_window_q since we do not need anymore the
reminder
                accum_window_q <= next_quotient;
                curr_state_q   <= MD_CHANGE_SIGN;
               end
               default: begin
                //this time we do not save the quotient anymore since we need only the reminder
                accum_window_q <= {1'b0, next_reminder[31:0]};
                curr_state_q   <= MD_CHANGE_SIGN;
               end
              endcase
            end

            MD_CHANGE_SIGN: begin
              curr_state_q   <= MD_FINISH;
              unique case(operator_i)
               MD_OP_DIV:
                accum_window_q <= (div_change_sign) ? alu_adder_i : accum_window_q;
               default:
                accum_window_q <= (rem_change_sign) ? alu_adder_i : accum_window_q;
              endcase
           end

            MD_FINISH: begin
              curr_state_q <= MD_IDLE;
            end

            default:;
          endcase // curr_state_q
      end
   end
  end


  assign ready_o      = (curr_state_q == MD_FINISH) | (curr_state_q == MD_LAST & (operator_i ==
MD_OP_MULL | operator_i == MD_OP_MULH));


endmodule // zeroriscy_mult

module zeroriscy_prefetch_buffer
(
  input  logic       clk,
  input  logic       rst_n,

  input  logic       req_i,

  input  logic       branch_i,
  input  logic [31:0] addr_i,


  input  logic       ready_i,
  output logic       valid_o,
  output logic [31:0] rdata_o,
  output logic [31:0] addr_o,
```

```
  // goes to instruction memory / instruction cache
  output logic       instr_req_o,
  input  logic       instr_gnt_i,
  output logic [31:0] instr_addr_o,
  input  logic [31:0] instr_rdata_i,
  input  logic       instr_rvalid_i,

  // Prefetch Buffer Status
  output logic       busy_o
);

  enum logic [1:0] {IDLE, WAIT_GNT, WAIT_RVALID, WAIT_ABORTED } CS, NS;

  logic [31:0] instr_addr_q, fetch_addr;
  logic       addr_valid;

  logic       fifo_valid;
  logic       fifo_ready;
  logic       fifo_clear;

  logic       valid_stored;


  ////////////////////////////////////////////////////////////////////////////////
  // prefetch buffer status
  ////////////////////////////////////////////////////////////////////////////////

  assign busy_o = (CS != IDLE) || instr_req_o;

  ////////////////////////////////////////////////////////////////////////////////
  // fetch fifo
  // consumes addresses and rdata
  ////////////////////////////////////////////////////////////////////////////////

  zeroriscy_fetch_fifo fifo_i
  (
    .clk                ( clk              ),
    .rst_n              ( rst_n            ),

    .clear_i            ( fifo_clear       ),

    .in_addr_i          ( instr_addr_q     ),
    .in_rdata_i         ( instr_rdata_i    ),
    .in_valid_i         ( fifo_valid       ),
    .in_ready_o         ( fifo_ready       ),


    .out_valid_o        ( valid_o          ),
    .out_ready_i        ( ready_i          ),
    .out_rdata_o        ( rdata_o          ),
    .out_addr_o         ( addr_o           ),

    .out_valid_stored_o   ( valid_stored     )
  );


  ////////////////////////////////////////////////////////////////////////////////
```

```
// fetch addr
////////////////////////////////////////////////////////////////////////////////

assign fetch_addr = {instr_addr_q[31:2], 2'b00} + 32'd4;

always_comb
begin
  fifo_clear = branch_i;
end

////////////////////////////////////////////////////////////////////////////////
// instruction fetch FSM
// deals with instruction memory / instruction cache
////////////////////////////////////////////////////////////////////////////////

always_comb
begin
  instr_req_o   = 1'b0;
  instr_addr_o  = fetch_addr;
  fifo_valid    = 1'b0;
  addr_valid    = 1'b0;
  NS            = CS;

  unique case(CS)
    // default state, not waiting for requested data
    IDLE:
    begin
      instr_addr_o = fetch_addr;
      instr_req_o  = 1'b0;

      if (branch_i)
        instr_addr_o = addr_i;

      if (req_i & (fifo_ready | branch_i )) begin
        instr_req_o = 1'b1;
        addr_valid  = 1'b1;


        if(instr_gnt_i) //~>  granted request
          NS = WAIT_RVALID;
        else begin //~> got a request but no grant
          NS = WAIT_GNT;
        end
      end
    end // case: IDLE

    // we sent a request but did not yet get a grant
    WAIT_GNT:
    begin
      instr_addr_o = instr_addr_q;
      instr_req_o  = 1'b1;

      if (branch_i) begin
        instr_addr_o = addr_i;
        addr_valid   = 1'b1;
      end
```

```
  if(instr_gnt_i)
    NS = WAIT_RVALID;
  else
    NS = WAIT_GNT;
end // case: WAIT_GNT

// we wait for rvalid, after that we are ready to serve a new request
WAIT_RVALID: begin
  instr_addr_o = fetch_addr;

  if (branch_i)
    instr_addr_o  = addr_i;


  if (req_i & (fifo_ready | branch_i)) begin
    // prepare for next request

    if (instr_rvalid_i) begin
      instr_req_o = 1'b1;
      fifo_valid  = 1'b1;
      addr_valid  = 1'b1;


      if (instr_gnt_i) begin
        NS = WAIT_RVALID;
      end else begin
        NS = WAIT_GNT;
      end
    end else begin
      // we are requested to abort our current request
      // we didn't get an rvalid yet, so wait for it
      if (branch_i) begin
        addr_valid = 1'b1;
        NS        = WAIT_ABORTED;
      end
    end
  end else begin
    // just wait for rvalid and go back to IDLE, no new request

    if (instr_rvalid_i) begin
      fifo_valid = 1'b1;
      NS        = IDLE;
    end
  end
end // case: WAIT_RVALID

// our last request was aborted, but we didn't yet get a rvalid and
// there was no new request sent yet
// we assume that req_i is set to high
WAIT_ABORTED: begin
  instr_addr_o = instr_addr_q;

  if (branch_i) begin
    instr_addr_o = addr_i;
    addr_valid   = 1'b1;
  end
```

```
      if (instr_rvalid_i) begin
        instr_req_o  = 1'b1;
        // no need to send address, already done in WAIT_RVALID

        if (instr_gnt_i) begin
          NS = WAIT_RVALID;
        end else begin
          NS = WAIT_GNT;
        end
      end
    end

    default:
    begin
      NS         = IDLE;
      instr_req_o = 1'b0;
    end
  endcase
end

////////////////////////////////////////////////////////////////////////////////
// registers
////////////////////////////////////////////////////////////////////////////////

always_ff @(posedge clk, negedge rst_n)
begin
  if(rst_n == 1'b0)
  begin
    CS            <= IDLE;
    instr_addr_q    <= '0;
  end
  else
  begin
    CS            <= NS;

    if (addr_valid) begin
      instr_addr_q    <= instr_addr_o;
    end
  end
end

Endmodule

`include "zeroriscy_config.sv"

module zeroriscy_register_file
#(
  parameter RV32E        = 0,
  parameter DATA_WIDTH    = 32
)
(
  // Clock and Reset
  input  logic                clk,
  input  logic                rst_n,

  input  logic                test_en_i,
```

```
  //Read port R1
  input  logic [4:0]           raddr_a_i,
  output logic [DATA_WIDTH-1:0]  rdata_a_o,

  //Read port R2
  input  logic [4:0]           raddr_b_i,
  output logic [DATA_WIDTH-1:0]  rdata_b_o,


  // Write port W1
  input  logic [4:0]            waddr_a_i,
  input  logic [DATA_WIDTH-1:0]   wdata_a_i,
  input  logic                  we_a_i

);


  localparam    ADDR_WIDTH = RV32E ? 4 : 5;
  localparam    NUM_WORDS  = 2**ADDR_WIDTH;

  logic [DATA_WIDTH-1:0]      mem[NUM_WORDS];

  logic [NUM_WORDS-1:1]       waddr_onehot_a;

  logic [NUM_WORDS-1:1]       mem_clocks;
  logic [DATA_WIDTH-1:0]      wdata_a_q;


  // Write port W1
  logic [ADDR_WIDTH-1:0]     raddr_a_int, raddr_b_int, waddr_a_int;

  assign raddr_a_int = raddr_a_i[ADDR_WIDTH-1:0];
  assign raddr_b_int = raddr_b_i[ADDR_WIDTH-1:0];
  assign waddr_a_int = waddr_a_i[ADDR_WIDTH-1:0];


  logic clk_int;

  int unsigned i;
  int unsigned j;
  int unsigned k;

  genvar x;

  //-----------------------------------------------------------------------------
  //-- READ : Read address decoder RAD
  //-----------------------------------------------------------------------------
  assign rdata_a_o = mem[raddr_a_int];
  assign rdata_b_o = mem[raddr_b_int];

  //-----------------------------------------------------------------------------
  // WRITE : SAMPLE INPUT DATA
  //-----------------------------------------------------------------------------

  cluster_clock_gating CG_WE_GLOBAL
  (
    .clk_i     ( clk            ),
```

```
  .en_i      ( we_a_i         ),
  .test_en_i ( test_en_i      ),
  .clk_o     ( clk_int        )
);

// use clk_int here, since otherwise we don't want to write anything anyway
always_ff @(posedge clk_int, negedge rst_n)
begin : sample_waddr
  if (~rst_n) begin
    wdata_a_q       <= '0;
  end else begin
    if(we_a_i)
      wdata_a_q <= wdata_a_i;
  end
end

//-----------------------------------------------------------------------------
//-- WRITE : Write Address Decoder (WAD), combinatorial process
//-----------------------------------------------------------------------------
always_comb
begin : p_WADa
  for(i = 1; i < NUM_WORDS; i++)
  begin : p_WordItera
    if ( (we_a_i == 1'b1 ) && (waddr_a_int == i) )
      waddr_onehot_a[i] = 1'b1;
    else
      waddr_onehot_a[i] = 1'b0;
  end
end


//-----------------------------------------------------------------------------
//-- WRITE : Clock gating (if integrated clock-gating cells are available)
//-----------------------------------------------------------------------------
generate
  for(x = 1; x < NUM_WORDS; x++)
  begin : CG_CELL_WORD_ITER
    cluster_clock_gating CG_Inst
    (
      .clk_i     ( clk_int                        ),
      .en_i      ( waddr_onehot_a[x]              ),
      .test_en_i ( test_en_i                      ),
      .clk_o     ( mem_clocks[x]                  )
    );
  end
endgenerate

//-----------------------------------------------------------------------------
//-- WRITE : Write operation
//-----------------------------------------------------------------------------
//-- Generate M = WORDS sequential processes, each of which describes one
//-- word of the memory. The processes are synchronized with the clocks
//-- ClocksxC(i), i = 0, 1, ..., M-1
//-- Use active low, i.e. transparent on low latches as storage elements
//-- Data is sampled on rising clock edge

always_latch
```

```
  begin : latch_wdata
    // Note: The assignment has to be done inside this process or Modelsim complains about it
    mem[0] = '0;

    for(k = 1; k < NUM_WORDS; k++)
    begin : w_WordIter
      if(mem_clocks[k] == 1'b1)
        mem[k] = wdata_a_q;
    end
  end


Endmodule

`include "zeroriscy_config.sv"

module zeroriscy_register_file
#(
  parameter RV32E        = 0,
  parameter DATA_WIDTH    = 32
)
(
  // Clock and Reset
  input  logic                clk,
  input  logic                rst_n,

  input  logic                test_en_i,

  //Read port R1
  input  logic [4:0]           raddr_a_i,
  output logic [DATA_WIDTH-1:0]  rdata_a_o,

  //Read port R2
  input  logic [4:0]           raddr_b_i,
  output logic [DATA_WIDTH-1:0]  rdata_b_o,


  // Write port W1
  input  logic [4:0]            waddr_a_i,
  input  logic [DATA_WIDTH-1:0]   wdata_a_i,
  input  logic                  we_a_i

);

  localparam    ADDR_WIDTH = RV32E ? 4 : 5;
  localparam    NUM_WORDS  = 2**ADDR_WIDTH;

  logic [NUM_WORDS-1:0][DATA_WIDTH-1:0] rf_reg;
  logic [NUM_WORDS-1:0][DATA_WIDTH-1:0] rf_reg_tmp;
  logic [NUM_WORDS-1:0]               we_a_dec;

  always_comb
  begin : we_a_decoder
    for (int i = 0; i < NUM_WORDS; i++) begin
      if (waddr_a_i == i)
        we_a_dec[i] = we_a_i;
      else
```

```
      we_a_dec[i] = 1'b0;
    end
  end


  genvar i;
  generate

    // loop from 1 to NUM_WORDS-1 as R0 is nil
    for (i = 1; i < NUM_WORDS; i++)
    begin : rf_gen

      always_ff @(posedge clk, negedge rst_n)
      begin : register_write_behavioral
        if (rst_n==1'b0) begin
          rf_reg_tmp[i] <= 'b0;
        end else begin
          if (we_a_dec[i])
            rf_reg_tmp[i] <= wdata_a_i;
        end
      end

    end

    // R0 is nil
    assign rf_reg[0] = '0;
    assign rf_reg[NUM_WORDS-1:1] = rf_reg_tmp[NUM_WORDS-1:1];

  endgenerate

  assign rdata_a_o = rf_reg[raddr_a_i];
  assign rdata_b_o = rf_reg[raddr_b_i];

Endmodule

`ifndef VERILATOR


`include "zeroriscy_config.sv"

import zeroriscy_defines::*;
import zeroriscy_tracer_defines::*;


// Source/Destination register instruction index
`define REG_S1 19:15
`define REG_S2 24:20
`define REG_S3 29:25
`define REG_D  11:07


module zeroriscy_tracer
#(
    parameter REG_ADDR_WIDTH      = 5
)
(
  // Clock and Reset
```

```
  input  logic       clk,
  input  logic       rst_n,

  input  logic       fetch_enable,
  input  logic [3:0]  core_id,
  input  logic [5:0]  cluster_id,

  input  logic [31:0] pc,
  input  logic [31:0] instr,
  input  logic       compressed,
  input  logic       id_valid,
  input  logic       is_decoding,
  input  logic       is_branch,
  input  logic       branch_taken,
  input  logic       pipe_flush,
  input  logic       mret_insn,
  input  logic       ecall_insn,
  input  logic       ebrk_insn,
  input  logic       csr_status,
  input  logic [31:0] rs1_value,
  input  logic [31:0] rs2_value,
  input  logic [31:0] lsu_value,

  input  logic [(REG_ADDR_WIDTH-1):0] ex_reg_addr,
  input  logic       ex_reg_we,
  input  logic [31:0] ex_reg_wdata,
  input  logic       data_valid_lsu,
  input  logic       ex_data_req,
  input  logic       ex_data_gnt,
  input  logic       ex_data_we,
  input  logic [31:0] ex_data_addr,
  input  logic [31:0] ex_data_wdata,

  input  logic [31:0] lsu_reg_wdata,

  input  logic [31:0] imm_u_type,
  input  logic [31:0] imm_uj_type,
  input  logic [31:0] imm_i_type,
  input  logic [11:0] imm_iz_type,
  input  logic [31:0] imm_z_type,
  input  logic [31:0] imm_s_type,
  input  logic [31:0] imm_sb_type
);

  integer     f;
  string      fn;
  integer     cycles;
  logic [ 4:0] rd, rs1, rs2, rs3;

  typedef struct {
    logic [(REG_ADDR_WIDTH-1):0] addr;
    logic [31:0] value;
  } reg_t;

  typedef struct {
    logic [31:0] addr;
    logic       we;
```

```
  logic [ 3:0] be;
  logic [31:0] wdata;
  logic [31:0] rdata;
} mem_acc_t;

class instr_trace_t;
  time        simtime;
  integer      cycles;
  logic [31:0] pc;
  logic [31:0] instr;
  string       str;
  reg_t        regs_read[$];
  reg_t        regs_write[$];
  mem_acc_t    mem_access[$];

  function new ();
    str       = "";
    regs_read  = {};
    regs_write = {};
    mem_access = {};
  endfunction

  function string regAddrToStr(input logic [(REG_ADDR_WIDTH-1):0] addr);
    begin
      if (addr < 10)
        return $sformatf(" x%0d", addr);
      else
        return $sformatf("x%0d", addr);
    end
  endfunction

  function void printInstrTrace();
    mem_acc_t mem_acc;
    begin
      $fwrite(f, "%t %15d %h %h %-36s", simtime,
                                 cycles,
                                 pc,
                                 instr,
                                 str);

      foreach(regs_write[i]) begin
        if (regs_write[i].addr != 0)
          $fwrite(f, " %s=%08x", regAddrToStr(regs_write[i].addr), regs_write[i].value);
      end

      foreach(regs_read[i]) begin
        if (regs_read[i].addr != 0)
          $fwrite(f, " %s:%08x", regAddrToStr(regs_read[i].addr), regs_read[i].value);
      end

      if (mem_access.size() > 0) begin
        mem_acc = mem_access.pop_front();

        $fwrite(f, "  PA:%08x", mem_acc.addr);
      end

      $fwrite(f, "\n");
```

```
  end
endfunction

function void printMnemonic(input string mnemonic);
  begin
    str = mnemonic;
  end
endfunction // printMnemonic

function void printRInstr(input string mnemonic);
  begin
    regs_read.push_back('{rs1, rs1_value});
    regs_read.push_back('{rs2, rs2_value});
    regs_write.push_back('{rd, 'x});
    str = $sformatf("%-16s x%0d, x%0d, x%0d", mnemonic, rd, rs1, rs2);
  end
endfunction // printRInstr

function void printIInstr(input string mnemonic);
  begin
    regs_read.push_back('{rs1, rs1_value});
    regs_write.push_back('{rd, 'x});
    str = $sformatf("%-16s x%0d, x%0d, %0d", mnemonic, rd, rs1, $signed(imm_i_type));
  end
endfunction // printIInstr

function void printIuInstr(input string mnemonic);
  begin
    regs_read.push_back('{rs1, rs1_value});
    regs_write.push_back('{rd, 'x});
    str = $sformatf("%-16s x%0d, x%0d, 0x%0x", mnemonic, rd, rs1, imm_i_type);
  end
endfunction // printIuInstr

function void printUInstr(input string mnemonic);
  begin
    regs_write.push_back('{rd, 'x});
    str = $sformatf("%-16s x%0d, 0x%0h", mnemonic, rd, {imm_u_type[31:12], 12'h000});
  end
endfunction // printUInstr

function void printUJInstr(input string mnemonic);
  begin
    regs_write.push_back('{rd, 'x});
    str =  $sformatf("%-16s x%0d, %0d", mnemonic, rd, $signed(imm_uj_type));
  end
endfunction // printUJInstr

function void printSBInstr(input string mnemonic);
  begin
    regs_read.push_back('{rs1, rs1_value});
    regs_read.push_back('{rs2, rs2_value});
    str =  $sformatf("%-16s x%0d, x%0d, %0d", mnemonic, rs1, rs2, $signed(imm_sb_type));
  end
endfunction // printSBInstr

function void printCSRInstr(input string mnemonic);
```

```
  logic [11:0] csr;
  begin
    csr = instr[31:20];

    regs_write.push_back('{rd, 'x});

    if (instr[14] == 1'b0) begin
      regs_read.push_back('{rs1, rs1_value});
      str = $sformatf("%-16s x%0d, x%0d, 0x%h", mnemonic, rd, rs1, csr);
    end else begin
      str = $sformatf("%-16s x%0d, 0x%h, 0x%h", mnemonic, rd, imm_z_type, csr);
    end
  end
endfunction // printCSRInstr

function void printLoadInstr();
  string mnemonic;
  logic [2:0] size;
  begin
    // detect reg-reg load and find size
    size = instr[14:12];
    if (instr[14:12] == 3'b111)
      size = instr[30:28];

    case (size)
      3'b000: mnemonic = "lb";
      3'b001: mnemonic = "lh";
      3'b010: mnemonic = "lw";
      3'b100: mnemonic = "lbu";
      3'b101: mnemonic = "lhu";
      3'b110: mnemonic = "p.elw";
      3'b011,
      3'b111: begin
        printMnemonic("INVALID");
        return;
      end
    endcase

    regs_write.push_back('{rd, 'x});

    if (instr[14:12] != 3'b111) begin
      // regular load
        regs_read.push_back('{rs1, rs1_value});
        str = $sformatf("%-16s x%0d, %0d(x%0d)", mnemonic, rd, $signed(imm_i_type), rs1);
    end else begin
        printMnemonic("INVALID");
    end
  end
endfunction

function void printStoreInstr();
  string mnemonic;
  begin

    case (instr[13:12])
      2'b00:  mnemonic = "sb";
      2'b01:  mnemonic = "sh";
```

```
      2'b10:  mnemonic = "sw";
      2'b11: begin
        printMnemonic("INVALID");
        return;
      end
    endcase

    if (instr[14] == 1'b0) begin
      // regular store
        regs_read.push_back('{rs2, rs2_value});
        regs_read.push_back('{rs1, rs1_value});
        str = $sformatf("%-16s x%0d, %0d(x%0d)", mnemonic, rs2, $signed(imm_s_type), rs1);
    end else begin
        printMnemonic("INVALID");
    end
  end
 endfunction // printSInstr

endclass

mailbox #(instr_trace_t) instr_ex = new ();
mailbox #(instr_trace_t) instr_wb = new ();

// cycle counter
always_ff @(posedge clk, negedge rst_n)
begin
 if (rst_n == 1'b0)
  cycles = 0;
 else
  cycles = cycles + 1;
end

// open/close output file for writing
initial
begin
 wait(rst_n == 1'b1);
 wait(fetch_enable == 1'b1);
 $sformat(fn, "trace_core_%h_%h.log", cluster_id, core_id);
 $display("[TRACER] Output filename is: %s", fn);
 f = $fopen(fn, "w");
 $fwrite(f, "            Time        Cycles PC      Instr    Mnemonic\n");

end

final
begin
 $fclose(f);
end

assign rd  = instr[`REG_D];
assign rs1 = instr[`REG_S1];
assign rs2 = instr[`REG_S2];
assign rs3 = instr[`REG_S3];

// log execution
always @(negedge clk)
begin
```

```
   instr_trace_t trace;
   mem_acc_t     mem_acc;
   // special case for WFI because we don't wait for unstalling there
   if ( (id_valid || mret_insn || ecall_insn || pipe_flush || ebrk_insn || csr_status || ex_data_req) &&
is_decoding)
   begin
     trace = new ();

     trace.simtime    = $time;
     trace.cycles     = cycles;
     trace.pc         = pc;
     trace.instr      = instr;

     // use casex instead of case inside due to ModelSim bug
     casex (instr)
       // Aliases
       32'h00_00_00_13:   trace.printMnemonic("nop");
       // Regular opcodes
       INSTR_LUI:       trace.printUInstr("lui");
       INSTR_AUIPC:     trace.printUInstr("auipc");
       INSTR_JAL:       trace.printUJInstr("jal");
       INSTR_JALR:      trace.printIInstr("jalr");
       // BRANCH
       INSTR_BEQ:       trace.printSBInstr("beq");
       INSTR_BNE:       trace.printSBInstr("bne");
       INSTR_BLT:       trace.printSBInstr("blt");
       INSTR_BGE:       trace.printSBInstr("bge");
       INSTR_BLTU:      trace.printSBInstr("bltu");
       INSTR_BGEU:      trace.printSBInstr("bgeu");
       // OPIMM
       INSTR_ADDI:      trace.printIInstr("addi");
       INSTR_SLTI:      trace.printIInstr("slti");
       INSTR_SLTIU:     trace.printIInstr("sltiu");
       INSTR_XORI:      trace.printIInstr("xori");
       INSTR_ORI:       trace.printIInstr("ori");
       INSTR_ANDI:      trace.printIInstr("andi");
       INSTR_SLLI:      trace.printIuInstr("slli");
       INSTR_SRLI:      trace.printIuInstr("srli");
       INSTR_SRAI:      trace.printIuInstr("srai");
       // OP
       INSTR_ADD:       trace.printRInstr("add");
       INSTR_SUB:       trace.printRInstr("sub");
       INSTR_SLL:       trace.printRInstr("sll");
       INSTR_SLT:       trace.printRInstr("slt");
       INSTR_SLTU:      trace.printRInstr("sltu");
       INSTR_XOR:       trace.printRInstr("xor");
       INSTR_SRL:       trace.printRInstr("srl");
       INSTR_SRA:       trace.printRInstr("sra");
       INSTR_OR:        trace.printRInstr("or");
       INSTR_AND:       trace.printRInstr("and");
       // SYSTEM (CSR manipulation)
       INSTR_CSRRW:     trace.printCSRInstr("csrrw");
       INSTR_CSRRS:     trace.printCSRInstr("csrrs");
       INSTR_CSRRC:     trace.printCSRInstr("csrrc");
       INSTR_CSRRWI:    trace.printCSRInstr("csrrwi");
       INSTR_CSRRSI:    trace.printCSRInstr("csrrsi");
       INSTR_CSRRCI:    trace.printCSRInstr("csrrci");
```

```
      // SYSTEM (others)
      INSTR_ECALL:      trace.printMnemonic("ecall");
      INSTR_EBREAK:     trace.printMnemonic("ebreak");
      INSTR_MRET:       trace.printMnemonic("mret");
      INSTR_WFI:        trace.printMnemonic("wfi");
      // RV32M
      INSTR_PMUL:       trace.printRInstr("mul");
      INSTR_PMUH:       trace.printRInstr("mulh");
      INSTR_PMULHSU:    trace.printRInstr("mulhsu");
      INSTR_PMULHU:     trace.printRInstr("mulhu");
      INSTR_DIV:        trace.printRInstr("div");
      INSTR_DIVU:       trace.printRInstr("divu");
      INSTR_REM:        trace.printRInstr("rem");
      INSTR_REMU:       trace.printRInstr("remu");
      {25'b?, OPCODE_LOAD}:       trace.printLoadInstr();
      {25'b?, OPCODE_STORE}:      trace.printStoreInstr();
      default:          trace.printMnemonic("INVALID");
    endcase // unique case (instr)

      // replace register written back
      foreach(trace.regs_write[i]) begin
       if ((trace.regs_write[i].addr == ex_reg_addr) && ex_reg_we)
         trace.regs_write[i].value = ex_reg_wdata;
      end
      // look for data accesses and log them
      if (ex_data_req) begin

        if(~ex_data_gnt) begin
          //we wait until the the gnt comes
          do @(negedge clk);
          while (!ex_data_gnt);
        end

        mem_acc.addr = ex_data_addr;
        mem_acc.we   = ex_data_we;

        if (mem_acc.we)
          mem_acc.wdata = ex_data_wdata;
        else
          mem_acc.wdata = 'x;
        //we wait until the the data instruction ends
        do @(negedge clk);
        while (!data_valid_lsu);

        if (~mem_acc.we)
          //load operations
          foreach(trace.regs_write[i])
              trace.regs_write[i].value = lsu_reg_wdata;
        trace.mem_access.push_back(mem_acc);
      end
    trace.printInstrTrace();

   end
  end // always @ (posedge clk)

endmodule
`endif
```